

Renamer

The complete developer & user guide

Every file, every class, every function and every variable explained line by line in simple English, with UML diagrams.

For everyday users, for contributors and for curious developers.

Project: Renamer v1.0.0 · Python 3.10+ · PySide6 · Pillow · pygame

Author of the project: Mohamed Yassin Khmiri · License: MIT

Guide generated October 2026 from the project source (main branch snapshot).

Contents

How to read this guide	4
Colour legend used in this guide	4
1.1 What problem does it solve?	5
1.2 The four things it can do	5
1.3 Installing	5
1.4 A tour of the window	5
1.5 Step-by-step: renaming your pictures	7
1.6 The three rename styles with examples	8
1.7 Supported images	8
1.8 Our safety promises	8
1.9 Sound and logs	9
1.10 Frequently asked questions	9
1.11 Glossary (simple words)	10
2.1 The project in numbers	11
2.2 Folder and file map	12
2.3 The layered architecture	13
2.4 Threads: why the window never freezes	13
2.5 UML class diagram	15
2.6 Start-up flow	17
2.7 The user interface layout (widget tree)	18
2.8 The five visual states	19
2.9 End-to-end sequences	20
2.10 What happens to ONE file (in-place mode)	23
2.11 Data that travels between parts	24
3.1 main.py — the front door	25
3.2 The three `__init__.py` files	28
3.3 core/renamer.py — deciding the new names	28
3.4 core/backup_manager.py — the safety net	31
3.5 core/metadata.py — lossless cleaning of JPEG and PNG	37
3.6 core/audio.py — optional music and sound cues	45
3.7 workers/image_worker.py — the batch engine	48
3.8 ui/main_window.py — the whole window	57
3.9 The other files	71
4.1 Atomic file replacement	74

4.2 Exclusive creation ('O_EXCL')	74
4.3 Naming and collision avoidance	74
4.4 Backup, verification and rollback	76
4.5 Cancellation and closing	76
4.6 Error handling strategy	76
4.7 Logging	76
4.8 Lazy and optional audio	77
4.9 Complete life of one run (summary)	77
5.1 Index of all functions and methods	78
5.2 All constants and magic numbers	79
5.3 Quirks and observations found while reading the code	80
5.4 How to extend the project	80
5.5 Packaging for Windows	81
5.6 Quick troubleshooting for developers	81
5.7 Credits and licence	82

How to read this guide

This guide has **five parts**. You do not need to read it in order. Pick the part that matches who you are.

Part	For whom	What you will find
1. Meet Renamer	Everyone	What the program does, how to install it, a tour of the window, examples of every rename style, safety promises, FAQ and glossary.
2. The big picture	Curious people and developers	How the code is organised, the layers, the threads, the data flow and all the UML diagrams (class, sequence, state, flow).
3. File by file	Developers	Each source file explained block by block: the real code with line numbers, then a plain-English explanation, plus tables of every variable, constant and function.
4. Mechanisms deep dive	Developers	How the clever safety tricks work: atomic writes, backups, rollback, cancellation, collision avoidance, lossless metadata removal.
5. Reference	Developers	Indexes of all functions, signals, constants and magic numbers, a list of quirks found while reading the code, how to extend the project, and packaging.

Colour legend used in this guide

Box	Meaning
Blue note	Extra background that is nice to know.
Green tip	A practical hint for users.
Amber warning	Something that can surprise you or a limitation of the program.
Purple developer box	A deeper technical remark for programmers.

About the code listings

Every listing shows the **real source code** of the project with its real line numbers (the number on the left). A line that is too long for the page is wrapped and marked with the symbol `⋮`. The explanation below each listing says what the lines do in simple words.

PART I

Meet Renamer

Renamer is a small Windows desktop program that gives your pictures clean, tidy names with one click. It is written in Python with the Qt toolkit (called PySide6) and it has a cartoon-laboratory theme. This part is written for normal users: no programming knowledge is needed.

1.1 What problem does it solve?

Cameras and phones give photos names such as `IMG_8841.JPG`, `DSC00023.jpg` or `WhatsApp Image 2024-... .jpeg`. Such names are hard to sort and hard to search. Renamer takes **all images in one folder** and gives them names such as `Image_001.jpg`, `Holiday_001.jpg` or `2024-07-15_14-32-09_001.jpg`. It can also wipe hidden personal information (like GPS position) from the pictures, and it protects your originals with backups.

1.2 The four things it can do

Mode	What happens	Originals
Rename only	Files are renamed directly inside the folder.	Changed (a backup is kept)
Rename + clean details	Renamed and hidden data (EXIF, GPS, camera info, comments) is removed from JPEG and PNG files.	Changed (a backup is kept)
Rename + ZIP	Renamed copies are put into a new ZIP file.	Never touched
Rename + clean + ZIP	Copies are renamed, cleaned and zipped. (ZIP mode always cleans.)	Never touched

1.3 Installing

- **Easiest way:** download `Renamer-Setup.exe` from the project's GitHub *Releases* page and run it. It installs for your user only (no administrator rights), adds a desktop icon and a Start-menu entry and registers a normal uninstaller. Works on Windows 10 and 11 (64-bit).
- **From source (for developers):** install Python 3.10 or newer, then in the project folder run `python -m pip install -r requirements.txt` and `python main.py`.

1.4 A tour of the window

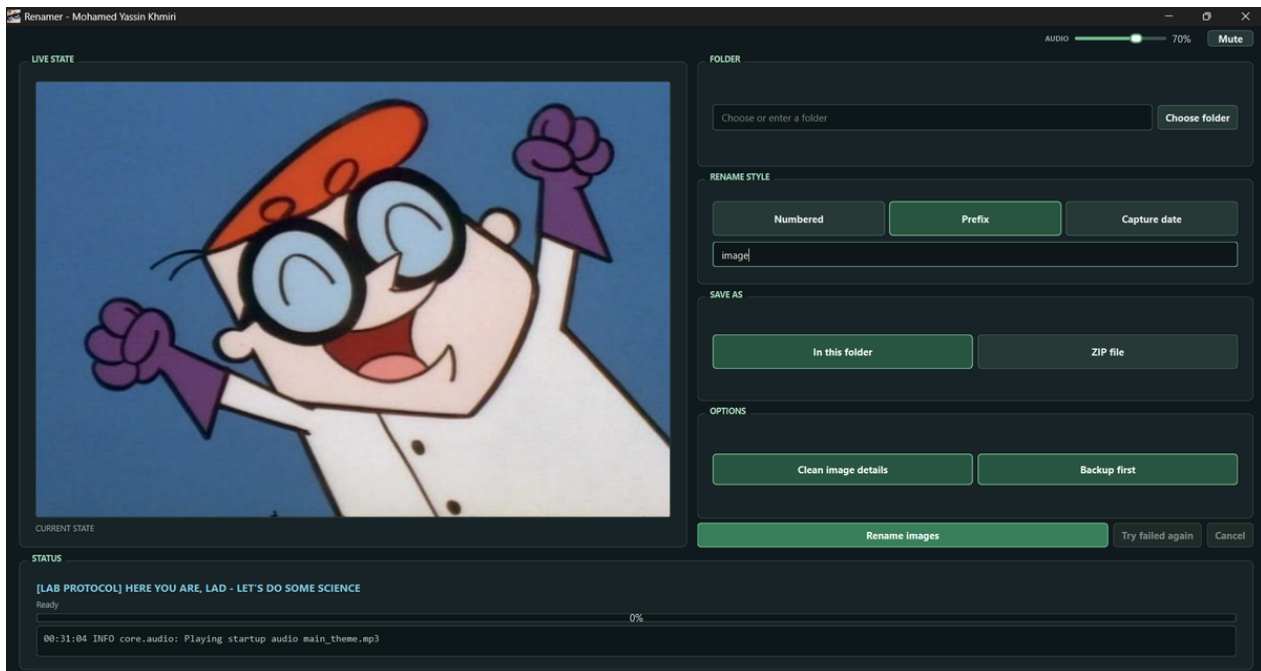


Figure 1. The main window. Left: the live picture for the current state. Right: your controls. Bottom: status and log.

Area	What it is for
AUDIO (top right)	A volume slider (0–100 %, starts at 70 %) and a <i>Mute</i> button for the background music.
LIVE STATE (left)	A cartoon picture that changes with what the program is doing (ready, working, caution, error, success).
FOLDER	Type a folder path or click <i>Choose folder</i> . The big action button stays grey until the folder really exists.
RENAME STYLE	<i>Numbered</i> , <i>Prefix</i> (a text box appears for your word) or <i>Capture date</i> .
SAVE AS	<i>In this folder</i> (change originals) or <i>ZIP file</i> (make renamed copies in a ZIP).
OPTIONS	<i>Clean image details</i> (remove hidden data) and <i>Backup first</i> . Both switch off and lock in ZIP mode.
Buttons	<i>Rename images</i> (or <i>Create ZIP</i>), <i>Try failed again</i> , <i>Cancel</i> .
STATUS (bottom)	A coloured banner for the state, a one-line message, a progress bar and a one-line event log.

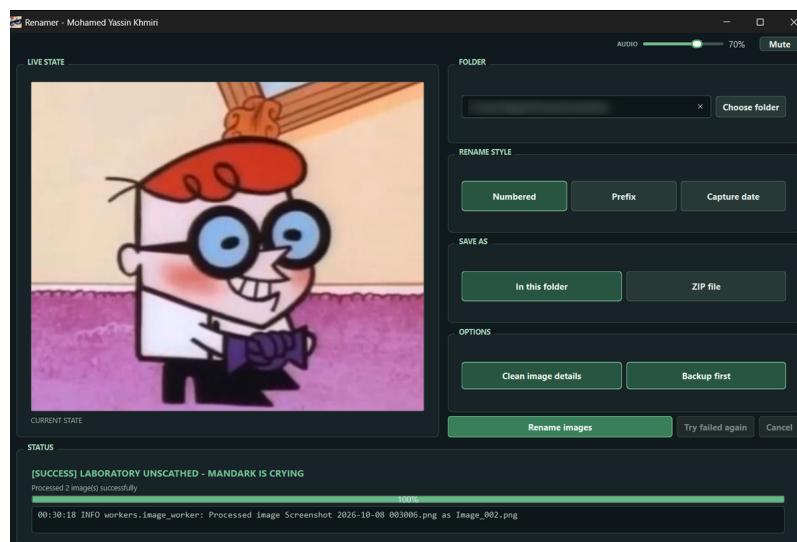


Figure 2. After a successful batch.

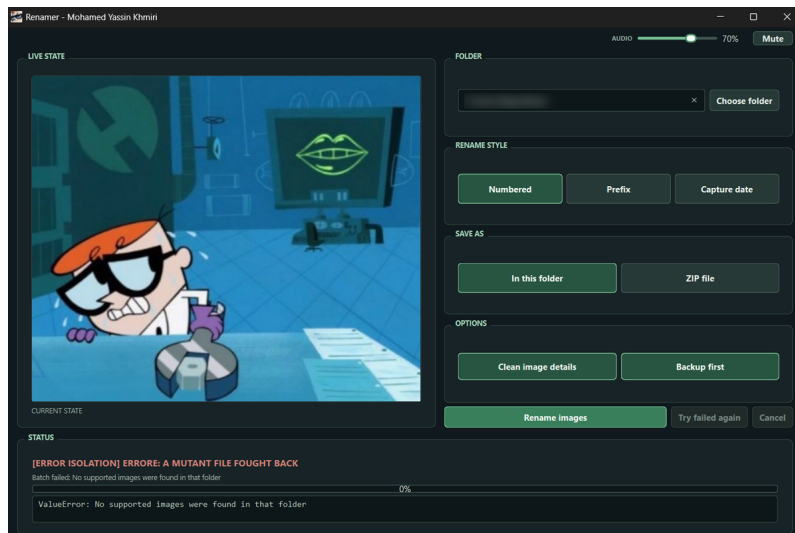


Figure 3. A clear error when the folder has no supported images.

1.5 Step-by-step: renaming your pictures

- Click **Choose folder** and pick the folder that holds your pictures (Figure 4 shows the standard Windows picker).
- Choose a **rename style** (see 1.6).
- Choose **In this folder** or **ZIP file**.
- Leave **Clean image details** and **Backup first** switched on if you are unsure. They are on by default.
- Press **Rename images** (or **Create ZIP**). In ZIP mode a Save dialog asks where to put the ZIP.
- Watch the progress bar. You can press **Cancel** at any time: everything is restored.
- If some files failed, press **Try failed again** to retry only those.

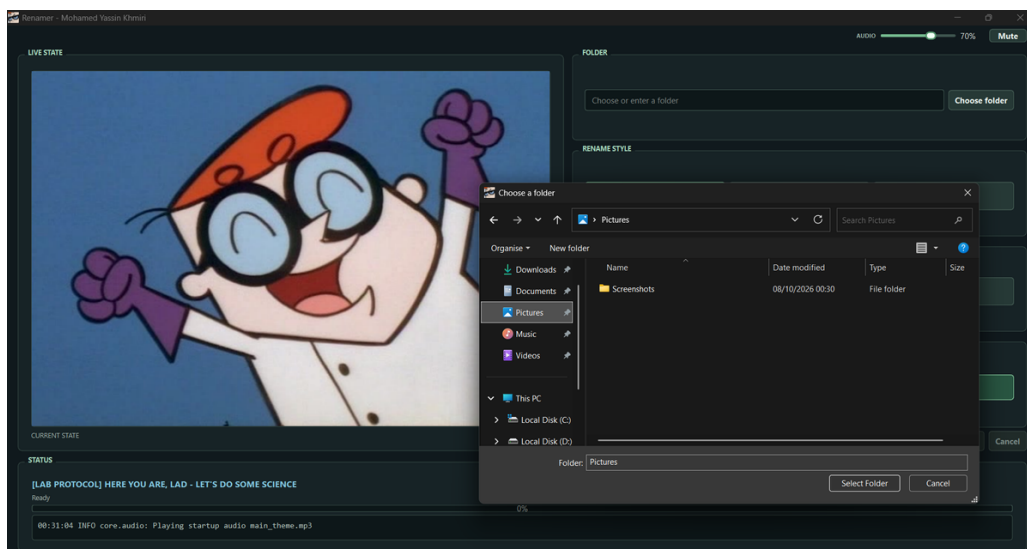


Figure 4. Choosing a folder with the standard Windows picker.

1.6 The three rename styles with examples

Imagine a folder with three photos called `beach.JPG`, `cat.png` and `dog.jpeg`. After sorting by name (ignoring capital letters) they get numbers 1, 2, 3. The number always has **at least 3 digits** (`001`) and gets longer only for very big batches (1000 files gives `0001`).

Style	You type	Result
Numbered	nothing	<code>Image_001.jpg</code> , <code>Image_002.png</code> , <code>Image_003.jpeg</code>
Prefix	<code>Holiday</code>	<code>Holiday_001.jpg</code> , <code>Holiday_002.png</code> , <code>Holiday_003.jpeg</code>
Capture date	nothing	<code>2024-07-15_14-32-09_001.jpg</code> ... (the time the photo was taken)

Extensions

The file **extension** is **kept** but written in lowercase (`.JPG` becomes `.jpg`). Pictures are never converted to another format.

Capture-date details

- Renamer first looks inside the picture for the camera's *EXIF* timestamp.
- If that is missing, it uses the file's creation time on your computer (or the last-modified time on systems without creation time).
- The number at the end keeps names unique when two photos were taken in the same second.

Prefix rules

- The prefix cannot be empty.
- It cannot contain these characters: `< > : " / \ | ? *` because Windows does not allow them in file names.
- Spaces at the start and end are trimmed.

1.7 Supported images

Renamer looks at **files directly inside the chosen folder** (it does not enter sub-folders) that are real files (not shortcuts/symlinks) and end with: `.jpg .jpeg .png .webp .bmp .tif .tiff .gif`. A wrong extension is not enough: when cleaning is requested, Renamer opens every file and checks it is really an image.

Format	Rename	Hidden-data removal
JPEG	✓	✓ lossless: EXIF, XMP, IPTC, comments removed; colour profile kept
PNG	✓	✓ lossless: text, EXIF, time and size-info chunks removed; colour data kept
WebP, BMP, TIFF, GIF	✓	✗ checked only, then left byte-for-byte unchanged

1.8 Our safety promises

Promise	How it is kept
Your pictures are backed up first	Before changing anything, an exact copy is made and checked with a SHA-256 fingerprint.
Nothing is overwritten by accident	If a name is already taken, a number like <code>_1</code> is added instead.

Promise	How it is kept
Cancel really cancels	Pressing Cancel (or closing the window) restores every original.
One bad file does not stop the rest	A failing file is skipped, logged, and can be retried.
Quality is never reduced	Images are never decoded and re-saved. The picture data is copied byte for byte.
A crash cannot leave half-written files	Every file is written to a temporary name first and swapped in at the end (atomic replace).

Where is my backup?

With *Backup first* on, a ZIP named like `RENAMER_backup_20261010_162803_123456_xxxx.zip` is placed in a hidden-style folder called `.RENAMER_backups` inside your picture folder. With it off, a temporary copy is still made and deleted after a successful run.

1.9 Sound and logs

If the file `assets/audio/main_theme.mp3` exists, it loops quietly in the background. Short cues play when the state changes (a small beep if no sound file is present). Sound is **optional**: if sound fails, the program just continues. A rotating log file is kept for support. Its folder is `RENAMER_logs` inside the Qt application-data folder (on Windows this is under `AppData\Local`, in sub-folders named after the organisation and the app), or `~/RENAMER/RENAMER_logs` if Qt gives no folder.

1.10 Frequently asked questions

Question	Answer
The <i>Rename images</i> button is grey.	The folder box is empty or the path does not exist. Type a valid folder or use <i>Choose folder</i> .
It says <i>No supported images were found</i> .	The folder has no files with the supported extensions (sub-folders are not searched).
Does it work on sub-folders?	No. Only the files directly inside the chosen folder.
Can I undo a rename afterwards?	Yes if <i>Backup first</i> was on: unzip the ZIP from <code>.RENAMER_backups</code> . The ZIP has the original names.
Why are some of my photos not cleaned?	Only JPEG and PNG are cleaned. WebP, BMP, TIFF and GIF are left unchanged on purpose to avoid damaging them.
I clicked Cancel and my files are back with old names.	That is correct: Cancel restores the original files.
Why is Dexter in the program?	It is a fan-made theme. The author notes the characters belong to their owners and the project is non-commercial.
The sound does not play.	pygame may not be installed or no audio device exists. The program still works.

1.11 Glossary (simple words)

Word	Meaning
Metadata / EXIF	Hidden information stored inside a photo: camera model, date, GPS position, settings.
Batch	One run of the program over all the pictures of a folder.
In-place	Changing the files where they already are.
Backup / snapshot	An exact copy made before changes so you can go back.
Rollback	Putting the originals back after a problem or a cancel.
Collision	Two files that would get the same name.
Thread	A separate worker inside the program so the window does not freeze.
Signal / slot	Qt's way for one part to send a message to another part.
Atomic	An action that happens completely or not at all, never half.
SHA-256	A fingerprint of a file; if one byte changes, the fingerprint changes.
Chunk / segment	A labelled block inside a PNG (chunk) or JPEG (segment) file.

PART 2

The big picture

Before diving into single lines of code, this part shows the whole system from above. If you only read one part as a developer, read this one.

2.1 The project in numbers

Item	Value
Language / version	Python 3.10 or newer (uses <code>X</code> <code>None</code> types, <code>slots=True</code> dataclasses)
GUI toolkit	PySide6 6.6 or newer (Qt for Python)
Image library	Pillow 10 or newer (used only to <i>check</i> images and read EXIF dates)
Audio library	pygame 2.5 or newer (optional, loaded lazily)
Source files (.py)	5 real modules + <code>main.py</code> + 3 one-line package markers
Classes	12 (3 data classes, 2 exceptions, 3 UI helper classes, 4 main classes: window, worker, backup manager, audio controller)
Installer	Inno Setup script <code>installer/Renamer.iss</code>
License	MIT, © 2026 Mohamed Yassin Khmiri
Project status (from README)	Finished. The author says no more updates are planned.

2.2 Folder and file map

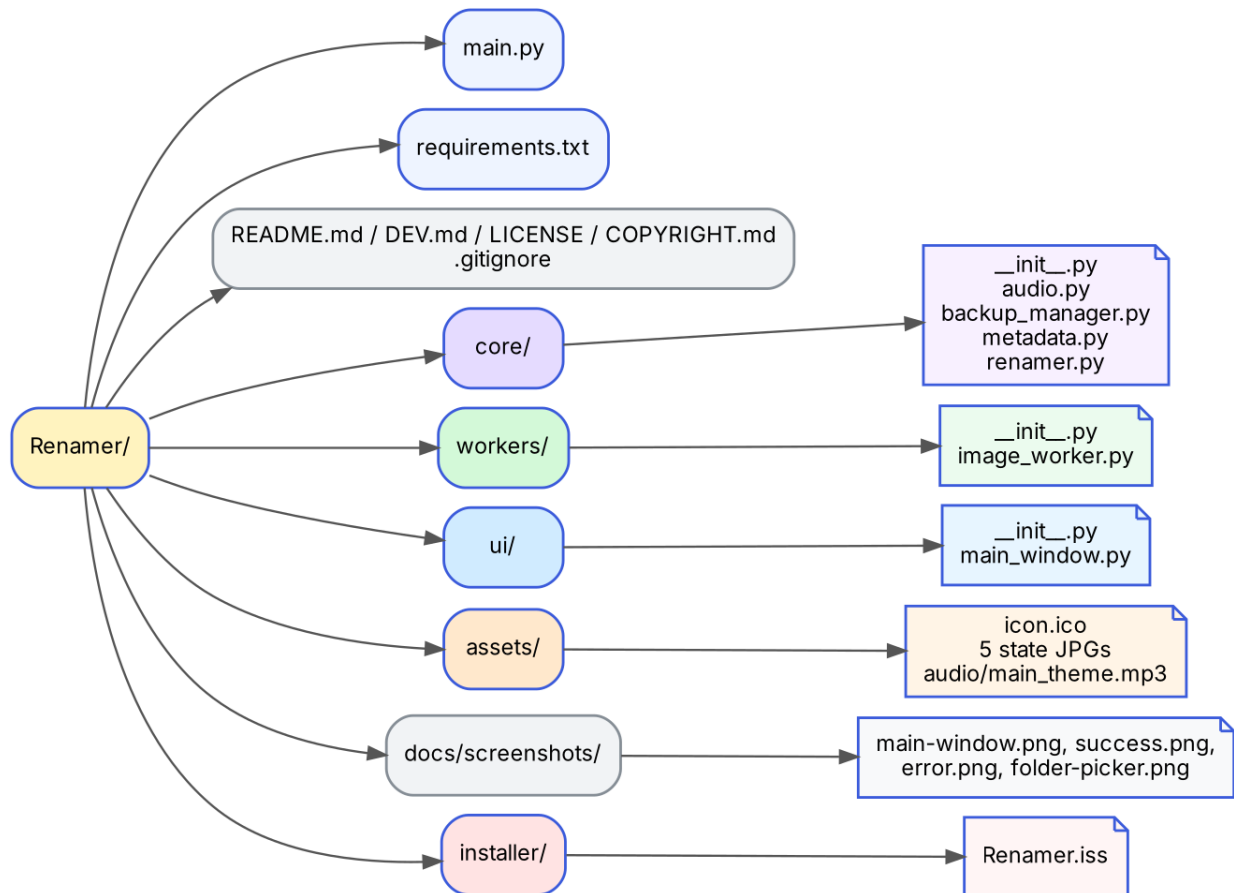


Figure 5. Every file in the project and the layer it belongs to.

2.3 The layered architecture

The code is split in clean layers. A layer may only talk to the layers below it. The **core** layer knows nothing about windows; the **UI** layer knows nothing about how a JPEG is built. This makes each piece easy to understand and to test alone.

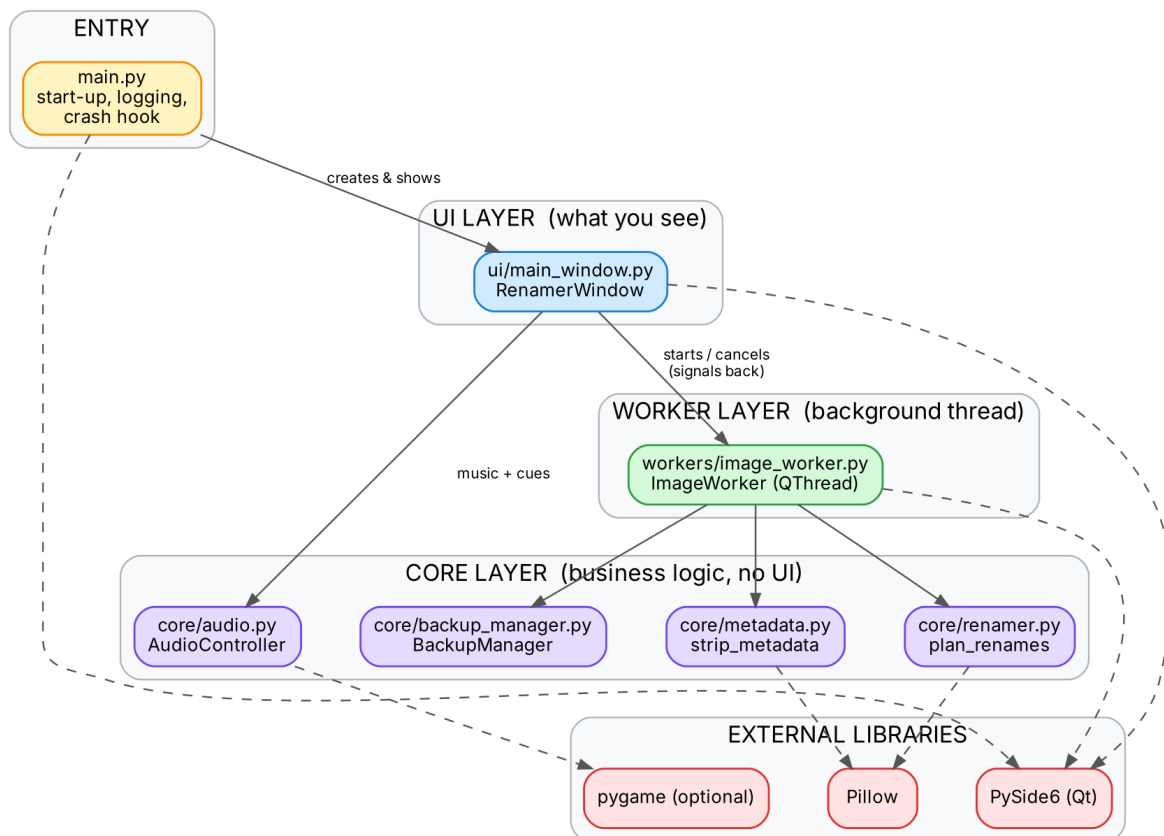


Figure 6. Layer diagram (UML component view). Solid arrows: the code calls the other part. Dashed arrows: uses an external library.

Layer	Files	Job	Knows about Qt?
Entry	<code>main.py</code>	Start the app, set up logs and the crash hook.	Yes (QApplication)
UI	<code>ui/main_window.py</code>	Draw the window, read the user's choices, show progress.	Yes
Worker	<code>workers/image_worker.py</code>	Run the whole batch in a background thread, send messages back.	Yes (QThread, Signal)
Core	<code>core/*.py</code>	Plan names, make backups, clean metadata, play sound.	No (pure Python), except audio which uses pygame

2.4 Threads: why the window never freezes

A graphical program has one special thread called the **GUI thread** (or main thread). It draws the window and reacts to clicks. If it were busy copying 500 photos, the window would freeze. So Renamer starts a second thread, **ImageWorker**, that does all slow work. The two threads never touch each other's objects directly. They talk only through **signals**: small messages that Qt delivers safely from one thread to the other.

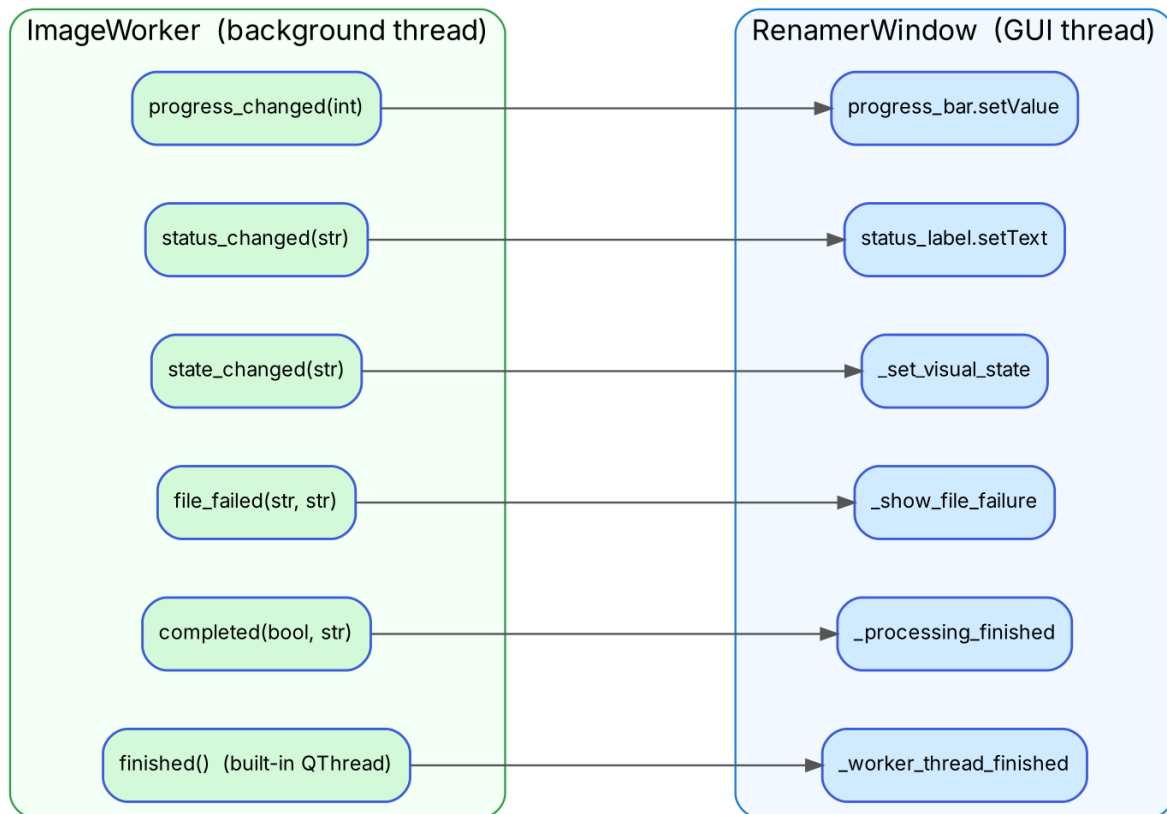


Figure 7. The five custom signals plus the built-in finished signal, and the window method that receives each one.

Signal	Payload	Sent when	Handled by
<code>progress_changed</code>	int 0–100	after each file	<code>progress_bar.setValue</code>
<code>status_changed</code>	text	a new sentence for the user	<code>status_label.setText</code>
<code>state_changed</code>	state key	state changes (caution/working/error/success)	<code>_set_visual_state</code>
<code>file_failed</code>	path, error text	one file could not be processed	<code>_show_file_failure</code>
<code>completed</code>	bool ok, message	the whole job ended	<code>_processing_finished</code>
<code>finished</code> (Qt built-in)	none	the thread has fully stopped	<code>_worker_thread_finished</code>

Cancellation is cooperative

The window never kills the thread. It calls `requestInterruption()` which only sets a flag. The worker checks `isInterruptionRequested()` between files and then raises its own `BatchCancelled` exception, which triggers the rollback. This is why cancelling is safe.

2.5 UML class diagram

The diagram shows every class, its important attributes and methods, and who owns or uses whom. A filled diamond means *owns* (when the window dies, the part dies with it). A dashed arrow means *uses temporarily*.

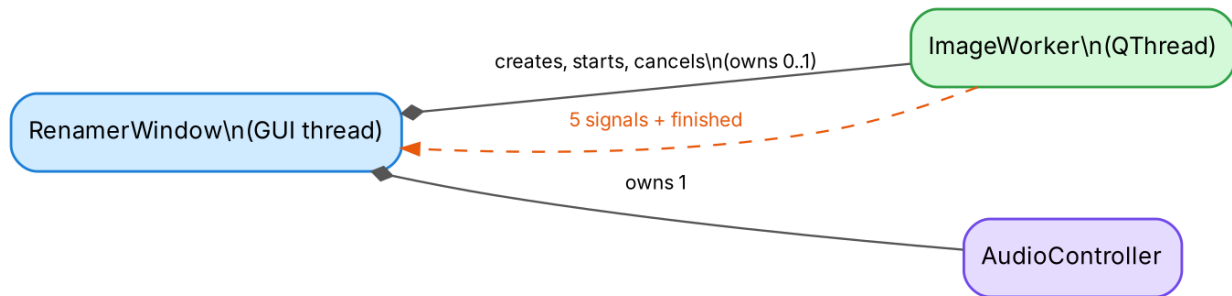


Figure 8a. The three big players and how they talk.

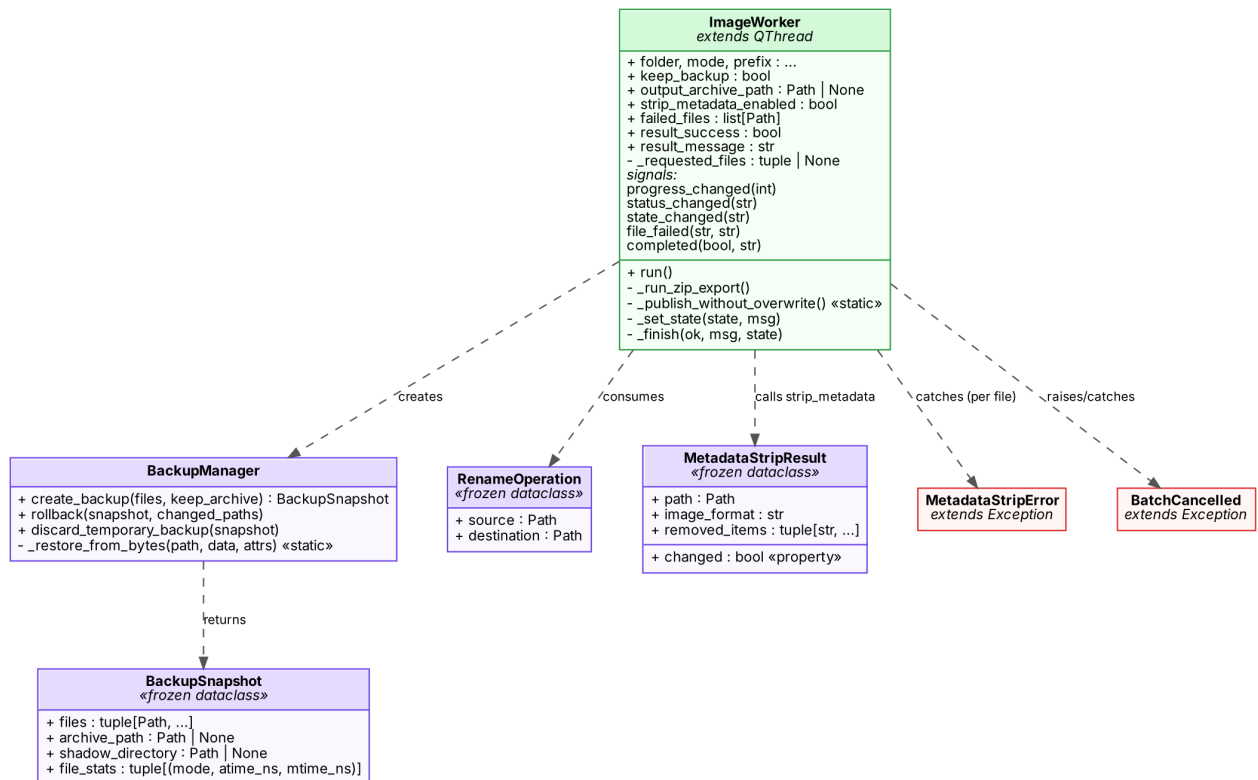


Figure 8b. Class diagram: worker, backup, data records and exceptions.

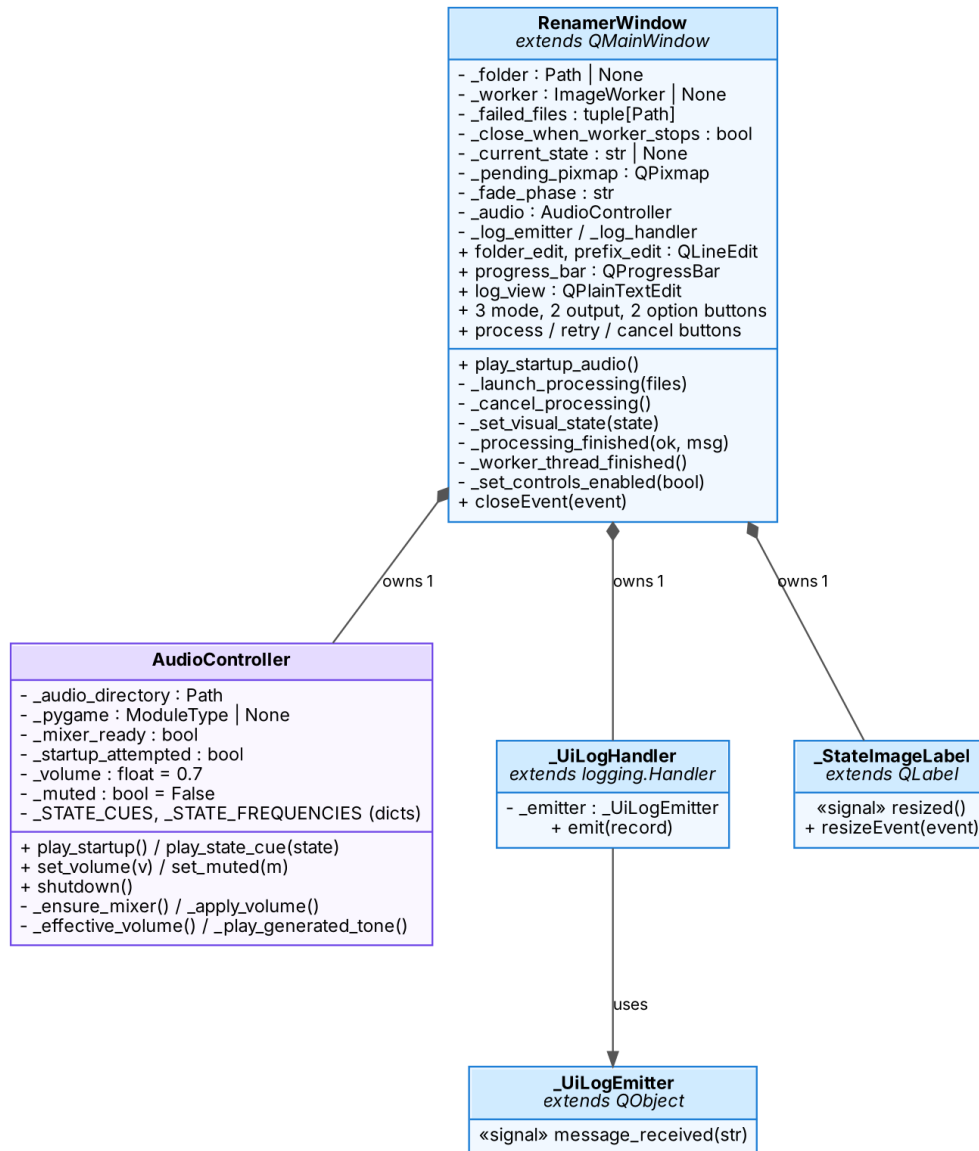


Figure 8c. Class diagram: window, audio controller and the three small UI helpers.

2.6 Start-up flow

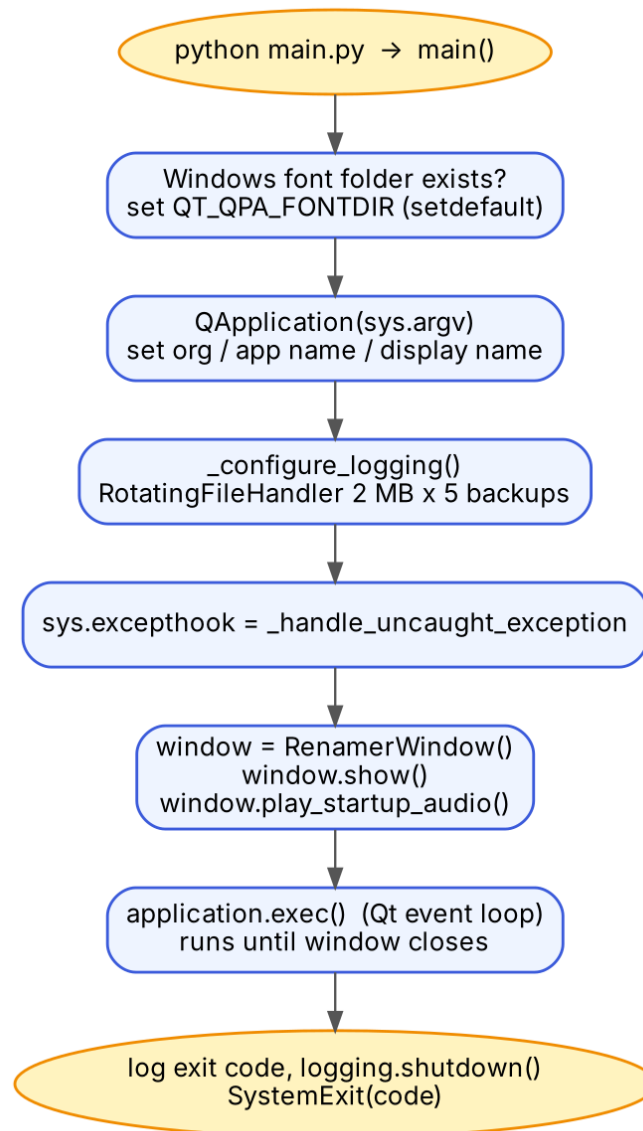


Figure 9. What happens between double-clicking and seeing the window.

2.7 The user interface layout (widget tree)

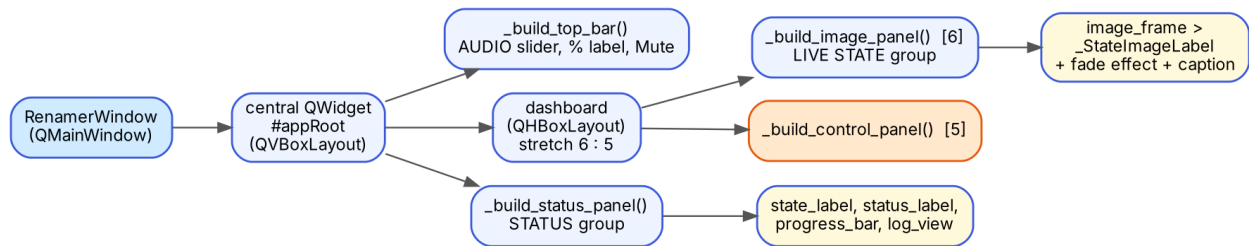


Figure 10a. Widget tree of the main window. Numbers in [] are layout stretch factors (image panel 6 : controls 5).

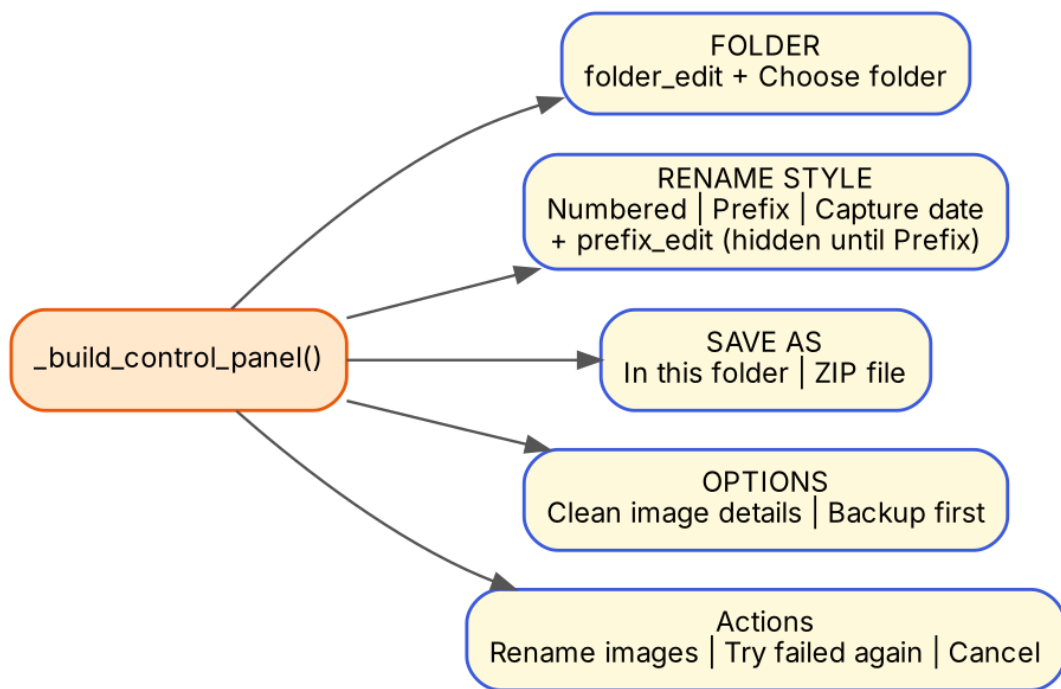


Figure 10b. What is inside the control panel.

2.8 The five visual states

The program is always in one of five **states**. Each state has a banner text, a colour, a picture, and a sound cue. States are plain strings and are changed by `RenamerWindow._set_visual_state`.

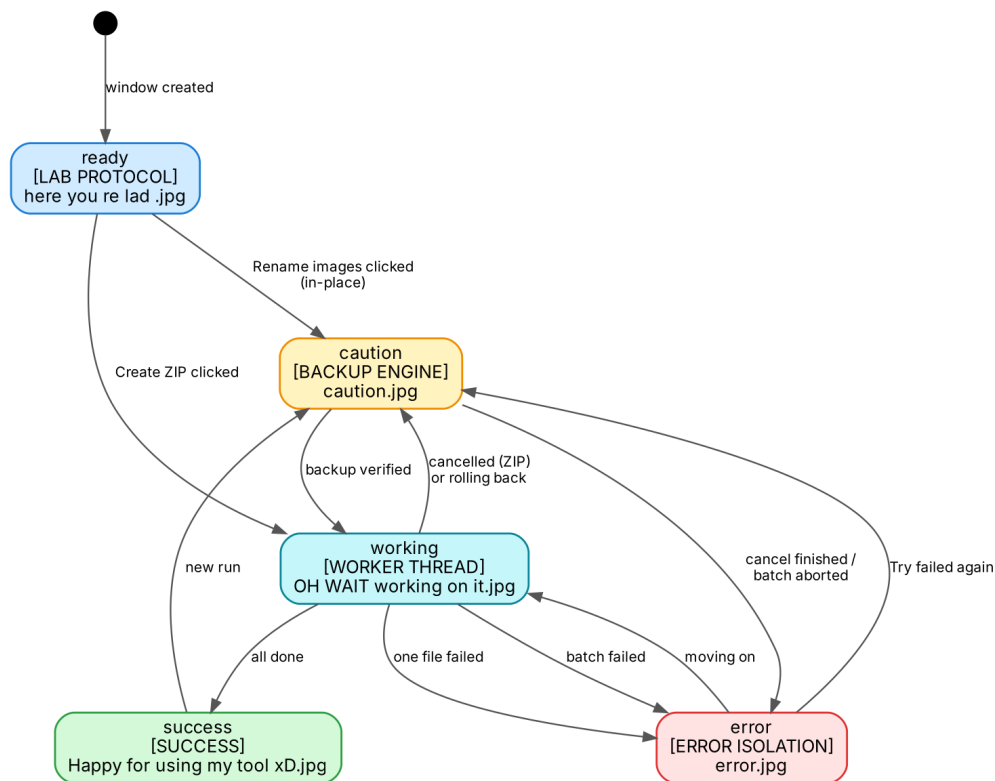


Figure 11. UML state diagram. The transitions follow the real calls to `_set_state/_finish` in the worker.

State	Banner	Colour	Picture file	Tone (no WAV)
ready	[LAB PROTOCOL] HERE YOU ARE, LAD - LET'S DO SOME SCIENCE	#83c9e2	here you re lad .jpg	660 Hz
working	[WORKER THREAD] OH WAIT, WORKING ON IT... (DON'T TOUCH THE DIAL)	#68c5e7	OH WAIT working on it.jpg	440 Hz
caution	[BACKUP ENGINE] CAUTION: TIME CAPSULE SEALED	#d5ad65	caution.jpg	330 Hz
error	[ERROR ISOLATION] ERRORE: A MUTANT FILE FOUGHT BACK	#d9847a	errore.jpg (missing) then error.jpg	220 Hz
success	[SUCCESS] LABORATORY UNSCATHED - MANDARK IS CRYING	#76c99b	Happy for using my tool xD.jpg	880 Hz

2.9 End-to-end sequences

The next four diagrams show who talks to whom, in time order from top to bottom. Orange arrows are Qt signals crossing from the worker thread to the GUI thread. Dashed arrows are return values.

Sequence 1 - In-place run (Rename images)

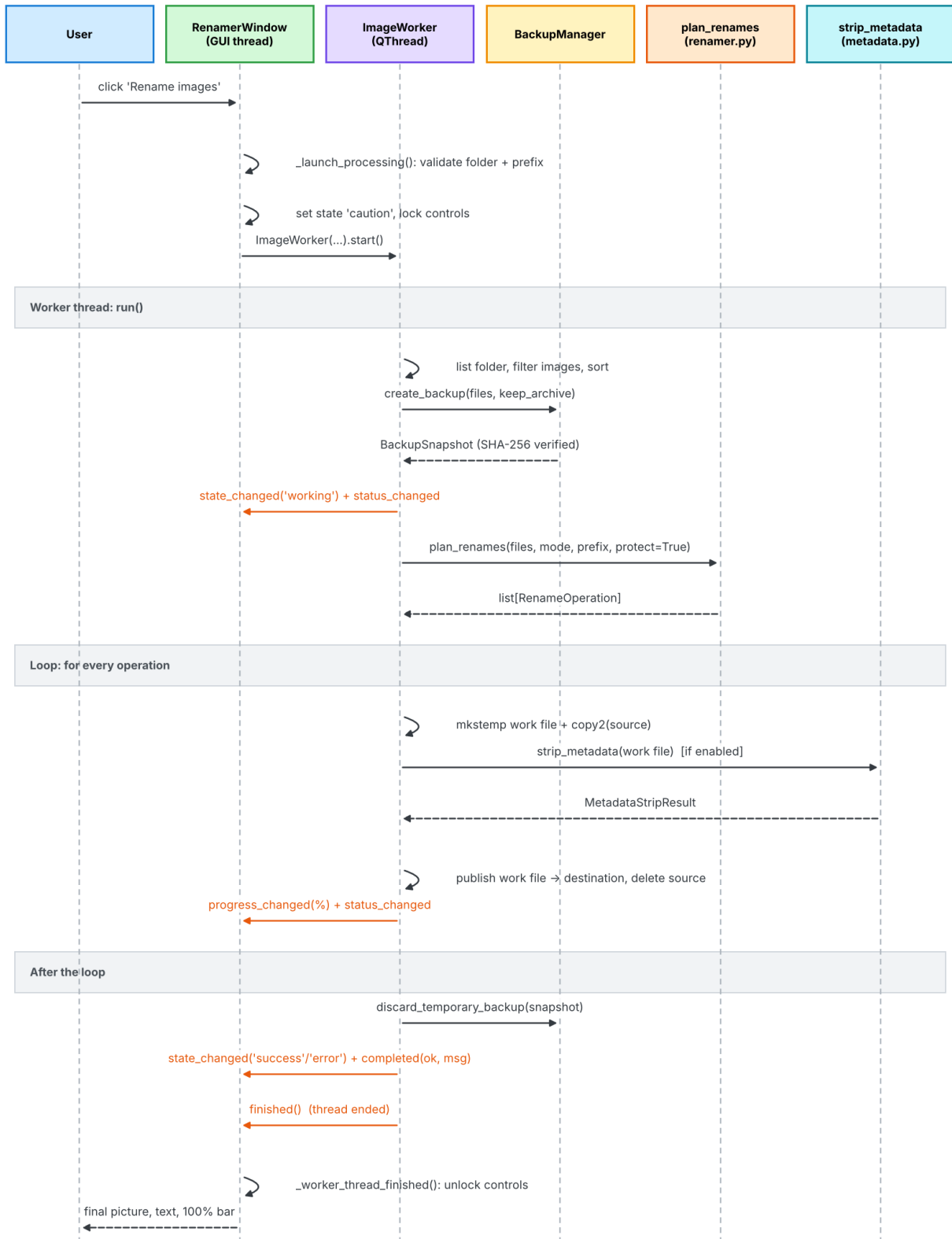


Figure 12. In-place run.

Sequence 2 - ZIP export (originals never touched)

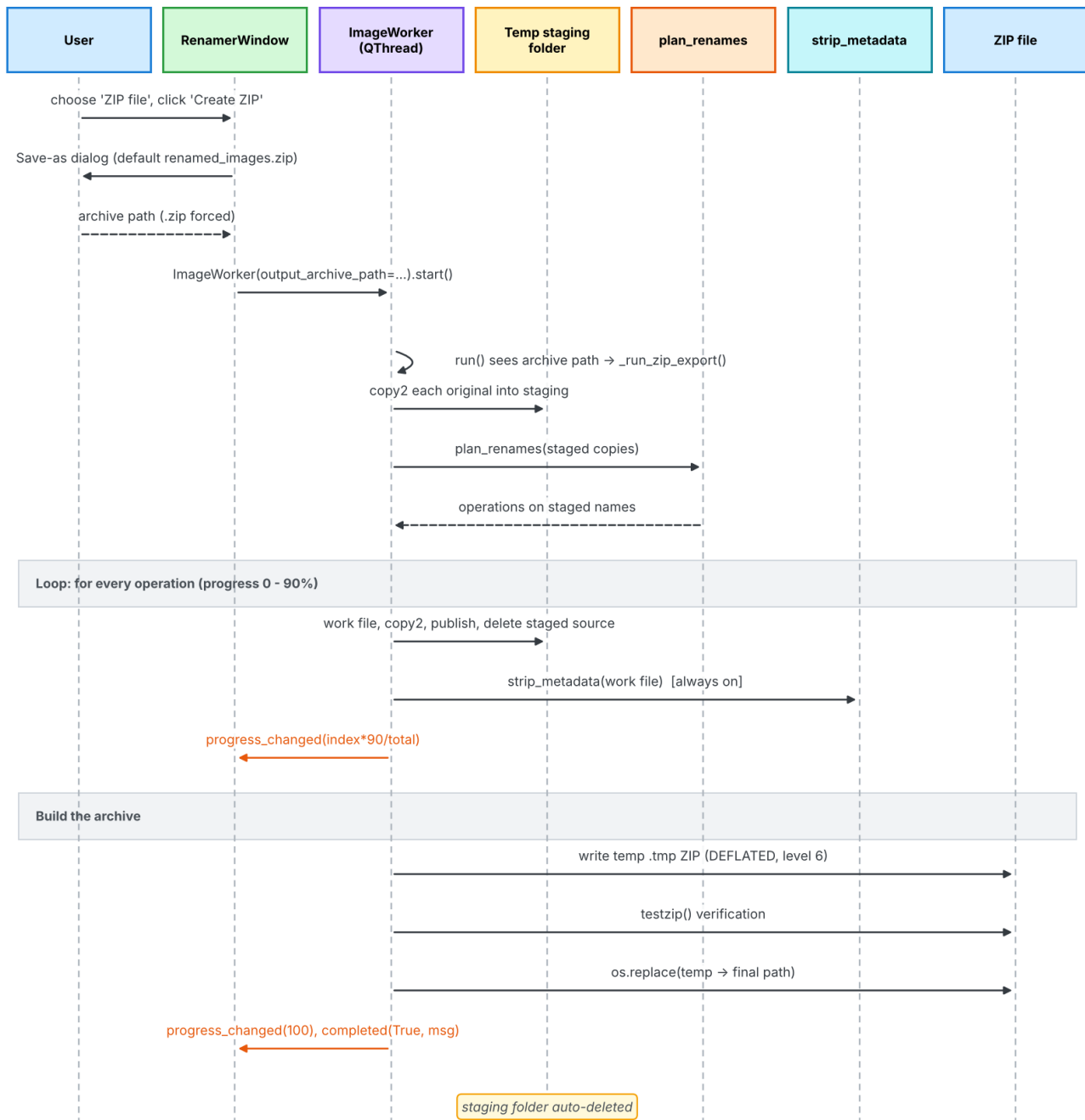


Figure 13. ZIP export.

Sequence 3 - Cancel, and automatic rollback

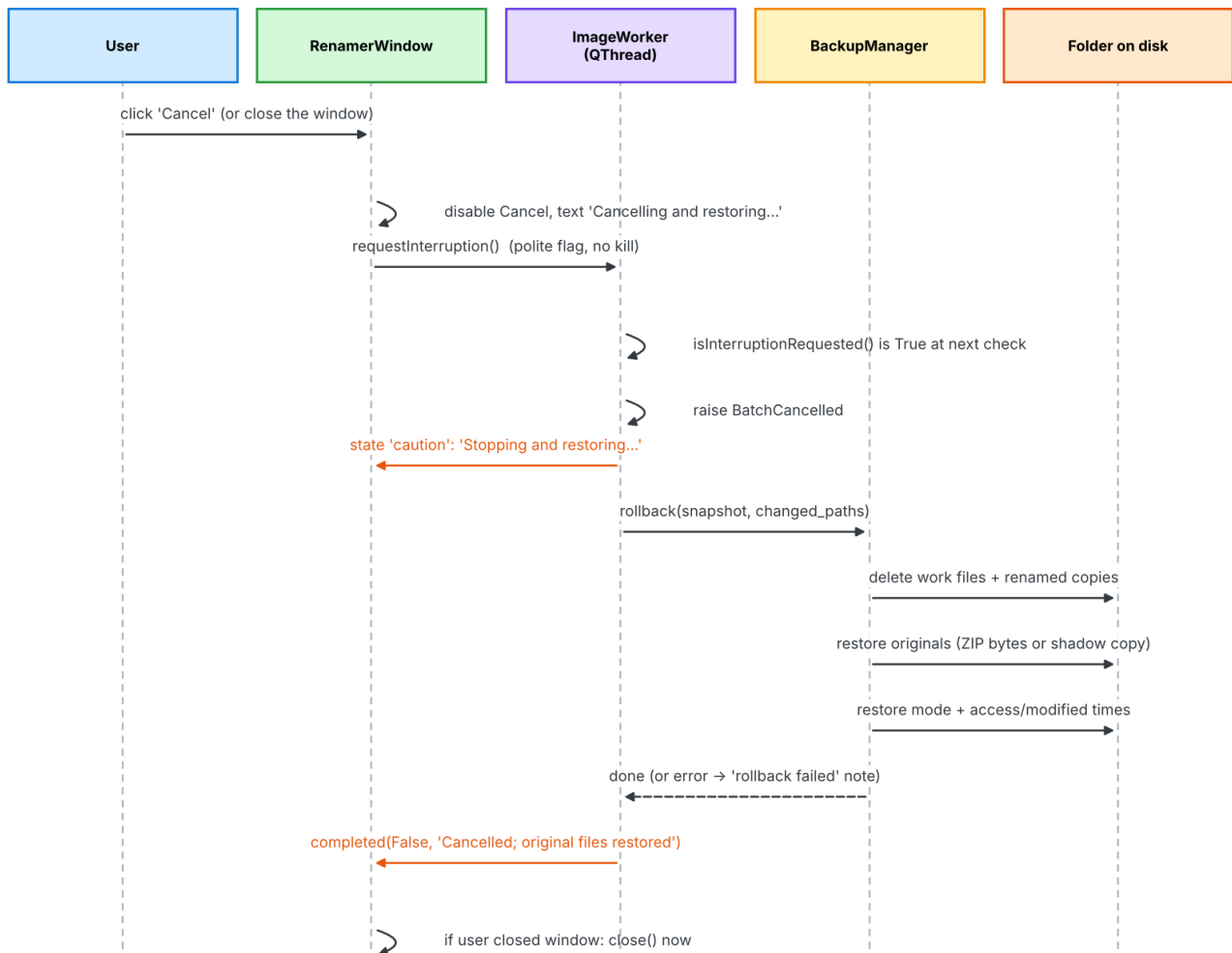


Figure 14. Cancel and rollback.

Sequence 4 - 'Try failed again'

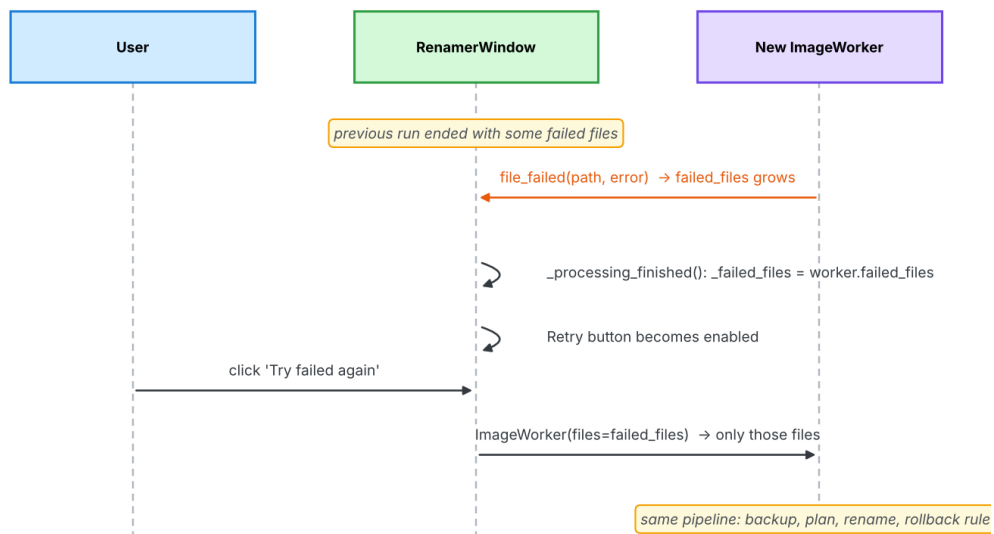


Figure 15. Retry of failed files.

2.10 What happens to ONE file (in-place mode)

This is the heart of the program. Every picture goes through the same small transaction. Study this flowchart: if you understand it, you understand Renamer.

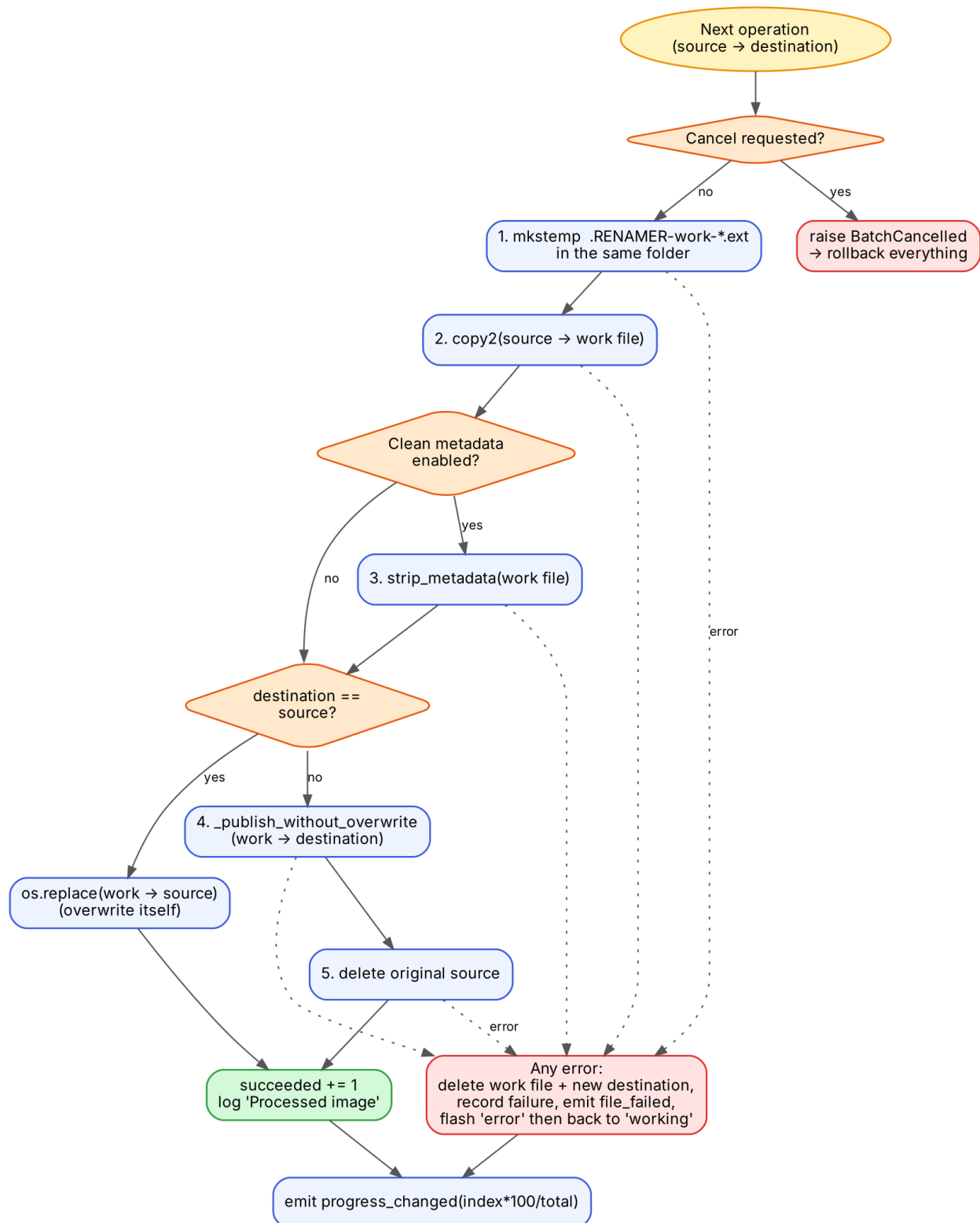


Figure 16. Per-file transaction in `ImageWorker.run`.

2.11 Data that travels between parts

Data	Type	Created by	Used by
Image list	<code>list[Path]</code>	<code>ImageWorker.run</code> (folder scan)	<code>create_backup</code> , <code>plan_renames</code>
<code>RenameOperation</code>	frozen dataclass (source, destination)	<code>plan_renames</code>	worker loop
<code>BackupSnapshot</code>	frozen dataclass	<code>create_backup</code>	<code>rollback</code> , <code>discard_temporary_backup</code>
<code>changed_paths</code>	<code>list[Path]</code>	worker (grows during the run)	<code>rollback</code> (what to delete first)
<code>MetadataStripResult</code>	frozen dataclass	<code>strip_metadata</code>	logging (worker ignores the value)
<code>failed_files</code>	<code>list[Path]</code>	worker	window, for <i>Try failed again</i>
State key	<code>str</code>	worker / window	<code>_set_visual_state</code> , <code>AudioController</code>

PART 3

File by file

Now we open every file. Each chapter starts with a short summary and a table of what the file contains. Then comes the real code in blocks, each followed by a plain-English explanation. The reading order follows how the program runs: start-up first, then the core helpers, then the worker, then the window.

Chapter	File	Lines	Role
3.1	<code>main.py</code>	90	Start-up, logging, crash hook
3.2	<code>core/__init__.py</code> , <code>ui/__init__.py</code> , <code>workers/__init__.py</code>	1 each	Package markers
3.3	<code>core/renamer.py</code>	133	Decides the new file names
3.4	<code>core/backup_manager.py</code>	197	Safe backups and rollback
3.5	<code>core/metadata.py</code>	272	Lossless metadata removal
3.6	<code>core/audio.py</code>	163	Music and sound cues
3.7	<code>workers/image_worker.py</code>	351	Background batch engine
3.8	<code>ui/main_window.py</code>	721	The whole window
3.9	Other files	-	requirements, installer, assets, docs, licence

3.1 main.py — the front door

`main.py` is the file you run. It does five jobs: (1) tell Qt where Windows fonts are, (2) create the Qt application object, (3) set up a rotating log file, (4) install a net that catches crashes, (5) show the window and wait until it closes.

What is inside

Name	Kind	Purpose
<code>logger</code>	module variable	A logger named " RENAMER " used by this file.
<code>_configure_logging()</code>	function → Path	Creates the log folder, attaches a rotating file handler to the root logger, returns the log file path.
<code>_handle_uncaught_exception(...)</code>	function	Replacement for Python's <code>sys.excepthook</code> ; writes crashes to the log.
<code>main()</code>	function → int	Builds and runs the application, returns the exit code.

Lines 1–17 Imports and the module logger

```

1 | """Application entry point for RENAMER."""
2 |
3 | from __future__ import annotations
4 |
5 | import logging
6 | import os
7 | import sys
8 | from logging.handlers import RotatingFileHandler
9 | from pathlib import Path
10 | from types import TracebackType
11 |

```

```

12 | from PySide6.QtCore import QStandardPaths
13 | from PySide6.QtWidgets import QApplication
14 |
15 | from ui.main_window import RenamerWindow
16 |
17 | logger = logging.getLogger("RENAMER")

```

`from __future__ import annotations` lets type hints be written as plain text so newer syntax like `X | None` also works on older Python. The imports bring in tools for logs (`logging`, `RotatingFileHandler`), files (`Path`, `os`), the Qt pieces needed here (`QStandardPaths`, `QApplication`) and the window class `RenamerWindow`. Line 17 makes one logger called `RENAMER`; every `logger.info(...)` in this file goes through it.

Lines 20–43 `_configure_logging`: where do the logs go?

```

20 | def _configure_logging() -> Path:
21 |     """Configure rotating local logs and return the active log file path.
22 |
23 |     Logs are stored in the Qt application-data directory, or in ``~/RENAMER``
24 |     when Qt does not provide a writable location.
25 |     """
26 |     data_directory = QStandardPaths.writableLocation(
27 |         QStandardPaths.StandardLocation.AppLocalDataLocation
28 |     )
29 |     log_root = Path(data_directory) if data_directory else Path.home() / "RENAMER"
30 |     log_directory = log_root / "RENAMER_logs"
31 |     log_directory.mkdir(parents=True, exist_ok=True)
32 |
33 |     log_path = log_directory / "RENAMER.log"
34 |     handler = RotatingFileHandler(log_path, maxBytes=2_000_000, backupCount=5, encoding="utf-8")
35 |     handler.setFormatter(
36 |         logging.Formatter(
37 |             "%(asctime)s %(levelname)s %(name)s %(threadName)s: %(message)s"
38 |         )
39 |     )
40 |     root_logger = logging.getLogger()
41 |     root_logger.setLevel(logging.INFO)
42 |     root_logger.addHandler(handler)
43 |     return log_path

```

Line 26-28 asks Qt for the per-user *application local data* folder (on Windows this is under `AppData\Local`). Line 29: if Qt returned a path, use it; otherwise fall back to `~/RENAMER` in the user's home. Line 30-31 builds the folder `RENAMER_logs` and creates it (with `parents=True` so missing parents are made and `exist_ok=True` so it is fine if it exists).

Line 33-34 creates a **RotatingFileHandler**: when the log file reaches 2,000,000 bytes (about 2 MB) it is renamed and a new one starts; up to 5 old files are kept (`backupCount=5`), so logs can never fill the disk. Lines 35-39 set the line format: *time, level, logger name, thread name, message*. Line 40-42 attaches the handler to the **root logger** and sets the level to INFO, so every module's logs (including the other files) end up in this file. Line 43 returns the path so `main()` can print it in the first log line.

Lines 46–64 `_handle_uncaught_exception`: the crash net

```

46 | def _handle_uncaught_exception(
47 |     exception_type: type[BaseException],
48 |     exception: BaseException,
49 |     traceback: TracebackType | None,
50 | ) -> None:
51 |     """Log uncaught exceptions while preserving Python's KeyboardInterrupt behavior.
52 |
53 |     Args:
54 |         exception_type: The class of the uncaught exception.
55 |         exception: The uncaught exception instance.
56 |         traceback: The traceback associated with the exception.
57 |     """
58 |     if isinstance(exception_type, KeyboardInterrupt):
59 |         sys.__excepthook__(exception_type, exception, traceback)
60 |         return
61 |     logger.critical(

```

```

62 |         "Uncaught exception",
63 |         exc_info=(exception_type, exception, traceback),
64 |     )

```

When any part of the program raises an error that nobody catches, Python calls `sys.excepthook` with three things: the error class, the error object and the traceback. This function is the replacement. If the user pressed Ctrl+C (`KeyboardInterrupt`) it passes control to Python's original hook `sys.__excepthook__` so the normal behaviour is kept (line 58-60). For anything else it writes a **CRITICAL** log entry that includes the full traceback (`exc_info=...`). So even if the window disappears, the reason is in the log file.

Lines 67–90 `main()`: build, show, run, clean up

```

67 | def main() -> int:
68 |     """Configure the application, show its main window, and run the Qt event loop.
69 |
70 |     Returns:
71 |         The exit code returned by Qt's event loop.
72 |     """
73 |     system_fonts = Path(os.environ.get("WINDIR", "C:/Windows")) / "Fonts"
74 |     if system_fonts.is_dir():
75 |         os.environ.setdefault("QT_QPA_FONTDIR", str(system_fonts))
76 |     application = QApplication(sys.argv)
77 |     application.setOrganizationName("Mohamed Yassin Khmiri")
78 |     application.setApplicationName("Renamer")
79 |     application.setApplicationDisplayName("Renamer - Mohamed Yassin Khmiri")
80 |     log_path = _configure_logging()
81 |     sys.excepthook = _handle_uncaught_exception
82 |     logger.info("RENAMER starting; log file: %s", log_path)
83 |
84 |     window = RenamerWindow()
85 |     window.show()
86 |     window.play_startup_audio()
87 |     exit_code = application.exec()
88 |     logger.info("RENAMER exiting with status %d", exit_code)
89 |     logging.shutdown()
90 |     return exit_code

```

Line 73-75: builds `C:/Windows/Fonts` (or the folder in the `WINDIR` environment variable) and, if it exists, sets `QT_QPA_FONTDIR` so Qt can find fonts when its own font folder is missing. `setdefault` means: do not overwrite if the user already set it.

Line 76-79: `QApplication(sys.argv)` is the object that owns the Qt event loop. The next three lines give the program its **organisation name**, **application name** and **window display name**. Order matters: they are set *before* `_configure_logging` because Qt uses organisation and application names to compute the data folder.

Line 80-82: configure logging, install the crash hook, write the first log line. Line 84-86: create the window, show it, and ask it to start the music (`play_startup_audio` is safe to call repeatedly). Line 87: `application.exec()` runs the event loop and blocks until the last window closes; the returned number is the exit code. Line 88-90 log the exit, flush and close all log handlers (`logging.shutdown()`), and return the code. The last two lines of the file (`if __name__ == "__main__": raise SystemExit(main())`) make python `main.py` call `main()` and exit with that code.

Why the exception hook matters in a packaged app

The installer builds the app with `--noconsole` (see Part 5). With no console window, tracebacks would be invisible. The hook plus the rotating file is the only record of a crash.

3.2 The three `__init__.py` files

Each of `core/__init__.py`, `ui/__init__.py` and `workers/__init__.py` contains only a one-line docstring ("""Business logic for Renamer."""", """User interface components for Renamer."""", """Background workers for Renamer."""). An `__init__.py` file tells Python that the folder is a **package**, so lines like `from core.renamer import plan_renames` work. They hold no code.

3.3 `core/renamer.py` — deciding the new names

This module is pure logic: given a list of files and a style, it returns a **plan**: a list saying “*file A becomes B*”. It does not rename anything itself. The plan is built so that two files never get the same name and unrelated existing files are never overwritten.

What is inside

Name	Kind	Purpose
<code>logger</code>	variable	Module logger (<code>core.renamer</code>).
<code>RenameMode</code>	type alias	<code>Literal["default", "prefix", "date"]</code> — the 3 allowed styles.
<code>RenameOperation</code>	frozen dataclass	One pair: source path and destination path.
<code>plan_renames(files, mode, prefix, protect_source_names)</code>	public function	Builds the list of <code>RenameOperation</code> .
<code>_base_stem(source, mode, prefix, index, width)</code>	private function	Makes the name part without extension, for one file.
<code>_validate_prefix(prefix)</code>	private function	Trims the prefix and rejects bad characters.
<code>_image_timestamp(image_path)</code>	private function	Returns <code>YYYY-MM-DD_HH-MM-SS</code> from EXIF or file times.

Lines 1–24 Imports, mode type, and the `RenameOperation` record

```

1 | """Filename generation and collision-safe planning for image batches."""
2 |
3 | from __future__ import annotations
4 |
5 | import logging
6 | import os
7 | from dataclasses import dataclass
8 | from datetime import datetime
9 | from pathlib import Path
10 | from typing import Literal
11 |
12 | from PIL import Image, UnidentifiedImageError
13 |
14 | logger = logging.getLogger(__name__)
15 |
16 | RenameMode = Literal["default", "prefix", "date"]
17 |
18 |
19 | @dataclass(frozen=True, slots=True)
20 | class RenameOperation:
21 |     """One source-to-destination rename operation."""
22 |
23 |     source: Path
24 |     destination: Path

```

`Literal[...]` is a type hint saying a variable may only hold one of those exact strings.

`@dataclass(frozen=True, slots=True)` creates a small class automatically: **frozen** means its fields cannot be changed after creation (safe to pass between threads), **slots** makes it use less memory. `RenameOperation` simply carries `source` and `destination` as `Path` objects.

Lines 27–59 `plan_renames`: the set-up part

```

27 | def plan_renames(
28 |     files: list[Path],
29 |     mode: RenameMode = "default",
30 |     prefix: str = "",
31 |     protect_source_names: bool = False,
32 | ) -> list[RenameOperation]:
33 |     """Build deterministic, collision-free rename destinations.
34 |
35 |     Files are numbered from one, using enough digits for the whole batch.
36 |     Existing files outside the batch are never overwritten.
37 |
38 |     Args:
39 |         files: Image paths to include in the plan.
40 |         mode: Naming strategy to apply to every path.
41 |         prefix: Name prefix used when ``mode`` is ``"prefix"``.
42 |         protect_source_names: Whether to avoid reusing another source name.
43 |
44 |     Returns:
45 |         Rename operations in case-insensitive source-name order.
46 |
47 |     Raises:
48 |         ValueError: If ``mode`` or a required prefix is invalid.
49 |     """
50 |     ordered_files = sorted((Path(file) for file in files), key=lambda item: item.name.casefold())
51 |     if mode not in {"default", "prefix", "date"}:
52 |         raise ValueError(f"Unsupported rename mode: {mode}")
53 |     if mode == "prefix":
54 |         prefix = _validate_prefix(prefix)
55 |
56 |     sources = {file.resolve() for file in ordered_files}
57 |     reserved: set[Path] = set()
58 |     width = max(3, len(str(len(ordered_files))))
59 |     operations: list[RenameOperation] = []

```

The function takes: `files` (list of paths), `mode` (default `"default"`), `prefix` (text for prefix mode), and `protect_source_names` (explained below).

Line 50 makes `ordered_files`: it turns every item into a `Path` and sorts by `item.name.casefold()`. `casefold()` is a stronger lower-case, so `B.jpg` and `a.jpg` sort as `a`, `B` (not all capitals first). Sorting gives the **same result every time** (deterministic).

Line 51-52 raises `ValueError` if `mode` is not one of the three. Line 53-54: in prefix mode the prefix is cleaned and checked (see `_validate_prefix`).

Line 56: `sources` is a set of the full resolved paths of all files in the batch (so we can ask “is this existing file one of ours?”). Line 57: `reserved` is an empty set that will remember names we already promised to other files. Line 58: `width = max(3, len(str(len(ordered_files))))` — the number of digits for numbering: at least 3, and longer for ≥ 1000 files (4 digits). Example: 5 files $\rightarrow$ width 3 (`001`); 1200 files $\rightarrow$ width 4 (`0001`). Line 59 prepares the output list.

Lines 61–79 `plan_renames`: the loop and collision protection

```

61 |     for index, source in enumerate(ordered_files, start=1):
62 |         source_resolved = source.resolve()
63 |         stem = _base_stem(source, mode, prefix, index, width)
64 |         candidate = source.with_name(f"{stem}{source.suffix.lower()}")
65 |         suffix = 1
66 |         while candidate.resolve() in reserved or (
67 |             candidate.exists()
68 |             and (
69 |                 candidate.resolve() not in sources
70 |                 or (protect_source_names and candidate.resolve() != source_resolved)
71 |             )
72 |         ):
73 |             candidate = source.with_name(f"{stem}_{suffix}{source.suffix.lower()}")
74 |             suffix += 1
75 |         reserved.add(candidate.resolve())
76 |         operations.append(RenameOperation(source, candidate))
77 |
78 |     logger.debug("Planned %d rename operation(s) using %s mode", len(operations), mode)
79 |     return operations

```

`enumerate(..., start=1)` counts from 1. For each file, line 63 builds the name stem, e.g. `Image_001`. Line 64 forms the candidate path in **the same folder** with the original extension in lowercase.

Lines 66-72 are the **collision test**. The candidate name is rejected (and a new one is tried) if any of these is true:

1. it is already in **reserved** (another file in this batch already got it); or
2. the candidate file **exists** on disk *and* it is **not** one of the batch's files (**not in sources**) — it would overwrite an unrelated file; or
3. the candidate exists, **protect_source_names** is on, and it is a *different* source file than the current one — it would overwrite another batch file before that file is processed.

While rejected, line 73 tries `Image_001_1.jpg`, then `_2`, `_3`... (`suffix += 1`). Once a free name is found, it is reserved (line 75) and the operation is stored (line 76). Line 78 logs how many operations were planned; line 79 returns the list.

Why `protect_source_names=True` is used by the worker

Suppose the folder has `a.jpg` and `Image_001.jpg`. Sorted: `a.jpg` is number 1 and wants the name `Image_001.jpg`, which is the *other* file in the batch. With protection on, `a.jpg` becomes `Image_001_1.jpg` instead, so the real `Image_001.jpg` is never overwritten during the run. The default is `False` so the pure function can also be used for other purposes. The worker always passes `True`.

Lines 82–99 `_base_stem` and `_validate_prefix`

```

82 | def _base_stem(source: Path, mode: RenameMode, prefix: str, index: int, width: int) -> str:
83 |     """Build one output name's stem from its selected naming style."""
84 |     if mode == "default":
85 |         return f"Image_{index:0{width}d}"
86 |     if mode == "prefix":
87 |         return f"{prefix}_{index:0{width}d}"
88 |     timestamp = _image_timestamp(source)
89 |     return f"{timestamp}_{index:0{width}d}"
90 |
91 |
92 | def _validate_prefix(prefix: str) -> str:
93 |     """Trim a custom prefix and reject blank or filename-unsafe characters."""
94 |     normalized = prefix.strip()
95 |     if not normalized:
96 |         raise ValueError("Enter a prefix before choosing custom-prefix mode")
97 |     if any(character in normalized for character in '<>:"/\\|?*'):
98 |         raise ValueError("Prefix cannot contain path separators or reserved filename characters")
99 |     return normalized

```

`_base_stem` produces the name without extension: **default** → `Image_{index:0{width}d}` (the format spec `0{width}d` pads with zeros to `width` digits); **prefix** → `<prefix>_<number>`; **date** → `<timestamp>_<number>`.

`_validate_prefix` removes spaces from both ends (`strip()`), raises an error if nothing is left (“Enter a prefix before choosing custom-prefix mode”) and raises another if any of `< > : \ " / \ \ | ? *` appears (the characters Windows forbids in file names).

Lines 102–133 `_image_timestamp`: when was the photo taken?

```

102 | def _image_timestamp(image_path: Path) -> str:
103 |     """Get an image's capture time, falling back to file timestamps.
104 |
105 |     EXIF capture, original, and digitized times are preferred. If none can be
106 |     read, use ``st_birthtime`` when available, then Windows ``st_ctime`` or
107 |     ``st_mtime`` on other platforms.
108 |     """
109 |     try:
110 |         with Image.open(image_path) as image:
111 |             exif_metadata = image.getexif()
112 |             capture_date_value = (
113 |                 exif_metadata.get(36867)
114 |                 or exif_metadata.get(306)
115 |                 or exif_metadata.get(36868)
116 |             )
117 |             if capture_date_value:
118 |                 capture_datetime = datetime.strptime(
119 |                     str(capture_date_value).strip(), "%Y:%m:%d %H:%M:%S"
120 |                 )
121 |                 return capture_datetime.strftime("%Y-%m-%d_%H-%M-%S")
122 |     except (OSError, ValueError, UnidentifiedImageError):
123 |         pass
124 |
125 |     file_details = image_path.stat()
126 |     creation_time = getattr(file_details, "st_birthtime", None)
127 |     if creation_time is not None:
128 |         image_timestamp = creation_time
129 |     elif os.name == "nt":
130 |         image_timestamp = file_details.st_ctime
131 |     else:
132 |         image_timestamp = file_details.st_mtime
133 |     return datetime.fromtimestamp(image_timestamp).strftime("%Y-%m-%d_%H-%M-%S")

```

First try (lines 109-123): open the image with Pillow and read its EXIF with `getexif()`. Three numeric EXIF tags are tried in order: **36867** `DateTimeOriginal` (when the shutter was pressed), **306** `DateTime` (last modified) and **36868** `DateTimeDigitized`. The first non-empty value is parsed with the EXIF format `YYYY:MM:DD HH:MM:SS` and re-written as `YYYY-MM-DD_HH-MM-SS`. Any problem (unreadable file, bad text, unknown image type) is silently ignored by `except (OSError, ValueError, UnidentifiedImageError): pass`.

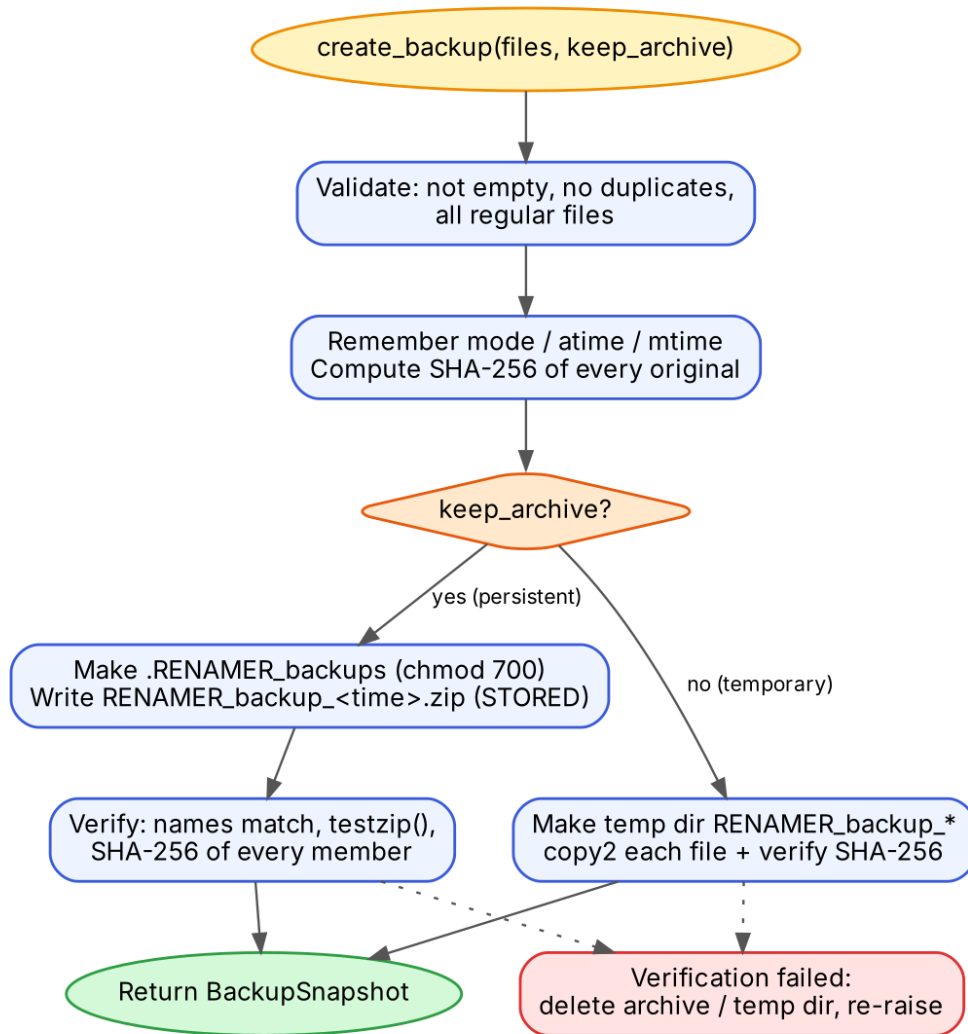
Fallback (lines 125-133): ask the operating system for the file's details. If the system provides `st_birthtime` (creation time, macOS/BSD and newer Pythons on some systems) use it; else on Windows (`os.name == "nt"`) use `st_ctime` (which on Windows is the creation time); else use `st_mtime` (last modified). Convert to local time and format it the same way.

Edge case

The EXIF value is parsed only if it exactly matches `%Y:%m:%d %H:%M:%S`. Strings like `0000:00:00 00:00:00` raise `ValueError` and fall back to file times.

3.4 `core/backup_manager.py` — the safety net

Before the worker changes any original file, this module makes a **verified exact copy**. If anything goes wrong later, `rollback` puts the originals back. Two kinds of backup exist: a **persistent ZIP** (kept for the user) and a **temporary shadow folder** (deleted after a successful run).

Figure 17. Flow of `BackupManager.create_backup`.

What is inside

Name	Kind	Purpose
<code>BackupSnapshot</code>	frozen dataclass	Receipt of a backup: <code>files</code> (original paths), <code>archive_path</code> (ZIP or None), <code>shadow_directory</code> (folder or None), <code>file_stats</code> (mode, atime_ns, mtime_ns per file).
<code>BackupManager.create_backup(files, keep_archive=True)</code>	method	Make and verify the backup, return a snapshot.
<code>BackupManager.rollback(snapshot, changed_paths=None)</code>	method	Delete generated files, restore originals.
<code>BackupManager.discard_temporary_backup(snapshot)</code>	method	Delete the shadow folder after success (ZIP is kept).
<code>BackupManager._restore_from_bytes(path, data, attrs)</code>	static method	Write bytes to a temp file, swap it in, restore permissions and times.
<code>_file_digest(path)</code>	function	SHA-256 of a file, read in 1 MB chunks.
<code>_stream_digest(file)</code>	function	SHA-256 of an open binary stream.

Lines 1–31 Imports, snapshot record and class

```

1 | """Create restorable ZIP or temporary shadow backups for image batches."""
2 |
3 | from __future__ import annotations
4 |
5 | import logging
6 | import os
7 | import shutil
8 | import tempfile
9 | import zipfile
10 | from dataclasses import dataclass
11 | from datetime import datetime
12 | from hashlib import sha256
13 | from pathlib import Path
14 | from typing import BinaryIO
15 |
16 | logger = logging.getLogger(__name__)
17 |
18 |
19 | @dataclass(frozen=True, slots=True)
20 | class BackupSnapshot:
21 |     """Verified pre-processing copy and original paths required for rollback."""
22 |
23 |     files: tuple[Path, ...]
24 |     archive_path: Path | None = None
25 |     shadow_directory: Path | None = None
26 |     file_stats: tuple[tuple[int, int, int], ...] = ()
27 |
28 |
29 | class BackupManager:
30 |     """Manage pre-processing copies and restore original image files."""
31 |

```

BackupSnapshot is the receipt returned by `create_backup`. Exactly one of `archive_path` / `shadow_directory` is set. `file_stats` holds, for each original, a tuple (`st_mode`, `st_atime_ns`, `st_mtime_ns`) — the permission bits and the access/modify times in nanoseconds — so a restore can bring back not just the bytes but also the original timestamps. **BackupManager** has no state of its own; it is a bag of related functions.

Lines 32–64 `create_backup`: checks and fingerprints

```

32 | def create_backup(self, files: list[Path], keep_archive: bool = True) -> BackupSnapshot:
33 |     """Create and verify a byte-exact copy before the caller mutates files.
34 |
35 |     The original filenames are retained in the archive or shadow directory.
36 |     A snapshot is returned only after every stored file matches its original
37 |     SHA-256 digest, so callers can safely begin renaming or metadata removal.
38 |
39 |     Args:
40 |         files: Original files to include in the snapshot.
41 |         keep_archive: Store a persistent ZIP archive when true; otherwise
42 |             use a temporary shadow directory.
43 |
44 |     Returns:
45 |         A verified snapshot that can restore each original file.
46 |
47 |     Raises:
48 |         ValueError: If files is empty, contains duplicates, or contains a
49 |             non-regular file.
50 |         OSError: If copying or verifying the snapshot fails.
51 |     """
52 |     normalized_files = tuple(Path(file).resolve(strict=True) for file in files)
53 |     if not normalized_files:
54 |         raise ValueError("Cannot create a backup for an empty file list")
55 |     if len(set(normalized_files)) != len(normalized_files):
56 |         raise ValueError("Cannot back up the same file more than once")
57 |     if any(not file.is_file() for file in normalized_files):
58 |         raise ValueError("Backups can only contain regular files")
59 |     root = Path(os.path.commonpath([str(file.parent) for file in normalized_files]))
60 |     file_stats = tuple(
61 |         (file.stat().st_mode, file.stat().st_atime_ns, file.stat().st_mtime_ns)
62 |         for file in normalized_files

```

```

63 |         )
64 |         original_digests = tuple(_file_digest(file) for file in normalized_files)

```

Line 52 resolves each path to its absolute real path (`strict=True` raises an error if it does not exist). Lines 53-58 refuse an empty list, duplicates, or anything that is not a regular file. Line 59: `os.path.commonpath` finds the **shared parent folder** (`root`) of all files; names inside the backup are stored relative to it. Lines 60-63 save (`mode`, `atime`, `mtime`) of every file. Line 64: compute the **SHA-256 digest** of every original *before* copying, to compare later.

Lines 66–105 Persistent ZIP backup

```

66 |         if keep_archive:
67 |             backup_directory = root / ".RENAMER_backups"
68 |             backup_directory.mkdir(exist_ok=True)
69 |             os.chmod(backup_directory, 0o700)
70 |             stamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
71 |             descriptor, temporary_name = tempfile.mkstemp(
72 |                 prefix=f"RENAMER_backup_{stamp}_", suffix=".zip", dir=backup_directory
73 |             )
74 |             os.close(descriptor)
75 |             archive_path = Path(temporary_name)
76 |             try:
77 |                 with zipfile.ZipFile(archive_path, "w", compression=zipfile.ZIP_STORED) as archive:
78 |                     for file in normalized_files:
79 |                         archive.write(file, file.relative_to(root).as_posix())
80 |
81 |                 with zipfile.ZipFile(archive_path, "r") as archive:
82 |                     expected_names = [file.relative_to(root).as_posix() for file in normalized_files]
83 |                     if archive.namelist() != expected_names:
84 |                         raise OSError("Backup verification failed: file list does not match originals")
85 |                     corrupt_file = archive.testzip()
86 |                     if corrupt_file is not None:
87 |                         raise OSError(f"Backup verification failed: corrupt archive member {corrupt_file}
88 |                                     ↪ ")
89 |                     for file, member, expected_digest in zip(
90 |                         normalized_files, expected_names, original_digests
91 |                     ):
92 |                         with archive.open(member, "r") as stored_file:
93 |                             if _stream_digest(stored_file) != expected_digest:
94 |                                 raise OSError(f"Backup verification failed for {file.name}")
95 |             except Exception:
96 |                 logger.exception("Failed to create or verify backup archive %s", archive_path)
97 |                 archive_path.unlink(missing_ok=True)
98 |                 raise
99 |
100 |             snapshot = BackupSnapshot(
101 |                 normalized_files,
102 |                 archive_path=archive_path,
103 |                 file_stats=file_stats,
104 |             )
105 |             logger.info("Created verified backup archive %s for %d file(s)", archive_path, len(files))
106 |             return snapshot

```

Creates (if needed) the folder `.RENAMER_backups` next to the images and sets permissions `0o700` (only the owner). A unique file name is made from the time plus microseconds, and `tempfile.mkstemp` reserves it atomically so two runs can never collide. The ZIP is written with `ZIP_STORED` (no compression: images are already compressed, and it is faster) and each file is stored under its path relative to `root`.

Verification (lines 81-93): reopen the ZIP; check that the member list equals the expected names in the same order; run `testzip()` (checks internal CRCs); and finally stream each member and compare its SHA-256 with the digest taken from the original. If *anything* fails, the `except` block logs it, deletes the half-made ZIP and re-raises, so a broken backup never exists. Only after all checks pass is the `BackupSnapshot` created and returned (lines 99-105).

Lines 107–125 Temporary shadow backup

```

107 |         shadow_directory = Path(tempfile.mkdtemp(prefix="RENAMER_backup_"))
108 |         try:
109 |             for file, expected_digest in zip(normalized_files, original_digests):
110 |                 shadow_file = shadow_directory / file.relative_to(root)
111 |                 shadow_file.parent.mkdir(parents=True, exist_ok=True)
112 |                 shutil.copy2(file, shadow_file)
113 |                 if _file_digest(shadow_file) != expected_digest:
114 |                     raise OSError(f"Backup verification failed for {file.name}")
115 |         except Exception:
116 |             logger.exception("Failed to create or verify temporary backup")
117 |             shutil.rmtree(shadow_directory, ignore_errors=True)
118 |             raise
119 |         snapshot = BackupSnapshot(
120 |             normalized_files,
121 |             shadow_directory=shadow_directory,
122 |             file_stats=file_stats,
123 |         )
124 |         logger.info("Created verified temporary backup for %d file(s)", len(files))
125 |         return snapshot

```

When `keep_archive` is false, a temporary directory `RENAMER_backup_*` is made in the system temp folder. Every file is copied with `shutil.copy2` (keeps timestamps) into the same relative path and its digest is checked immediately. On failure the folder is removed and the error is re-raised. The snapshot stores `shadow_directory` instead of `archive_path`.

Lines 127–164 rollback and discard_temporary_backup

```

127 |     def rollback(self, snapshot: BackupSnapshot, changed_paths: list[Path] | None = None) -> None:
128 |         """Restore original files after removing generated paths.
129 |
130 |         Args:
131 |             snapshot: Verified backup created before processing.
132 |             changed_paths: Generated or changed files to remove before restore.
133 |
134 |         Raises:
135 |             ValueError: If the snapshot contains no restorable storage.
136 |             OSError: If a file cannot be removed or restored.
137 |         """
138 |         logger.warning("Rolling back %d original file(s)", len(snapshot.files))
139 |         for path in changed_paths or []:
140 |             candidate = Path(path)
141 |             if candidate.exists() and candidate.is_file():
142 |                 candidate.unlink()
143 |
144 |         root = Path(os.path.commonpath([str(file.parent) for file in snapshot.files]))
145 |         if snapshot.archive_path is not None:
146 |             with zipfile.ZipFile(snapshot.archive_path, "r") as archive:
147 |                 for original, saved_file_attributes in zip(snapshot.files, snapshot.file_stats):
148 |                     self._restore_from_bytes(
149 |                         original,
150 |                         archive.read(original.relative_to(root).as_posix()),
151 |                         saved_file_attributes,
152 |                     )
153 |         elif snapshot.shadow_directory is not None:
154 |             for original in snapshot.files:
155 |                 shutil.copy2(snapshot.shadow_directory / original.relative_to(root), original)
156 |             shutil.rmtree(snapshot.shadow_directory, ignore_errors=True)
157 |         else:
158 |             raise ValueError("Backup snapshot has no restorable storage")
159 |         logger.info("Rollback restored %d original file(s)", len(snapshot.files))
160 |
161 |     def discard_temporary_backup(self, snapshot: BackupSnapshot) -> None:
162 |         """Remove shadow-copy storage after success; retain archive backups."""
163 |         if snapshot.shadow_directory is not None:
164 |             shutil.rmtree(snapshot.shadow_directory, ignore_errors=True)

```

rollback first logs a warning, then deletes every path in `changed_paths` that exists and is a file (the renamed copies, temporary work files, and also the current originals — they will be restored next). It recomputes the `root`, then:

- *ZIP backup*: for each original, reads its bytes out of the ZIP and calls `_restore_from_bytes` with the saved attributes. - *Shadow backup*: copies each file back with `copy2` and then removes the shadow folder. - Neither: raises `ValueError` (nothing to restore from).

discard_temporary_backup: if the snapshot has a shadow folder, delete it (`ignore_errors=True`). ZIP backups are intentionally kept.

Lines 166–197 `_restore_from_bytes` and the digest helpers

```

166 |     @staticmethod
167 |     def _restore_from_bytes(
168 |         path: Path,
169 |         data: bytes,
170 |         saved_file_attributes: tuple[int, int, int],
171 |     ) -> None:
172 |         """Write original bytes and restore file mode and access/modification times."""
173 |         descriptor, temporary_name = tempfile.mkstemp(prefix=f".{path.name}.", dir=path.parent)
174 |         try:
175 |             with os.fdopen(descriptor, "wb") as temporary_file:
176 |                 temporary_file.write(data)
177 |                 os.replace(temporary_name, path)
178 |                 os.chmod(path, saved_file_attributes[0])
179 |                 os.utime(path, ns=(saved_file_attributes[1], saved_file_attributes[2]))
180 |         except Exception:
181 |             logger.exception("Failed to restore backup for %s", path)
182 |             Path(temporary_name).unlink(missing_ok=True)
183 |             raise
184 |
185 |
186 | def _file_digest(path: Path) -> bytes:
187 |     """Calculate a file's SHA-256 digest without reading it all at once."""
188 |     with path.open("rb") as image_file:
189 |         return _stream_digest(image_file)
190 |
191 |
192 | def _stream_digest(image_file: BinaryIO) -> bytes:
193 |     """Hash a binary stream a chunk at a time and return its SHA-256 digest."""
194 |     digest = sha256()
195 |     for chunk in iter(lambda: image_file.read(1024 * 1024), b''):
196 |         digest.update(chunk)
197 |     return digest.digest()

```

`_restore_from_bytes` is a mini atomic write: create a hidden temporary file in the *same* folder, write the bytes, then `os.replace` it over the target (an atomic swap on the same drive). Then `os.chmod` restores permissions and `os.utime(ns=...)` restores the access and modification times. On any error the temp file is deleted and the error is re-raised.

`_file_digest` opens the file and hands it to `_stream_digest`, which reads **1 MB at a time** (`iter(lambda: f.read(1024*1024), b'')` stops at the empty bytes object that marks end of file) and feeds a `sha256` object. Memory use stays small even for huge photos. The result is the 32-byte binary digest (not hex), which is perfectly fine for comparing.

Behaviour worth knowing

In rollback, files are restored only for the originals listed in the snapshot. Files that were *renamed* are deleted first via `changed_paths`; the original names come back from the backup. Because the ZIP stores originals under their original names, a user can also restore manually by unzipping.

3.5 core/metadata.py — lossless cleaning of JPEG and PNG

This is the most technical file. It removes hidden information from pictures **without decoding the picture**. A normal way would be: open the image with Pillow, save it again. That would *re-compress* a JPEG and slowly destroy quality. Instead this module reads the file as a stream of labelled blocks and copies only the blocks that are safe to keep. The pixel data is copied untouched, byte for byte.

A very short course on file structure

Format	Structure	What is removed here	What is kept
JPEG	A series of <i>segments</i> . Each starts with FF + a marker byte, then a 2-byte length (big-endian, includes itself) and a payload. After the <i>Start of Scan</i> marker (FF DA) comes the compressed picture data until the end marker FF D9 .	APP0–APPF segments (EXIF in APP1, XMP, IPTC in APP13, etc.) and COM (comment, FE)	ICC colour profile (APP2 with ICC_PROFILE\0), Adobe marker (APPE with Adobe), and every non-APP segment (tables, frame header...) plus all picture data
PNG	An 8-byte signature, then <i>chunks</i> : 4-byte length, 4-byte type (letters), data, 4-byte CRC32. Ends with an IEND chunk.	Chunks eXIf tEXt zTXt iTXt tIME pHYs , any chunk with a malformed name, and any <i>ancillary</i> chunk not on the allow-list	Required (critical) chunks IHDR PLTE IDAT IEND and the allow-listed colour/animation chunks

What is inside

Name	Kind	Purpose
MetadataStripError	exception	Raised for invalid or unsupported images.
MetadataStripResult	frozen dataclass	path, image_format, removed_items ; property changed .
_PNG_METADATA_CHUNKS	set of bytes	Chunks always removed.
_PNG_PRESERVED Ancillary_CHUNKS	set of bytes	Optional chunks that affect rendering and are therefore kept.
_PNG_SIGNATURE	bytes	89 50 4E 47 0D 0A 1A 0A .
_PNG_CHUNK_BUFFER_SIZE	int	64 KB read buffer for chunk data.
_JPEG_STANDALONE_MARKERS	set of ints	Markers with no length field: 0x01 and 0xD0–0xD8.
strip_metadata(path)	public function	Validate, pick the format, write a cleaned temp file, atomically swap it in.
_strip_jpeg(src, dst)	function	Copy a JPEG without metadata segments.
_strip_png(src, dst)	function	Copy a PNG without metadata chunks, checking every CRC.
_keep_png_chunk(type)	function	Decide whether a PNG chunk is kept.

Lines 1–62 Imports, error, result record and constants

```

1 | """Losslessly remove common metadata from JPEG and PNG images.
2 |
3 | Image data is copied as-is: this module never saves decoded pixels through
4 | Pillow, which would recompress JPEGs. Other supported formats are validated
5 | and left byte-for-byte unchanged.
6 | """
7 |
8 | from __future__ import annotations
9 |
10 | import logging
11 | import os
12 | import shutil
13 | import stat
14 | import tempfile
15 | import zlib
16 | from dataclasses import dataclass
17 | from pathlib import Path
18 | from typing import BinaryIO
19 |
20 | from PIL import Image, UnidentifiedImageError
21 |
22 | logger = logging.getLogger(__name__)
23 |
24 |
25 | class MetadataStripError(Exception):
26 |     """Raised when an image is invalid or its format is not supported."""
27 |
28 |
29 | @dataclass(frozen=True, slots=True)
30 | class MetadataStripResult:
31 |     """Summary of metadata removed from one image."""
32 |
33 |     path: Path
34 |     image_format: str
35 |     removed_items: tuple[str, ...]
36 |
37 |     @property
38 |     def changed(self) -> bool:
39 |         """Whether any metadata segments or chunks were removed."""
40 |         return bool(self.removed_items)
41 |
42 |
43 | _PNG_METADATA_CHUNKS = {b"eXIf", b"tEXt", b"zTXt", b"iTXt", b"tIME", b"pHYs"}
44 | _PNG_PRESERVED Ancillary_CHUNKS = {
45 |     # These affect color interpretation, rendering, or animated image data.
46 |     b"acTL",
47 |     b"bKGD",
48 |     b"cHRM",
49 |     b"cICP",
50 |     b"cLLi",
51 |     b"fcTL",
52 |     b"fdAT",
53 |     b"gAMA",
54 |     b"iCCP",
55 |     b"mDCv",
56 |     b"sBIT",
57 |     b"sRGB",
58 |     b"tRNS",
59 | }
60 | _PNG_SIGNATURE = b"\x89PNG\r\n\x1a\n"
61 | _PNG_CHUNK_BUFFER_SIZE = 64 * 1024
62 | _JPEG_STANDALONE_MARKERS = {0x01, *range(0xD0, 0xD9)}

```

`MetadataStripResult.changed` is a **property**: it is `True` when `removed_items` is not empty.

Constants worth understanding. `_PNG_METADATA_CHUNKS` are the chunks that carry personal or descriptive data: `eXIf` (EXIF), `tEXt/zTXt/iTXt` (text notes, compressed or international), `tIME` (last-change time) and `pHYs` (physical pixel size). `_PNG_PRESERVED Ancillary_CHUNKS` lists optional chunks that change how the image **looks** or animates, so they stay: `acTL` `fcTL` `fdAT` (animated PNG), `bKGD` (background), `cHRM` `gAMA` `sRGB` `iCCP` `sBIT` `cICP` `mDCv` `cLLi` (colour handling and HDR info) and `tRNS` (transparency).

`_JPEG_STANDALONE_MARKERS = {0x01, *range(0xD0, 0xD9)}` are markers that stand alone with no length: TEM (01) and RST0-RST7/SOI (D0-D8). The star `*` unpacks the range into the set. Note that `0xD9` (end of image) is *not* in the range because `range(0xD0, 0xD9)` stops before `0xD9`; it is handled separately in code.

Lines 65–106 `strip_metadata`, part 1: validate and choose a path

```

65 | def strip_metadata(image_path: str | Path) -> MetadataStripResult:
66 |     """Remove metadata from JPEG/PNG without recompressing the image.
67 |
68 |     EXIF, XMP, IPTC, comments, and textual PNG chunks are removed. PNG
69 |     color/rendering chunks and JPEG ICC profiles are retained so metadata
70 |     removal does not silently change how the image is displayed. Other image
71 |     formats are validated but left unchanged, preserving animation and pixels.
72 |     The original file is replaced only after the complete output is written.
73 |
74 |     Raises:
75 |         MetadataStripError: If the file is invalid or is not a supported image.
76 |         OSError: If the image cannot be read or atomically replaced.
77 |
78 |     Args:
79 |         image_path: Path to the image to validate and process.
80 |
81 |     Returns:
82 |         The resolved path, detected format, and descriptions of removed data.
83 |     """
84 |     path = Path(image_path).resolve(strict=True)
85 |     if not path.is_file():
86 |         raise MetadataStripError(f"Not a regular image file: {path}")
87 |
88 |     try:
89 |         with Image.open(path) as image:
90 |             image_format = image.format
91 |             image.verify()
92 |     except (OSError, UnidentifiedImageError) as error:
93 |         raise MetadataStripError(f"Unable to validate image: {path}") from error
94 |
95 |     if image_format not in {"JPEG", "PNG", "WEBP", "BMP", "TIFF", "GIF"}:
96 |         raise MetadataStripError(
97 |             f"Unsupported image format {image_format!r}"
98 |         )
99 |
100 |     if image_format not in {"JPEG", "PNG"}:
101 |         logger.info(
102 |             "Metadata cleanup is not available for %s; leaving %s unchanged",
103 |             image_format,
104 |             path.name,
105 |         )
106 |     return MetadataStripResult(path, image_format, ())

```

The path is resolved (`strict=True` → error if missing) and must be a regular file. Then Pillow opens it only to read `image.format` and call `image.verify()`, which checks the file's integrity **without decoding all pixels**. Any Pillow/OS error becomes a `MetadataStripError("Unable to validate image")`. If the format is not one of JPEG, PNG, WEBP, BMP, TIFF, GIF it is rejected as unsupported.

For WebP/BMP/TIFF/GIF the function logs *“cleanup is not available... leaving unchanged”* and **returns an empty result straight away** — the file is not touched. That is why these formats are “validated only”.

Lines 108–137 `strip_metadata`, part 2: write-then-swap

```

108 |     descriptor, temporary_name = tempfile.mkstemp(
109 |         prefix=f"{path.name}.", suffix=".tmp", dir=path.parent
110 |     )
111 |     temporary_path = Path(temporary_name)
112 |
113 |     # Write to a same-directory temporary file so replacement stays atomic.
114 |     try:
115 |         with os.fdopen(descriptor, "wb") as destination, path.open("rb") as source:
116 |             if image_format == "JPEG":
117 |                 removed_items = _strip_jpeg(source, destination)
118 |             else:

```

```

119 |         removed_items = _strip_png(source, destination)
120 |
121 |     if removed_items:
122 |         os.chmod(temporary_path, stat.S_IMODE(path.stat().st_mode))
123 |         os.replace(temporary_path, path)
124 |     else:
125 |         temporary_path.unlink()
126 |
127 |     result = MetadataStripResult(path, image_format, tuple(removed_items))
128 |     logger.info(
129 |         "Metadata removal from %s: %s",
130 |         path.name,
131 |         ", ".join(result.removed_items) or "nothing found",
132 |     )
133 |     return result
134 | except Exception:
135 |     logger.exception("Metadata removal failed for %s", path)
136 |     temporary_path.unlink(missing_ok=True)
137 |     raise

```

A temporary file is created in the **same folder** (mkstemp, name like `.photo.jpg.xxxx.tmp`). Same folder matters: `os.replace` is only atomic inside one drive. The source is opened for reading and the temp file for writing, and the right stripper runs. If it removed something, permissions of the original are copied to the temp file (`os.chmod(..., stat.S_IMODE(mode))`) and the temp file replaces the original in a single atomic step. If nothing was removed, the temp file is simply deleted and the original is left untouched (no needless rewrite). Any exception deletes the temp file and is re-raised. The original is therefore *never* left half-written.

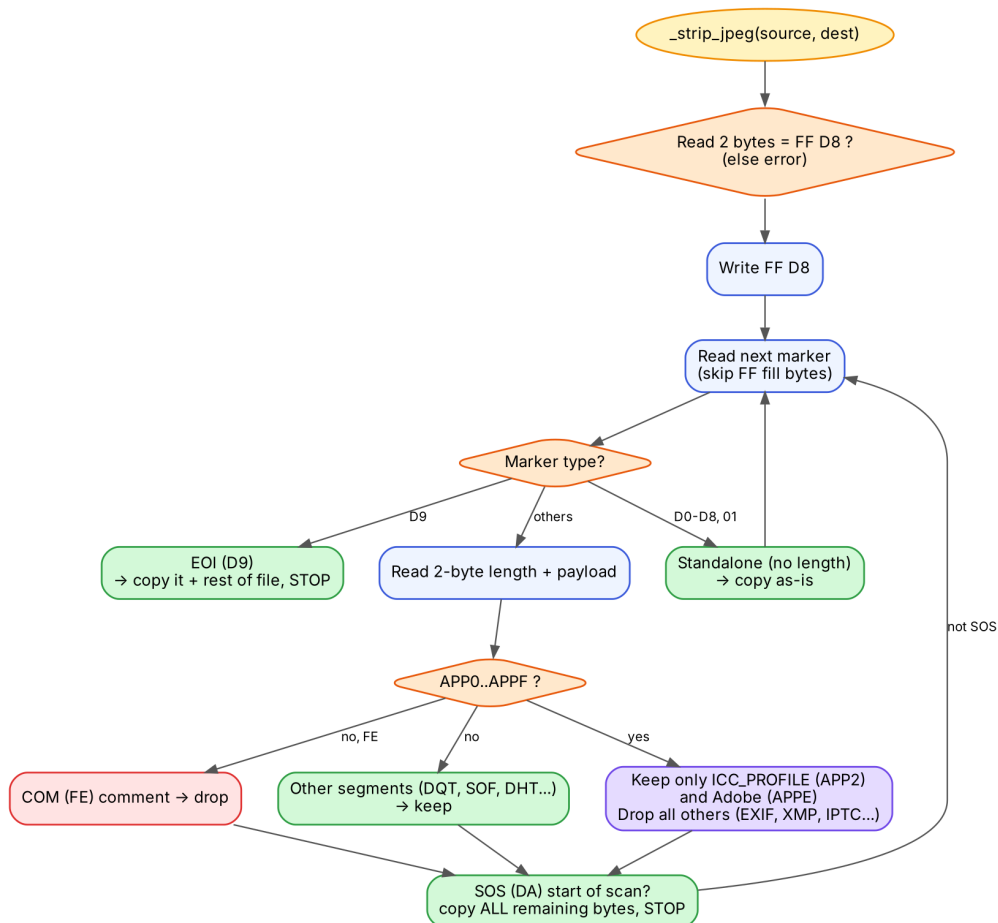


Figure 18. Flow of `_strip_jpeg`.

Lines 140–210 `_strip_jpeg: copy segments, skip metadata`

```

140 | def _strip_jpeg(source: BinaryIO, destination: BinaryIO) -> list[str]:
141 |     """Copy JPEG segments, omitting metadata and comments.
142 |
143 |     Preserved segment bytes and compressed scan data are copied without
144 |     decoding or recompressing the image.
145 |     """
146 |     if source.read(2) != b"\xff\xd8":
147 |         raise MetadataStripError("Invalid JPEG start marker")
148 |     destination.write(b"\xff\xd8")
149 |     removed_items: list[str] = []
150 |
151 |     while True:
152 |         marker_start = source.read(1)
153 |         if marker_start != b"\xff":
154 |             raise MetadataStripError("Malformed JPEG marker stream")
155 |
156 |         fill_bytes = bytearray()
157 |         marker_byte = source.read(1)
158 |         while marker_byte == b"\xff":
159 |             fill_bytes.extend(marker_byte)
160 |             marker_byte = source.read(1)
161 |         if not marker_byte:
162 |             raise MetadataStripError("Unexpected end of JPEG file")
163 |
164 |         marker = marker_byte[0]
165 |         marker_bytes = marker_start + bytes(fill_bytes) + marker_byte
166 |         if marker == 0x00:
167 |             raise MetadataStripError("Unexpected stuffed byte outside JPEG image data")
168 |
169 |         if marker in _JPEG_STANDALONE_MARKERS:
170 |             destination.write(marker_bytes)
171 |             continue
172 |         if marker == 0xD9:
173 |             destination.write(marker_bytes)
174 |             shutil.copyfileobj(source, destination)
175 |             break
176 |
177 |         length_bytes = source.read(2)
178 |         if len(length_bytes) != 2:
179 |             raise MetadataStripError("Truncated JPEG segment length")
180 |         segment_length = int.from_bytes(length_bytes, "big")
181 |         if segment_length < 2:
182 |             raise MetadataStripError("Invalid JPEG segment length")
183 |         payload = source.read(segment_length - 2)
184 |         if len(payload) != segment_length - 2:
185 |             raise MetadataStripError("Truncated JPEG segment")
186 |
187 |         keep_segment = True
188 |         label = f"APP{marker - 0xE0}" if 0xE0 <= marker <= 0xEF else None
189 |         if label is not None:
190 |             # Preserve ICC color profiles and Adobe's CMYK/YCCK interpretation.
191 |             keep_segment = (marker == 0xE2 and payload.startswith(b"ICC_PROFILE\x00")) or (
192 |                 marker == 0xEE and payload.startswith(b"Adobe")
193 |             )
194 |         elif marker == 0xFE:
195 |             label = "COM"
196 |             keep_segment = False
197 |
198 |         if keep_segment:
199 |             destination.write(marker_bytes)
200 |             destination.write(length_bytes)
201 |             destination.write(payload)
202 |         else:
203 |             removed_items.append(label or f"JPEG marker 0x{marker:02X}")
204 |
205 |         if marker == 0xDA:
206 |             # Metadata is stored before the first scan in standard JPEG files.
207 |             shutil.copyfileobj(source, destination)
208 |             break
209 |
210 |     return removed_items

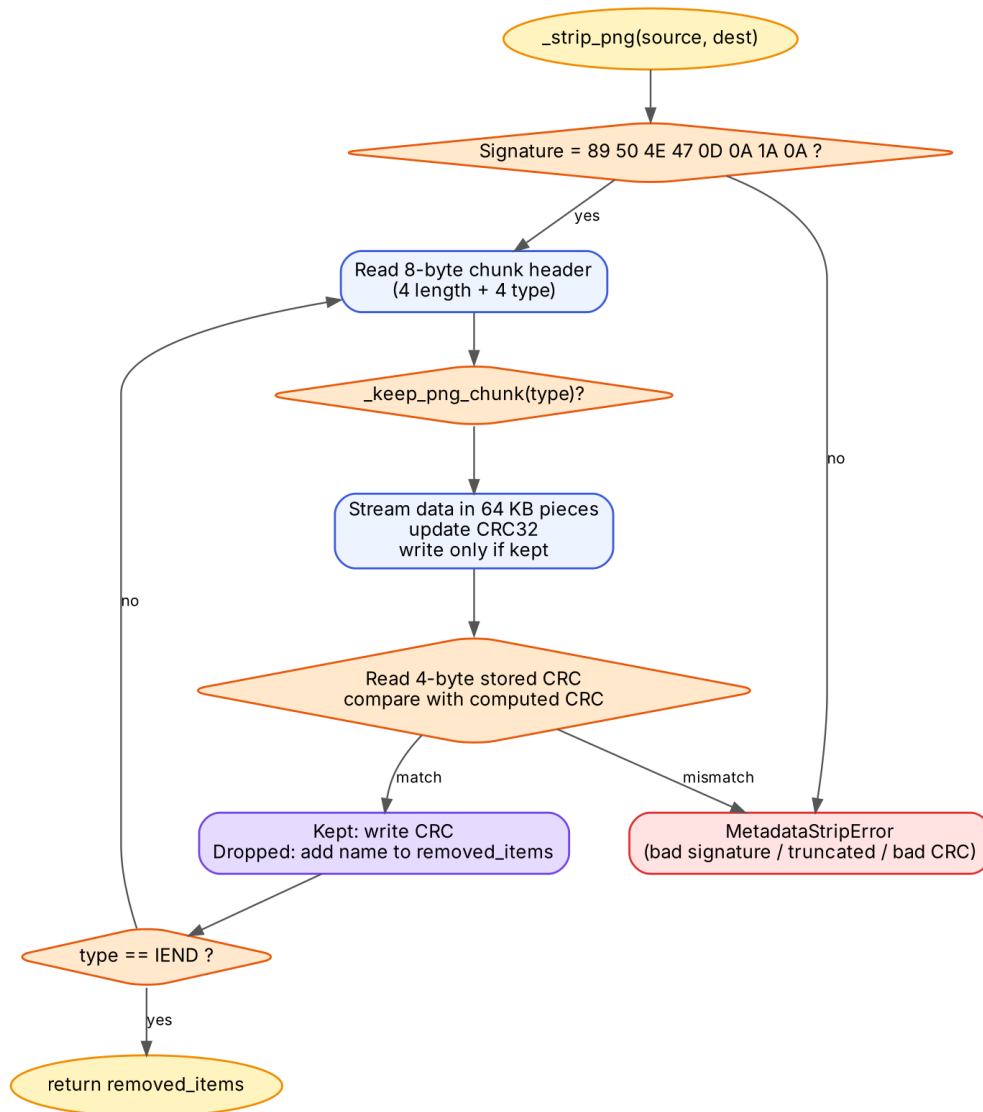
```

Lines 146-148 check the first two bytes are **FF D8** (Start Of Image) and write them. Then the loop reads one marker per round: a marker is **FF** followed by a marker byte, possibly with extra **FF** fill bytes in between (lines 156-160; they are remembered in `fill_bytes` so the output stays identical). A marker byte of **00** here is illegal (line 166). Standalone markers are copied and the loop continues. **D9** (End Of Image) is written and **everything left in the file** is copied with `shutil.copyfileobj`, then the loop ends.

For all other markers, lines 177-185 read the two length bytes, check the length is at least 2 (it counts itself) and read exactly `length - 2` payload bytes; short reads mean a truncated file and raise `MetadataStripError`.

Lines 187-196 decide what to keep. `keep_segment` starts `True`. For APP markers (**0xE0-0xEF**) the label is **APP0...APP15**; they are dropped, except `**APP2` starting with **ICC_PROFILE\0** (**the colour profile; without it colours may look wrong**) and APPE starting with **Adobe**** (tells CMYK/YCCK files how to interpret colours). The comment segment **FE** is always dropped (label **COM**). Everything else (quantisation tables, Huffman tables, the frame header) is kept. Kept segments are written as marker + length + payload; dropped ones add their label to `removed_items`.

Lines 205-208: when the **Start Of Scan** (**DA**) segment has been handled, the rest of the file is compressed picture data. It is copied as one block with no parsing and the loop ends. (The code comment explains: in standard JPEGs the metadata comes before the first scan.) The function returns the list of removed labels, e.g. `['APP1', 'APP13', 'COM']`.

Figure 19. Flow of `_strip_png`.**Lines 213–260 `_strip_png`: stream chunk by chunk and check CRC**

```

213 | def _strip_png(source: BinaryIO, destination: BinaryIO) -> list[str]:
214 |     """Copy valid PNG chunks, omit metadata, and validate every chunk CRC.
215 |
216 |     Chunk payloads are streamed to avoid buffering large images in memory.
217 |     """
218 |     if source.read(len(_PNG_SIGNATURE)) != _PNG_SIGNATURE:
219 |         raise MetadataStripError("Invalid PNG signature")
220 |     destination.write(_PNG_SIGNATURE)
221 |     removed_items: list[str] = []
222 |
223 |     while True:
224 |         chunk_header = source.read(8)
225 |         if len(chunk_header) != 8:
226 |             raise MetadataStripError("Truncated PNG chunk header")
227 |
228 |         data_length = int.from_bytes(chunk_header[:4], "big")
229 |         chunk_type = chunk_header[4:]
230 |         keep_chunk = _keep_png_chunk(chunk_type)
231 |         chunk_name = chunk_type.decode("ascii", errors="replace")
232 |         if keep_chunk:
233 |             destination.write(chunk_header)
234 |
235 |         checksum = zlib.crc32(chunk_type)
236 |         remaining = data_length

```

```

237 |         while remaining:
238 |             data = source.read(min(remaining, _PNG_CHUNK_BUFFER_SIZE))
239 |             if not data:
240 |                 raise MetadataStripError(f"Truncated PNG {chunk_name} chunk")
241 |             checksum = zlib.crc32(data, checksum)
242 |             if keep_chunk:
243 |                 destination.write(data)
244 |                 remaining -= len(data)
245 |
246 |             stored_checksum = source.read(4)
247 |             if len(stored_checksum) != 4:
248 |                 raise MetadataStripError(f"Truncated PNG {chunk_name} checksum")
249 |             if checksum & 0xFFFFFFFF != int.from_bytes(stored_checksum, "big"):
250 |                 raise MetadataStripError(f"Invalid PNG {chunk_name} checksum")
251 |
252 |             if keep_chunk:
253 |                 destination.write(stored_checksum)
254 |             else:
255 |                 removed_items.append(chunk_name)
256 |
257 |             if chunk_type == b"IEND":
258 |                 break
259 |
260 |         return removed_items

```

Check the 8-byte signature and write it. Then for each chunk: read the 8-byte header; the first 4 bytes are the **data length**, the last 4 the **chunk type** (e.g. `b"IDAT"`). `_keep_png_chunk` decides if we keep it; if yes, the header is written immediately.

A running CRC32 starts from the chunk type (`zlib.crc32(chunk_type)`) and is updated with each piece of data (lines 235-244). Data is read in pieces of at most 64 KB so big images never fill memory; kept chunks are written piece by piece. After the data, 4 CRC bytes are read and compared with the computed value (`& 0xFFFFFFFF` forces an unsigned 32-bit number). A mismatch raises `MetadataStripError("Invalid PNG ... checksum")` — **this also detects corrupted files**. Kept chunks get their CRC written; dropped chunks add their name to `removed_items`. The loop ends after the `IEND` chunk. Nothing after `IEND` is copied.

Lines 263–272 `_keep_png_chunk`: the allow-list rule

```

263 | def _keep_png_chunk(chunk_type: bytes) -> bool:
264 |     """Return whether a PNG chunk is required or needed to render the image.
265 |
266 |     Known metadata and malformed chunk names are rejected; ancillary chunks
267 |     are kept only when listed in the rendering-preservation allowlist.
268 |     """
269 |     if chunk_type in _PNG_METADATA_CHUNKS or len(chunk_type) != 4:
270 |         return False
271 |     is_ancillary = bool(chunk_type[0] & 0x20)
272 |     return not is_ancillary or chunk_type in _PNG_PRESERVED AncILLARY_CHUNKS

```

First: if the chunk is in the always-remove set, or its name is not exactly 4 bytes → **drop**. Then it checks the PNG naming rule: the **5th bit of the first letter** says whether the chunk is *ancillary* (optional). Lower-case first letter (`bit & 0x20`) means optional; upper-case means *critical* (`IHDR`, `PLTE`, `IDAT`, `IEND`). The final line says: keep it if it is critical, **or** if it is on the preserved list. In plain words: unknown optional chunks are dropped (safer for privacy), critical ones are always kept.

Behaviour worth knowing

`PHYs` (pixel density, e.g. 300 DPI) is removed even though it is not personal. That changes the *print size* hint but not the pixels. Also, only chunks before the `IEND` are processed; the code stops at `IEND`.

3.6 core/audio.py — optional music and sound cues

`AudioController` hides all sound work behind a few safe methods. The rule of this file: **sound must never break the program**. `pygame` is imported only when first needed (lazy loading), and every call is wrapped in `try/except`.

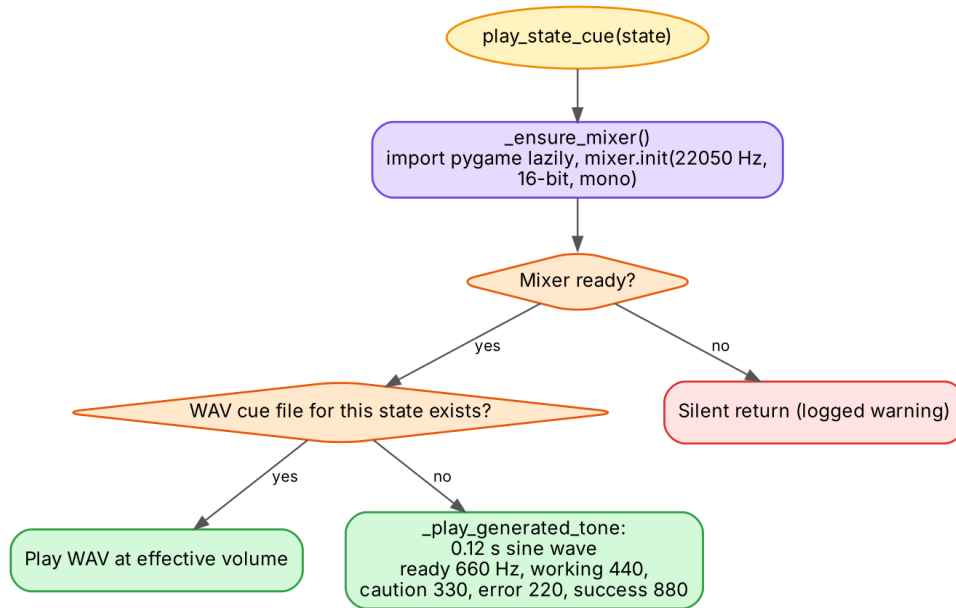


Figure 20. Flow of `play_state_cue`.

Variables of `AudioController`

Name	Type / default	Meaning
<code>_STATE_CUES</code>	class constant dict	state → WAV file name (<code>ready.wav</code> , <code>working.wav</code> , <code>caution.wav</code> , <code>error.wav</code> , <code>success.wav</code>).
<code>_STATE_FREQUENCIES</code>	class constant dict	state → generated tone in Hz: ready 660, working 440, caution 330, error 220, success 880.
<code>_audio_directory</code>	Path	Folder with the audio files (<code>assets/audio</code>).
<code>_pygame</code>	ModuleType None	The imported <code>pygame</code> module (None until loaded).
<code>_mixer_ready</code>	bool False	True after the mixer initialised successfully.
<code>_startup_attempted</code>	bool False	Makes the startup music play at most once.
<code>_volume</code>	float 0.7	Chosen volume 0.0–1.0.
<code>_muted</code>	bool False	When true the effective volume is 0.

Lines 1–46 Header, constants and `__init__`

```

1 | """Optional pygame-backed startup music and state cues."""
2 |
3 | from __future__ import annotations
4 |
5 | import importlib
6 | import logging
7 | import math
8 | import os
9 | from array import array
10 | from pathlib import Path
11 | from types import ModuleType
12 | from typing import Final

```

```

13 |
14 | logger = logging.getLogger(__name__)
15 |
16 |
17 | class AudioController:
18 |     """Safely manage optional startup music, state cues, and volume."""
19 |
20 |     _STATE_CUES: Final[dict[str, str]] = {
21 |         "ready": "ready.wav",
22 |         "working": "working.wav",
23 |         "caution": "caution.wav",
24 |         "error": "error.wav",
25 |         "success": "success.wav",
26 |     }
27 |     _STATE_FREQUENCIES: Final[dict[str, int]] = {
28 |         "ready": 660,
29 |         "working": 440,
30 |         "caution": 330,
31 |         "error": 220,
32 |         "success": 880,
33 |     }
34 |
35 |     def __init__(self, audio_directory: Path) -> None:
36 |         """Store the asset directory and initialize playback state.
37 |
38 |         The pygame mixer is loaded lazily when playback is first requested.
39 |         """
40 |         self._audio_directory = audio_directory
41 |         self._pygame: ModuleType | None = None
42 |         self._mixer_ready = False
43 |         self._startup_attempted = False
44 |         self._volume = 0.7
45 |         self._muted = False
46 |

```

`Final[...]` documents that the dictionaries are not meant to be changed. `importlib` is used to import pygame *by name at run time*; this keeps the program working when pygame is missing. `array` and `math` are used to build the generated beep. The constructor only stores values — **no sound hardware is touched yet**.

Lines 47–63 `_ensure_mixer`: the lazy loader

```

47 |     def _ensure_mixer(self) -> bool:
48 |         """Load pygame and initialize its mixer, returning whether it is ready."""
49 |         if self._mixer_ready:
50 |             return True
51 |         try:
52 |             os.environ["PYGAME_HIDE_SUPPORT_PROMPT"] = "1"
53 |             if self._pygame is None:
54 |                 self._pygame = importlib.import_module("pygame")
55 |                 mixer = getattr(self._pygame, "mixer")
56 |                 if not mixer.get_init():
57 |                     mixer.init(frequency=22_050, size=-16, channels=1)
58 |                 self._mixer_ready = True
59 |                 self._apply_volume()
60 |                 return True
61 |         except Exception:
62 |             logger.warning("Audio is unavailable; continuing without sound", exc_info=True)
63 |             return False

```

If the mixer is already ready, return `True` at once. Otherwise: set the environment variable

`PYGAME_HIDE_SUPPORT_PROMPT=1` (silences pygame's banner), import pygame once, and if the mixer is not initialised start it with `**22,050 Hz, 16-bit signed samples (size=-16), 1 channel (mono)**`. Mark ready and apply the volume. Any exception is logged as a warning and `False` is returned — the caller then simply stays silent.

Lines 65–103 `play_startup` and `play_state_cue`

```

65 |     def play_startup(self) -> None:
66 |         """Play the startup track once, when the main window is shown."""
67 |         if self._startup_attempted:
68 |             return
69 |         self._startup_attempted = True
70 |         startup_track = self._audio_directory / "main_theme.mp3"
71 |         if not startup_track.is_file():
72 |             logger.info("Startup audio not found at %s", startup_track)
73 |             return
74 |         if not self._ensure_mixer():
75 |             return
76 |         try:
77 |             mixer = getattr(self._pygame, "mixer")
78 |             mixer.music.load(str(startup_track))
79 |             mixer.music.set_volume(self._effective_volume())
80 |             mixer.music.play(loops=-1)
81 |             logger.info("Playing startup audio %s", startup_track.name)
82 |         except Exception:
83 |             logger.warning("Unable to play startup audio", exc_info=True)
84 |
85 |     def play_state_cue(self, state: str) -> None:
86 |         """Play the WAV cue for ``state`` or synthesize its configured tone.
87 |
88 |         Unknown states have no generated fallback tone.
89 |         """
90 |         cue_name = self._STATE_CUES.get(state)
91 |         cue_path = self._audio_directory / cue_name if cue_name else None
92 |         if not self._ensure_mixer():
93 |             return
94 |         try:
95 |             mixer = getattr(self._pygame, "mixer")
96 |             if cue_path is not None and cue_path.is_file():
97 |                 sound = mixer.Sound(str(cue_path))
98 |                 sound.set_volume(self._effective_volume())
99 |                 sound.play()
100 |             else:
101 |                 self._play_generated_tone(mixer, state)
102 |         except Exception:
103 |             logger.warning("Unable to play %s audio cue", state, exc_info=True)

```

`play_startup` runs once (guard `_startup_attempted`). It looks for `main_theme.mp3`; if missing, logs and returns. After the mixer is ready it loads the file into pygame's **music** player, sets the volume and plays with `loops=-1` (endless loop).

`play_state_cue(state)` looks up the WAV name for the state. After making sure the mixer is ready it plays that WAV as a **Sound** (short effect, at the effective volume). If the WAV file does not exist, it falls back to `_play_generated_tone`. Unknown states do nothing. Errors only produce a warning.

Lines 105–130 `_play_generated_tone`: building a beep from maths

```

105 |     def _play_generated_tone(self, mixer: object, state: str) -> None:
106 |         """Play a short sine-wave cue when ``state`` has a configured frequency.
107 |
108 |         Generated audio is skipped unless the mixer uses signed 16-bit samples.
109 |         """
110 |         frequency = self._STATE_FREQUENCIES.get(state)
111 |         if frequency is None:
112 |             return
113 |         sample_rate, sample_format, channels = getattr(mixer, "get_init")()
114 |         if sample_format != -16:
115 |             logger.debug("Skipping generated audio cue for unsupported mixer format %s", sample_format)
116 |             return
117 |
118 |         sample_count = int(sample_rate * 0.12)
119 |         samples = array(
120 |             "h",
121 |             (
122 |                 int(7000 * math.sin(2 * math.pi * frequency * index / sample_rate))
123 |                 for index in range(sample_count)
124 |             ),

```

```

125 |         )
126 |         if channels > 1:
127 |             samples = array("h", (sample for value in samples for sample in (value,) * channels))
128 |             sound = getattr(mixer, "Sound")(buffer=samples.tobytes())
129 |             sound.set_volume(self._effective_volume())
130 |             sound.play()

```

Gets the frequency for the state; none → return. Reads the mixer settings (`sample_rate`, `sample_format`, `channels`); if the sample format is not `-16` (signed 16-bit) it refuses (the maths below produces 16-bit numbers). Length: `sample_rate * 0.12` samples = **0.12 seconds**. Each sample is `7000 * sin(2π · frequency · i / sample_rate)` — a pure **sine wave** with amplitude 7000 out of a possible 32767 (quiet, about 21%). They are stored in a typed `array("h")` ("h" = signed 16-bit). For stereo, every sample is repeated once per channel. The raw bytes (`tobytes()`) are given to `mixer.Sound(buffer=...)`, the volume is set and it plays.

Lines 132–162 Volume, mute and shutdown

```

132 |     def set_volume(self, volume: float) -> None:
133 |         """Clamp and store playback volume in the inclusive range zero to one."""
134 |         self._volume = max(0.0, min(1.0, volume))
135 |         self._apply_volume()
136 |
137 |     def set_muted(self, muted: bool) -> None:
138 |         """Mute or restore the configured playback volume."""
139 |         self._muted = muted
140 |         self._apply_volume()
141 |
142 |     def _effective_volume(self) -> float:
143 |         """Return the chosen volume, or zero when audio is muted."""
144 |         return 0.0 if self._muted else self._volume
145 |
146 |     def _apply_volume(self) -> None:
147 |         """Send the effective volume to pygame when the mixer is available."""
148 |         if self._mixer_ready and self._pygame is not None:
149 |             try:
150 |                 getattr(self._pygame, "mixer").music.set_volume(self._effective_volume())
151 |             except Exception:
152 |                 logger.debug("Unable to update audio volume", exc_info=True)
153 |
154 |     def shutdown(self) -> None:
155 |         """Release the mixer without making shutdown failure fatal."""
156 |         if not self._mixer_ready or self._pygame is None:
157 |             return
158 |         try:
159 |             getattr(self._pygame, "mixer").quit()
160 |         except Exception:
161 |             logger.debug("Audio mixer shutdown failed", exc_info=True)
162 |         self._mixer_ready = False

```

`set_volume` clamps to 0–1 with `max(0.0, min(1.0, volume))`. `set_muted` stores the flag. Both call `_apply_volume`, which pushes `_effective_volume()` (0 when muted) to pygame's music channel *only if the mixer is ready*. Note: this changes the **background music**; state cues read the effective volume each time they are played. `shutdown` quits the mixer (errors ignored) and marks it not ready; called from the window's `closeEvent`.

3.7 workers/image_worker.py — the batch engine

`ImageWorker` is a `QThread`. When the window calls `.start()`, Qt creates a new operating-system thread and runs the method `run()` inside it. All heavy file work happens here. The class has two big paths: **in-place** (`run`) and **ZIP export** (`_run_zip_export`).

Module-level names

Name	Value	Meaning
<code>logger</code>	<code>logging.getLogger(__name__)</code>	Logger <code>workers.image_worker</code> .
<code>IMAGE_EXTENSIONS</code>	frozenset of <code>.jpg .jpeg .png .webp .bmp .tiff .tif .gif</code>	The only extensions that count as images.
<code>WorkerState</code>	<code>Literal["ready", "working", "caution", "error", "success"]</code>	Allowed state keys.
<code>ERROR_ISOLATION_TAG</code>	<code>"[ERROR ISOLATION] ERROR: A MUTANT FILE FOUGHT BACK"</code>	Banner prefix used for error messages (shared with the window).
<code>BatchCancelled</code>	exception class	Raised internally when the user cancels.

Attributes of ImageWorker

Attribute	Type	Meaning
<code>folder</code>	<code>Path</code>	Folder to process.
<code>mode</code>	<code>RenameMode</code>	<code>default</code> , <code>prefix</code> or <code>date</code> .
<code>prefix</code>	<code>str</code>	Text for prefix mode.
<code>keep_backup</code>	<code>bool</code>	True: persistent ZIP backup; False: temporary shadow copy.
<code>output_archive_path</code>	<code>Path None</code>	If set, ZIP export mode is used.
<code>strip_metadata_enabled</code>	<code>bool</code>	Clean metadata? Forced to True when exporting a ZIP.
<code>_requested_files</code>	<code>tuple[Path] None</code>	If given (retry), only these files are processed; None means scan the folder.
<code>failed_files</code>	<code>list[Path]</code>	Files that failed (used for retry).
<code>result_success</code>	<code>bool</code>	Final success flag.
<code>result_message</code>	<code>str</code>	Final message.

Lines 1–27 Imports and module constants

```

1 | """Cancellable background transactions for image batches."""
2 |
3 | from __future__ import annotations
4 |
5 | import logging
6 | import os
7 | import shutil
8 | import tempfile
9 | import zipfile
10 | from pathlib import Path
11 | from typing import Literal, Sequence
12 |
13 | from PySide6.QtCore import QThread, Signal
14 |
15 | from core.backup_manager import BackupManager, BackupSnapshot
16 | from core.metadata import strip_metadata
17 | from core.renamer import RenameMode, plan_renames
18 |
19 | logger = logging.getLogger(__name__)
20 | IMAGE_EXTENSIONS = frozenset({".jpg", ".jpeg", ".png", ".webp", ".bmp", ".tiff", ".tif", ".gif"})
21 | WorkerState = Literal["ready", "working", "caution", "error", "success"]
22 | ERROR_ISOLATION_TAG = "[ERROR ISOLATION] ERROR: A MUTANT FILE FOUGHT BACK"
23 |

```

```

24 |
25 | class BatchCancelled(Exception):
26 |     """Raised internally when the user requests a safe batch cancellation."""
27 |

```

Sequence is a hint for any list-like input. **QThread** and **Signal** come from Qt. The worker imports the three core helpers it needs: the backup manager, **strip_metadata** and the planner. **ERROR_ISOLATION_TAG** is defined here (not in the window) so both sides use the same text.

Lines 29–65 Class header, signals and constructor

```

29 | class ImageWorker(QThread):
30 |     """Process a folder while keeping the UI thread responsive.
31 |
32 |     Each image is handled independently. Failed images remain untouched and
33 |     later images continue; cancelling or a batch-level failure restores every
34 |     input from the verified pre-processing backup.
35 |     """
36 |
37 |     progress_changed = Signal(int)
38 |     status_changed = Signal(str)
39 |     state_changed = Signal(str)
40 |     file_failed = Signal(str, str)
41 |     completed = Signal(bool, str)
42 |
43 |     def __init__(
44 |         self,
45 |         folder: str | Path,
46 |         mode: RenameMode = "default",
47 |         prefix: str = "",
48 |         keep_backup: bool = True,
49 |         strip_metadata_enabled: bool = True,
50 |         output_archive_path: str | Path | None = None,
51 |         parent=None,
52 |         files: Sequence[Path] | None = None,
53 |     ) -> None:
54 |         """Store batch options and initialize the worker's result state."""
55 |         super().__init__(parent)
56 |         self.folder = Path(folder)
57 |         self.mode = mode
58 |         self.prefix = prefix
59 |         self.keep_backup = keep_backup
60 |         self.output_archive_path = Path(output_archive_path) if output_archive_path else None
61 |         self.strip_metadata_enabled = strip_metadata_enabled or self.output_archive_path is not None
62 |         self._requested_files = tuple(Path(path) for path in files) if files is not None else None
63 |         self.failed_files: list[Path] = []
64 |         self.result_success = False
65 |         self.result_message = ""

```

Five signals are declared **at class level** — this is how Qt requires it. **Signal(int)** means the signal carries one integer; **Signal(bool, str)** carries a boolean and a string. The constructor stores all options. Line 60 turns the optional archive path into a **Path**. **Line 61 is important:** **strip_metadata_enabled or self.output_archive_path is not None** means ZIP mode *forces* cleaning on. Line 62 converts the optional **files** list (used by retry) into a tuple of paths.

Lines 67–95 run(): choose the path and collect images

```

67 |     def run(self) -> None:
68 |         """Run the in-place batch or hand ZIP requests to the export path."""
69 |         if self.output_archive_path is not None:
70 |             self._run_zip_export()
71 |             return
72 |
73 |         manager = BackupManager()
74 |         snapshot: BackupSnapshot | None = None
75 |         changed_paths: list[Path] = []
76 |
77 |         try:
78 |             candidates = (
79 |                 self._requested_files

```

```

80 |         if self._requested_files is not None
81 |             else tuple(self.folder.iterdir())
82 |     )
83 |     files = sorted(
84 |         (
85 |             path
86 |             for path in candidates
87 |             if path.parent.resolve() == self.folder.resolve()
88 |             and path.is_file()
89 |             and not path.is_symlink()
90 |             and path.suffix.lower() in IMAGE_EXTENSIONS
91 |         ),
92 |         key=lambda path: path.name.casefold(),
93 |     )
94 |     if not files:
95 |         raise ValueError("No supported images were found in that folder")

```

If an archive path exists the work is handed to `_run_zip_export` and `run` returns. Otherwise the in-place path starts with three local variables: a `BackupManager`, `snapshot` (None for now) and `changed_paths` (every file the batch may modify; used by rollback). All following work sits inside one big `try` so any failure can trigger rollback.

The candidate list is either the retry files or everything in the folder (`folder.iterdir()`). The filter keeps a path only if **(a)** its parent is exactly the chosen folder (so retry cannot sneak in files from elsewhere), **(b)** it is a file, **(c)** it is not a symbolic link, and **(d)** its lowercase extension is in `IMAGE_EXTENSIONS`. The result is sorted case-insensitively. No images → `ValueError("No supported images were found in that folder")`, which ends in the “error” state.

Lines 97–111 Backup and plan

```

97 |         self._set_state("caution", "Making sure your original files are backed up...")
98 |         snapshot = manager.create_backup(files, keep_archive=self.keep_backup)
99 |         if snapshot.archive_path is not None:
100 |             logger.info("Verified original backup retained at %s", snapshot.archive_path)
101 |         else:
102 |             logger.info("Verified temporary rollback copy is ready")
103 |
104 |         operations = plan_renames(
105 |             files,
106 |             self.mode,
107 |             self.prefix,
108 |             protect_source_names=True,
109 |         )
110 |         changed_paths = [operation.source for operation in operations]
111 |         self._set_state("working", "Backup is ready. Renaming your images...")

```

Shows the *caution* state (the “time capsule” picture), creates the verified backup, logs where it is, creates the rename plan with `protect_source_names=True`, and fills `changed_paths` with all source files. Then the state changes to *working* with the message “Backup is ready. Renaming your images...”.

Lines 113–175 The per-file loop (heart of the program)

```

113 |         failures: list[tuple[Path, str]] = []
114 |         succeeded = 0
115 |         total = len(operations)
116 |         for index, operation in enumerate(operations, start=1):
117 |             if self.isInterruptionRequested():
118 |                 raise BatchCancelled("Cancellation requested")
119 |
120 |             work_path: Path | None = None
121 |             destination_created = False
122 |             try:
123 |                 descriptor, work_name = tempfile.mkstemp(
124 |                     prefix=".RENAMER-work-",
125 |                     suffix=operation.source.suffix,
126 |                     dir=operation.source.parent,
127 |                 )
128 |                 os.close(descriptor)
129 |                 work_path = Path(work_name)
130 |                 changed_paths.append(work_path)

```

```

131 |         self.status_changed.emit(f"Renaming {operation.source.name}...")
132 |         shutil.copy2(operation.source, work_path)
133 |         if self.strip_metadata_enabled:
134 |             strip_metadata(work_path)
135 |
136 |         if operation.destination == operation.source:
137 |             os.replace(work_path, operation.destination)
138 |         else:
139 |             self._publish_without_overwrite(work_path, operation.destination)
140 |             destination_created = True
141 |             changed_paths.append(operation.destination)
142 |             try:
143 |                 operation.source.unlink()
144 |             except OSError:
145 |                 operation.destination.unlink(missing_ok=True)
146 |                 destination_created = False
147 |                 changed_paths.remove(operation.destination)
148 |                 raise
149 |
150 |         succeeded += 1
151 |         logger.info(
152 |             "Processed image %s as %s",
153 |             operation.source.name,
154 |             operation.destination.name,
155 |         )
156 |     except Exception as error:
157 |         if work_path is not None:
158 |             work_path.unlink(missing_ok=True)
159 |             changed_paths.remove(work_path)
160 |         if destination_created:
161 |             operation.destination.unlink(missing_ok=True)
162 |             if operation.destination in changed_paths:
163 |                 changed_paths.remove(operation.destination)
164 |         failures.append((operation.source, str(error)))
165 |         self.failed_files.append(operation.source)
166 |         self.file_failed.emit(str(operation.source), str(error))
167 |         self._set_state(
168 |             "error",
169 |             f"{ERROR_ISOLATION_TAG}\nSkipped {operation.source.name}; continuing batch.",
170 |         )
171 |         logger.exception("Skipped image %s after processing error", operation.source)
172 |         self._set_state("working", f"Moving on to {operation.source.name}...")
173 |
174 |     self.progress_changed.emit(round(index * 100 / total))
175 |

```

For every operation: (1) If cancellation was requested raise **BatchCancelled**. (2) Create a **hidden work file** **.RENAMER-work-xxxx.ext** in the source folder; **mkstemp** returns an open descriptor which is closed at once (only the *name* is needed) and the work path is added to **changed_paths** so a rollback would delete it. (3) Emit "Renaming <name>...". (4) **copy2** the source into the work file. (5) If cleaning is enabled, **strip_metadata(work_path)**.

(6) Publish: if the destination equals the source (the file already had the right name) **os.replace(work, destination)** simply swaps the cleaned copy over the original. Otherwise **_publish_without_overwrite(work, destination)** puts the work file under the new name, then the old source is deleted (line 143). If deleting the source fails, the new destination is removed again, removed from **changed_paths**, and the error is re-raised so the file counts as failed — **the original is never lost**.

(7) On success **succeeded** goes up and a log line is written. **On any exception** (lines 156-172): delete the work file, delete a destination that was already created, remember the failure (list + **failed_files** + signal), flash the *error* state with the banner and "Skipped <name>; continuing batch.", log the traceback, then switch back to *working*. The loop **continues** with the next file. Finally, progress is emitted: **round(index * 100 / total)**.

Lines 176–206 After the loop, and the safety `except`

```

176 |         if self.isInterruptRequested():
177 |             raise BatchCancelled("Cancellation requested")
178 |
179 |         if failures:
180 |             message = (
181 |                 f"Completed {succeeded} of {total} images; "
182 |                 f"{len(failures)} failed. See the log for details."
183 |             )
184 |         else:
185 |             message = f"Processed {succeeded} image(s) successfully"
186 |
187 |         manager.discard_temporary_backup(snapshot)
188 |         final_state: WorkerState = "error" if failures else "success"
189 |         self._finish(not failures, message, final_state)
190 |     except Exception as error:
191 |         cancelled = isinstance(error, BatchCancelled)
192 |         if cancelled:
193 |             self._set_state("caution", "Stopping and restoring your original files...")
194 |             logger.info("Batch cancelled; restoring original files")
195 |         else:
196 |             logger.exception("Batch processing aborted")
197 |
198 |         if snapshot is not None:
199 |             try:
200 |                 manager.rollback(snapshot, changed_paths)
201 |             except Exception:
202 |                 logger.exception("Unable to restore the full batch from backup")
203 |                 error = RuntimeError(f"{error}; rollback failed; see log for details")
204 |
205 |         message = "Cancelled; original files restored" if cancelled else f"Batch failed: {error}"
206 |         self._finish(False, message, "error")

```

One more cancellation check (so a cancel during the last file is honoured). Then the final message: with failures “Completed X of N images; Y failed. See the log for details.”, else “Processed N image(s) successfully”. The temporary backup is discarded (ZIP backups stay), and `_finish` is called. Note: **partial success is reported as an error state** (`final_state = "error" if failures`), and `succeeded=False`.

The outer `except` handles two situations. **Cancelled** (`BatchCancelled`): show *caution* “Stopping and restoring your original files...”. **Any other batch-level failure** (e.g. backup failed): log with traceback. If a snapshot exists, `manager.rollback(snapshot, changed_paths)` is run; if the rollback itself fails the error text becomes “... ; rollback failed; see log for details”. The final message is either “Cancelled; original files restored” or “Batch failed: <error>” and the state is *error*.

Important behaviour

Individual **file** failures do *not* roll back the batch (the failed file simply stays unchanged and the others are kept). Only a **batch-level** error or a **cancel** triggers the full rollback. Also, if a batch finishes with failures, the backup ZIP (if persistent) remains so the user can compare.

Lines 208–257 `_run_zip_export`, part 1: stage copies

```

208 |     def _run_zip_export(self) -> None:
209 |         """Create a processed ZIP from staged copies, leaving source files untouched."""
210 |         archive_path = self.output_archive_path
211 |         temporary_archive: Path | None = None
212 |         try:
213 |             if archive_path is None:
214 |                 raise ValueError("ZIP export requires an output archive path")
215 |             if not archive_path.parent.is_dir():
216 |                 raise ValueError("The ZIP destination folder does not exist")
217 |
218 |             candidates = (
219 |                 self._requested_files
220 |                 if self._requested_files is not None
221 |                 else tuple(self.folder.iterdir())
222 |             )
223 |             files = sorted(

```

```

224 |         (
225 |             path
226 |             for path in candidates
227 |             if path.parent.resolve() == self.folder.resolve()
228 |             and path.is_file()
229 |             and not path.is_symlink()
230 |             and path.suffix.lower() in IMAGE_EXTENSIONS
231 |         ),
232 |         key=lambda path: path.name.casefold(),
233 |     )
234 | if not files:
235 |     raise ValueError("No supported images were found in that folder")
236 |
237 | self._set_state("working", "Getting your renamed pictures ready for the ZIP...")
238 | with tempfile.TemporaryDirectory(prefix="RENAMER_export_") as staging_name:
239 |     staging = Path(staging_name)
240 |     staged_sources: list[Path] = []
241 |     for source in files:
242 |         if self.isInterruptionRequested():
243 |             raise BatchCancelled("Cancellation requested")
244 |         staged = staging / source.name
245 |         shutil.copy2(source, staged)
246 |         staged_sources.append(staged)
247 |
248 |     operations = plan_renames(
249 |         staged_sources,
250 |         self.mode,
251 |         self.prefix,
252 |         protect_source_names=True,
253 |     )
254 |     total = len(operations)
255 |     for index, operation in enumerate(operations, start=1):
256 |         if self.isInterruptionRequested():
257 |             raise BatchCancelled("Cancellation requested")

```

Checks that an archive path exists and that its folder exists. Collects and filters images with the **same filter** as the in-place path. Then it makes a **temporary staging folder** `RENAMER_export_*` (a `TemporaryDirectory`, so it is deleted automatically at the end of the `with` block even on errors) and copies every original into it with `copy2`, checking for cancel each time. The rename plan is made over the **staged copies**, so the user's folder is never involved again — originals cannot be changed.

Lines 258–282 `_run_zip_export`, part 2: process each staged file

```

258 |         work_path: Path | None = None
259 |         try:
260 |             descriptor, work_name = tempfile.mkstemp(
261 |                 prefix=".RENAMER-work-",
262 |                 suffix=operation.source.suffix,
263 |                 dir=staging,
264 |             )
265 |             os.close(descriptor)
266 |             work_path = Path(work_name)
267 |             shutil.copy2(operation.source, work_path)
268 |             strip_metadata(work_path)
269 |             if operation.destination == operation.source:
270 |                 os.replace(work_path, operation.destination)
271 |             else:
272 |                 self._publish_without_overwrite(work_path, operation.destination)
273 |                 operation.source.unlink()
274 |                 self.status_changed.emit(f"Adding {operation.destination.name} to the ZIP...")
275 |         except Exception as error:
276 |             if work_path is not None:
277 |                 work_path.unlink(missing_ok=True)
278 |             self.failed_files.append(self.folder / operation.source.name)
279 |             self.file_failed.emit(str(self.folder / operation.source.name), str(error))
280 |             raise RuntimeError(f"Unable to process {operation.source.name}: {error}") from error
281 |         self.progress_changed.emit(round(index * 90 / total))
282 |

```

The same work-file → clean → publish pattern, but inside the staging folder, and cleaning is **always** performed (`strip_metadata(work_path)` has no condition). Here a failure **stops the whole export**: the exception handler

records the file (using the *original* folder + name), emits `file_failed` and raises a `RuntimeError("Unable to process ...")`. Progress here goes only to **90%** (`index * 90 / total`); the last 10% is reserved for building the ZIP.

Lines 283–325 `_run_zip_export`, part 3: build, verify, replace

```

283 |         if self.isInterruptionRequested():
284 |             raise BatchCancelled("Cancellation requested")
285 |
286 |         descriptor, temporary_name = tempfile.mkstemp(
287 |             prefix=f"{archive_path.stem}.", suffix=".tmp", dir=archive_path.parent
288 |         )
289 |         os.close(descriptor)
290 |         temporary_archive = Path(temporary_name)
291 |         with zipfile.ZipFile(
292 |             temporary_archive,
293 |             "w",
294 |             compression=zipfile.ZIP_DEFLATED,
295 |             compresslevel=6,
296 |         ) as archive:
297 |             for operation in operations:
298 |                 archive.write(
299 |                     operation.destination,
300 |                     arcname=operation.destination.name,
301 |                 )
302 |         with zipfile.ZipFile(temporary_archive, "r") as archive:
303 |             corrupt_member = archive.testzip()
304 |             if corrupt_member is not None:
305 |                 raise OSError(f"ZIP verification failed for {corrupt_member}")
306 |         os.replace(temporary_archive, archive_path)
307 |         temporary_archive = None
308 |
309 |         self.progress_changed.emit(100)
310 |         self._finish(
311 |             True,
312 |             f"Exported {len(files)} processed image(s) to {archive_path}",
313 |             "success",
314 |         )
315 |     except Exception as error:
316 |         if temporary_archive is not None:
317 |             temporary_archive.unlink(missing_ok=True)
318 |         cancelled = isinstance(error, BatchCancelled)
319 |         if cancelled:
320 |             message = "ZIP export cancelled; original files were not changed"
321 |             self._finish(False, message, "caution")
322 |         else:
323 |             logger.exception("ZIP export failed")
324 |             self._finish(False, f"ZIP export failed: {error}", "error")
325 |

```

After a last cancel check, a temporary archive named `.<zipname>.xxxx.tmp` is created **next to the final ZIP**. All renamed files are written with **ZIP_DEFLATED, compresslevel 6** (normal compression).

`arcname=destination.name` stores each file at the ZIP's top level with its new name. The archive is reopened and `testzip()` is run; a problem raises `OSError`. Then `os.replace(temporary_archive, archive_path)` atomically makes the final ZIP appear, and `temporary_archive = None` so cleanup does not delete it. Progress jumps to 100 and `_finish` reports "Exported N processed image(s) to <path>".

If anything fails: the temporary ZIP is deleted. Cancel → state *caution*, message "ZIP export cancelled; original files were not changed". Other errors → state *error*, message "ZIP export failed: ...".

Lines 326–351 `_publish_without_overwrite`, `_set_state` and `_finish`

```

326 | @staticmethod
327 | def _publish_without_overwrite(work_path: Path, destination: Path) -> None:
328 |     """Reserve a new destination exclusively, then atomically publish it."""
329 |     descriptor = os.open(
330 |         destination,
331 |         os.O_CREAT | os.O_EXCL | os.O_WRONLY,
332 |         0o600,
333 |     )
334 |     os.close(descriptor)
335 |     try:
336 |         os.replace(work_path, destination)
337 |     except Exception:
338 |         destination.unlink(missing_ok=True)
339 |         raise
340 |
341 | def _set_state(self, state: WorkerState, message: str) -> None:
342 |     """Send a state key and its human-readable status to the UI."""
343 |     self.state_changed.emit(state)
344 |     self.status_changed.emit(message)
345 |
346 | def _finish(self, succeeded: bool, message: str, state: WorkerState) -> None:
347 |     """Store the final result and notify connected UI slots."""
348 |     self.result_success = succeeded
349 |     self.result_message = message
350 |     self._set_state(state, message)
351 |     self.completed.emit(succeeded, message)

```

`_publish_without_overwrite` is a clever safety trick. `os.replace` would silently overwrite an existing file. To prevent this, the destination is first *created exclusively*: `os.open(destination, O_CREAT | O_EXCL | O_WRONLY, 0o600)`. `O_EXCL` makes the operating system **fail if the file already exists**. If creation succeeds, the empty placeholder is closed and `os.replace` swaps the work file onto it. If the replace fails, the placeholder is deleted. This closes the small time gap between “I checked the name is free” and “I use the name”.

`_set_state` emits two signals: the state key and a message. `_finish` stores `result_success` and `result_message`, sets the final state and emits `completed(succeeded, message)`.

Decision table: what happens on each event (in-place mode)

Event	Result	Files after	State shown
No image in folder	<code>ValueError</code> before backup	Untouched	error
Backup creation fails	Batch failure, no snapshot to restore	Untouched	error
One file fails	Skipped, batch goes on	That file unchanged, others renamed	error at the end
All files fine	Success	All renamed	success
User presses Cancel	Rollback	Originals restored	caution then error (“Cancelled...”)
Unexpected error mid-run	Rollback	Originals restored	error
Rollback itself fails	Message says rollback failed	Maybe mixed: use the backup ZIP	error

3.8 ui/main_window.py — the whole window

The biggest file (721 lines). It builds every widget, applies a dark green theme, listens to the worker's signals, fades pictures, shows logs, and makes sure closing the window never loses data. It contains one constant, three helper classes and the main class `RenamerWindow`.

Names at module level

Name	Kind	Purpose
<code>logger</code>	variable	Logger <code>ui.main_window</code> .
<code>STATE_PRESENTATION</code>	<code>dict[str, (banner, (image_names...))]</code>	For each of the 5 states: banner text and the list of picture file names to try in order.
<code>_UiLogEmitter</code>	<code>QObject</code>	Owns one signal <code>message_received(str)</code> so log text can reach the GUI thread safely.
<code>_UiLogHandler</code>	<code>logging.Handler</code>	Copies INFO+ log records into the emitter.
<code>_StateImageLabel</code>	<code>QLabel</code>	A label that emits <code>resized()</code> whenever its size changes.
<code>RenamerWindow</code>	<code>QMainWindow</code>	The window itself.

Lines 1–57 Imports and the state table

```

1 | """Two-column PySide6 dashboard for RENAMER."""
2 |
3 | from __future__ import annotations
4 |
5 | import logging
6 | import os
7 | from pathlib import Path
8 | from typing import Final
9 |
10 | from PySide6.QtCore import QEasingCurve, QObject, QPropertyAnimation, QTimer, Qt, Signal
11 | from PySide6.QtGui import QCloseEvent, QFont, QIcon, QPixmap, QResizeEvent
12 | from PySide6.QtWidgets import (
13 |     QButtonGroup,
14 |     QFileDialog,
15 |     QFrame,
16 |     QGraphicsOpacityEffect,
17 |     QGroupBox,
18 |     QHBoxLayout,
19 |     QLabel,
20 |     QLineEdit,
21 |     QMainWindow,
22 |     QPlainTextEdit,
23 |     QProgressBar,
24 |     QPushButton,
25 |     QSizePolicy,
26 |     QSlider,
27 |     QVBoxLayout,
28 |     QWidget,
29 | )
30 |
31 | from core.audio import AudioController
32 | from workers.image_worker import ERROR_ISOLATION_TAG, ImageWorker
33 |
34 | logger = logging.getLogger(__name__)
35 |
36 | STATE_PRESENTATION: Final[dict[str, tuple[str, tuple[str, ...]]]] = {
37 |     "ready": (
38 |         "[LAB PROTOCOL] HERE YOU ARE, LAD - LET'S DO SOME SCIENCE",
39 |         ("here you re lad .jpg",),
40 |     ),
41 |     "working": (
42 |         "[WORKER THREAD] OH WAIT, WORKING ON IT... (DON'T TOUCH THE DIAL)",

```

```

43 |         ("OH WAIT working on it.jpg"),
44 |     ),
45 |     "caution": (
46 |         "[BACKUP ENGINE] CAUTION: TIME CAPSULE SEALED",
47 |         ("caution.jpg"),
48 |     ),
49 |     "error": (
50 |         "[ERROR ISOLATION] ERROR: A MUTANT FILE FOUGHT BACK",
51 |         ("errore.jpg", "error.jpg"),
52 |     ),
53 |     "success": (
54 |         "[SUCCESS] LABORATORY UNSCATHED - MANDARK IS CRYING",
55 |         ("Happy for using my tool xD.jpg"),
56 |     ),
57 | }

```

The imports list every Qt widget class used: buttons, labels, line edits, a slider, a progress bar, group boxes, layouts, the file dialog, an opacity effect and a property animation for the fade. `STATE_PRESENTATION` maps a state key to a **tuple of (banner text, tuple of picture names)**. The error state lists `errore.jpg` first and `error.jpg` second, so a nicer picture can be dropped in later; today only `error.jpg` exists, so the second is used. The code later picks the **first file that exists**.

Lines 60–91 The three tiny helper classes

```

60 | class _UiLogEmitter(QObject):
61 |     """Bridge Python log records onto the Qt UI thread."""
62 |
63 |     message_received = Signal(str)
64 |
65 |
66 | class _UiLogHandler(logging.Handler):
67 |     """Forward formatted application records to the log panel."""
68 |
69 |     def __init__(self, emitter: _UiLogEmitter) -> None:
70 |         """Connect this logging handler to the UI-thread signal emitter."""
71 |         super().__init__(logging.INFO)
72 |         self._emitter = emitter
73 |
74 |     def emit(self, record: logging.LogRecord) -> None:
75 |         """Format a log record and forward it to the read-only log panel."""
76 |         try:
77 |             self._emitter.message_received.emit(self.format(record))
78 |         except Exception:
79 |             self.handleError(record)
80 |
81 |
82 | class _StateImageLabel(QLabel):
83 |     """Notify the window when the available image area changes size."""
84 |
85 |     resized = Signal()
86 |
87 |     def resizeEvent(self, event) -> None:
88 |         """Notify the window after the preview label changes size."""
89 |         super().resizeEvent(event)
90 |         self.resized.emit()
91 |

```

Why do they exist? Python's logging may be called from *any thread* (the worker logs a lot). Qt widgets may only be changed from the GUI thread. `_UiLogHandler.emit` formats the record and sends it through the signal `message_received`; Qt delivers it to the GUI thread where `log_view.appendPlainText` shows it. `emit` is wrapped in `try/except` and calls the standard `handleError` so a logging problem cannot crash the program. `_UiLogHandler` starts with level INFO (`super().__init__(logging.INFO)`).

`_StateImageLabel` only adds one thing: after the normal resize handling it emits `resized`, which the window uses to re-scale the picture.

Instance attributes of RenamerWindow

Attribute	Type / start value	Meaning
<code>_folder</code>	<code>Path</code> <code>None</code>	The validated folder; <code>None</code> disables the main button.
<code>_worker</code>	<code>ImageWorker</code> <code>None</code>	The running worker; <code>None</code> when idle.
<code>_failed_files</code>	<code>tuple[Path, ...]</code> empty	Failures from the last run, for retry.
<code>_close_when_worker_stops</code>	<code>bool</code> <code>False</code>	Set when the user closes the window during a run.
<code>_root_logger</code>	<code>logging.Logger</code>	Root logger (to attach/remove the UI handler).
<code>_current_state</code>	<code>str</code> <code>None</code>	Current visual state key.
<code>_pending_pixmap</code>	<code>QPixmap</code>	Next picture waiting for the fade-out to finish.
<code>_fade_phase</code>	<code>str</code> <code>"idle"</code>	<code>idle</code> , <code>out</code> or <code>in</code> — which half of the fade is running.
<code>_display_pixmap</code>	<code>QPixmap</code>	The un-scaled picture currently shown (created on first use).
<code>_assets_directory</code>	<code>Path</code>	<code><project>/assets</code> (parent of <code>ui/</code>).
<code>_audio</code>	<code>AudioController</code>	Sound player for <code>assets/audio</code> .
<code>_log_emitter</code> , <code>_log_handler</code>	helper objects	Bridge from logging to the log box.
<code>_image_opacity</code> , <code>_image_fade</code>	effect, animation	Opacity effect and its 210 ms animation.

Lines 93–136 Constructor and font helper

```

93 | class RenamerWindow(QMainWindow):
94 |     """Compact image-renaming window with a live state preview."""
95 |
96 |     def __init__(self) -> None:
97 |         """Set up window state, assets, widgets, logging, and startup behavior."""
98 |         super().__init__()
99 |         self._configure_fonts()
100 |         self.setWindowTitle("Renamer - Mohamed Yassin Khmiri")
101 |         self.setMinimumSize(760, 650)
102 |         self.resize(1120, 720)
103 |
104 |         self._folder: Path | None = None
105 |         self._worker: ImageWorker | None = None
106 |         self._failed_files: tuple[Path, ...] = ()
107 |         self._close_when_worker_stops = False
108 |         self._root_logger = logging.getLogger()
109 |         self._current_state: str | None = None
110 |         self._pending_pixmap = QPixmap()
111 |         self._fade_phase = "idle"
112 |
113 |         self._assets_directory = Path(__file__).resolve().parents[1] / "assets"
114 |         self._audio = AudioController(self._assets_directory / "audio")
115 |         icon_path = self._assets_directory / "icon.ico"
116 |         if icon_path.is_file():
117 |             self.setWindowIcon(QIcon(str(icon_path)))
118 |
119 |         self._build_interface()
120 |         self._log_emitter = _UiLogEmitter(self)
121 |         self._log_emitter.message_received.connect(self.log_view.appendPlainText)
122 |         self._log_handler = _UiLogHandler(self._log_emitter)
123 |         self._log_handler.setFormatter(
124 |             logging.Formatter("%(asctime)s %(levelname)s %(name)s: %(message)s", "%H:%M:%S")
125 |         )
126 |         self._root_logger.addHandler(self._log_handler)
127 |
128 |         self._set_visual_state("ready", play_audio=False)
129 |         QTimer.singleShot(0, self.play_startup_audio)

```

```

130 |
131 |     @staticmethod
132 |     def _configure_fonts() -> None:
133 |         """Point Qt at installed Windows fonts when its bundled font path is missing."""
134 |         system_fonts = Path(os.environ.get("WINDIR", "C:/Windows")) / "Fonts"
135 |         if system_fonts.is_dir():
136 |             os.environ.setdefault("QT_QPA_FONTDIR", str(system_fonts))

```

`super().__init__()` starts Qt's own setup. The window title is set, the minimum size is 760×650 pixels and the first size is 1120×720. Lines 104-111 create the state variables listed above. Line 113 finds the assets folder:

`Path(__file__).resolve().parents[1] / "assets"` means *this file → its folder `ui` → its parent (project root) → `assets`*. The audio controller is created (not started). If `icon.ico` exists it becomes the window icon.

Line 119 builds all widgets. Lines 120-126 create the log bridge: an emitter, a handler with the time format `HH:MM:SS LEVEL logger: message`, and the handler is attached to the `root logger`, so all messages from every module appear in the little log box. Line 128 shows the *ready* state **without sound**. Line 129 uses `QTimer.singleShot(0, ...)` to start the music **as soon as the event loop starts** (delay 0 ms but after the window is shown). Because `main.py` also calls `play_startup_audio`, the guard inside `AudioController` makes the second call harmless.

`_configure_fonts` repeats the font-folder setting from `main.py` (as a `staticmethod`) — useful if the window is created from somewhere else.

Lines 138–192 `_build_interface`: theme and skeleton

```

138 |     def _build_interface(self) -> None:
139 |         """Apply the visual theme and arrange the single-window dashboard."""
140 |         self.setStyleSheet(
141 |             "QMainWindow, QWidget#appRoot { background: #10191d; color: #e8efeb; }"
142 |             "QLabel { color: #dce7e2; }"
143 |             "QLabel#appTitle { color: #d9e9df; font-size: 11px; font-weight: 700; }"
144 |             "QLabel#eyebrow, QLabel#sectionLabel { color: #9ab4aa; font-size: 9px; font-weight: 800; }"
145 |             "QLabel#imageCaption { color: #a3b6ae; font-size: 10px; }"
146 |             "QLabel#stateBanner { color: #b4e5f7; font-size: 12px; font-weight: 800; }"
147 |             "QToolTip { background: #e5f4eb; color: #15221c; border: 1px solid #7eb992; "
148 |             "border-radius: 4px; padding: 5px 7px; }"
149 |             "QGroupBox { background: #172326; border: 1px solid #334541; "
150 |             "border-radius: 7px; margin-top: 10px; padding: 10px 10px 8px; "
151 |             "font-size: 10px; font-weight: 800; color: #dce9e1; }"
152 |             "QGroupBox::title { subcontrol-origin: margin; left: 10px; padding: 0 4px; color: #a6d9bd; }"
153 |             "QLineEdit { background: #0d1719; border: 1px solid #405650; border-radius: 4px; "
154 |             "padding: 6px 8px; color: #edf3ee; selection-background-color: #367960; }"
155 |             "QLineEdit:focus { border-color: #80c7a0; }"
156 |             "QPushButton { background: #263936; color: #eaf3ed; border: 1px solid #496158; "
157 |             "border-radius: 5px; padding: 6px 9px; font-weight: 700; }"
158 |             "QPushButton:hover { background: #3a5c4f; border-color: #9ad3ad; color: #ffffff; }"
159 |             "QPushButton:pressed { background: #172721; border-color: #68aa82; padding-top: 7px; }"
160 |             "QPushButton:disabled { color: #798985; background: #202b29; border-color: #36433f; }"
161 |             "QPushButton#primaryButton { background: #397f5c; border-color: #7cc596; color: #f7fff8; }"
162 |             "QPushButton#primaryButton:hover { background: #48956d; border-color: #b3ebc3; }"
163 |             "QPushButton#choiceButton { min-height: 30px; padding: 5px 8px; text-align: center; }"
164 |             "QPushButton#choiceButton:checked { background: #285541; border-color: #8bd0a2; color: #f5ff"
165 |             "    f6; }"
166 |             "QPushButton#toggleButton { min-height: 28px; padding: 4px 8px; text-align: center; }"
167 |             "QPushButton#toggleButton:checked { background: #285541; border-color: #8bd0a2; color: #f5ff"
168 |             "    f6; }"
169 |             "QPushButton#muteButton { min-height: 18px; max-height: 18px; padding: 0 4px; }"
170 |             "QProgressBar { background: #0d1719; color: #e3eee7; border: 1px solid #405650; "
171 |             "border-radius: 3px; min-height: 9px; max-height: 9px; text-align: center; }"
172 |             "QProgressBar::chunk { background: #67b888; border-radius: 2px; }"
173 |             "QPlainTextEdit { background: #0d1719; color: #b9cbc3; border: 1px solid #334541; "
174 |             "border-radius: 4px; padding: 4px; selection-background-color: #367960; }"
175 |             "QSlider::groove:horizontal { height: 4px; background: #405650; border-radius: 2px; }"
176 |             "QSlider::sub-page:horizontal { background: #76bd91; border-radius: 2px; }"
177 |             "QSlider::handle:horizontal { width: 12px; margin: -4px 0; border-radius: 6px; "
178 |             "background: #f1fff5; border: 2px solid #53976d; }"

```

```

179 |         central = QWidget(self)
180 |         central.setObjectName("appRoot")
181 |         root_layout = QVBoxLayout(central)
182 |         root_layout.setContentsMargins(16, 5, 16, 10)
183 |         root_layout.setSpacing(6)
184 |         root_layout.addLayout(self._build_top_bar())
185 |
186 |         dashboard = QHBoxLayout()
187 |         dashboard.setSpacing(12)
188 |         dashboard.addWidget(self._build_image_panel(), 6)
189 |         dashboard.addWidget(self._build_control_panel(), 5)
190 |         root_layout.addLayout(dashboard, 1)
191 |         root_layout.addWidget(self._build_status_panel())
192 |         self.setCentralWidget(central)

```

`setStyleSheet` applies a **Qt style sheet** (like CSS). The selectors target widget types and object names (`QPushButton#primaryButton`). The look is a dark teal/green theme; see the colour table below. Lines 179-192 build the skeleton: a central widget named `appRoot` with a vertical layout (margins 16/5/16/10, spacing 6). Inside: the top bar, then a horizontal **dashboard** (spacing 12) containing the image panel (stretch 6) and the control panel (stretch 5), and finally the status panel. The dashboard gets stretch 1 so it takes all spare height.

Role	Colour	Where used
Window background	#10191d	QMainWindow, #appRoot
Group box background	#172326, border #334541	QGroupBox
Input / log background	#0d1719, border #405650	QLineEdit, QProgressBar, QPlainTextEdit
Button	#263936, border #496158	QPushButton
Button hover / pressed	#3a5c4f / #172721	QPushButton states
Primary button	#397f5c (hover #48956d)	#primaryButton (Rename images)
Checked choice/toggle	#285541, border #8bd0a2	#choiceButton, #toggleButton
Progress chunk	#67b888	QProgressBar::chunk
Slider groove / fill / handle	#405650 / #76bd91 / #f1fff5	QSlider
Tooltip	bg #e5f4eb, text #15221c	QToolTip

Lines 194–225 Top bar: volume slider and mute button

```

194 |     def _build_top_bar(self) -> QHBoxLayout:
195 |         """Create the volume slider and mute control row."""
196 |         header = QHBoxLayout()
197 |         header.setContentsMargins(0, 0, 0, 2)
198 |         header.addStretch(1)
199 |
200 |         audio = QHBoxLayout()
201 |         audio.setSpacing(7)
202 |         audio_label = QLabel("AUDIO")
203 |         audio_label.setObjectName("eyebrow")
204 |         audio.addWidget(audio_label)
205 |         self.volume_slider = QSlider(Qt.Orientation.Horizontal)
206 |         self.volume_slider.setRange(0, 100)
207 |         self.volume_slider.setValue(70)
208 |         self.volume_slider.setFixedWidth(112)
209 |         self.volume_slider.setAccessibleName("Audio volume")
210 |         self.volume_slider.setToolTip("Set the background music volume.")
211 |         self.volume_slider.valueChanged.connect(self._change_volume)
212 |         audio.addWidget(self.volume_slider)
213 |         self.volume_label = QLabel("70%")
214 |         self.volume_label.setMinimumWidth(36)
215 |         self.volume_label.setToolTip("Current background music volume.")
216 |         audio.addWidget(self.volume_label)
217 |         self.mute_button = QPushButton("Mute")
218 |         self.mute_button.setObjectName("muteButton")

```

```

219 |         self.mute_button.setCheckable(True)
220 |         self.mute_button.setFixedSize(56, 22)
221 |         self.mute_button.setToolTip("Mute or restore the background music.")
222 |         self.mute_button.toggled.connect(self._audio.set_muted)
223 |         audio.addWidget(self.mute_button)
224 |         header.addLayout(audio)
225 |         return header

```

A row with a stretch on the left pushing everything to the right. The slider is horizontal, range 0–100, start 70, fixed width 112 px, with an *accessible name* for screen readers. Moving it calls `_change_volume`. The label shows “70%”. The mute button is **checkable** (it stays pressed). Its `toggled(bool)` signal is connected **directly** to `self._audio.set_muted`, so checked = muted.

Lines 227–264 Image panel and the fade effect

```

227 |     def _build_image_panel(self) -> QGroupBox:
228 |         """Build the state-art preview and its fade transition."""
229 |         panel = QGroupBox("LIVE STATE")
230 |         panel.setToolTip("Shows the picture for the current app state.")
231 |         panel.setMinimumWidth(0)
232 |         layout = QVBoxLayout(panel)
233 |         layout.setContentsMargins(9, 13, 9, 8)
234 |         layout.setSpacing(5)
235 |
236 |         self.image_frame = QFrame()
237 |         self.image_frame.setObjectName("imageFrame")
238 |         self.image_frame.setStyleSheet(
239 |             "QFrame#imageFrame { background: #0a1117; border: 1px solid #334541; border-radius: 5px; }"
240 |         )
241 |         image_layout = QVBoxLayout(self.image_frame)
242 |         image_layout.setContentsMargins(0, 0, 0, 0)
243 |         self.image_label = _StateImageLabel("Image preview")
244 |         self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
245 |         self.image_label.setToolTip("The active state picture appears here.")
246 |         self.image_label.setMinimumSize(0, 0)
247 |         self.image_label.setSizePolicy(QSizePolicy.Policy.Ignored, QSizePolicy.Policy.Ignored)
248 |         self.image_label.setStyleSheet("background: transparent; color: #738895;")
249 |         self.image_label.resized.connect(self._scale_display_pixmap)
250 |         self._image_opacity = QGraphicsOpacityEffect(self.image_label)
251 |         self._image_opacity.setOpacity(1.0)
252 |         self.image_label.setGraphicsEffect(self._image_opacity)
253 |         self._image_fade = QPropertyAnimation(self._image_opacity, b"opacity", self)
254 |         self._image_fade.setDuration(210)
255 |         self._image_fade.setEasingCurve(QEasingCurve.Type.InOutCubic)
256 |         self._image_fade.finished.connect(self._finish_image_fade)
257 |         image_layout.addWidget(self.image_label, 1)
258 |         layout.addWidget(self.image_frame, 1)
259 |
260 |         caption = QLabel("CURRENT STATE")
261 |         caption.setObjectName("imageCaption")
262 |         caption.setToolTip("This picture changes while the job runs.")
263 |         layout.addWidget(caption)
264 |         return panel

```

A group box titled LIVE STATE holds a framed area with the picture label. `QSizePolicy.Ignored` for both directions tells the layout “do not let the picture's own size force the window bigger”; the picture is scaled to fit instead. `resized` is connected to `_scale_display_pixmap`. A `QGraphicsOpacityEffect` (opacity 1.0) is attached to the label and a `QPropertyAnimation` animates its `opacity` property for **210 ms** with an *InOutCubic* easing curve. When the animation finishes it calls `_finish_image_fade` (two-step fade: out, then in).

Lines 266–293 Control panel: folder box

```

266 |     def _build_control_panel(self) -> QWidget:
267 |         """Create folder, rename, export, safety, and action controls."""
268 |         panel = QWidget()
269 |         layout = QVBoxLayout(panel)
270 |         layout.setContentsMargins(0, 0, 0, 0)
271 |         layout.setSpacing(5)
272 |
273 |         directory_box = QGroupBox("FOLDER")
274 |         directory_box.setToolTip("Pick the folder with the images you want to rename.")
275 |         directory_layout = QHBoxLayout(directory_box)
276 |         directory_layout.setContentsMargins(8, 13, 8, 7)
277 |         directory_layout.setSpacing(6)
278 |         path_row = QHBoxLayout()
279 |         path_row.setSpacing(6)
280 |         self.folder_edit = QLineEdit()
281 |         self.folder_edit.setPlaceholderText("Choose or enter a folder")
282 |         self.folder_edit.setToolTip("Enter a folder path or choose one with Browse.")
283 |         self.folder_edit.setClearButtonEnabled(True)
284 |         self.folder_edit.editingFinished.connect(self._sync_folder_from_input)
285 |         self.folder_edit.textChanged.connect(self._folder_text_changed)
286 |         path_row.addWidget(self.folder_edit, 1)
287 |         self.folder_button = QPushButton("Choose folder")
288 |         self.folder_button.setFixedWidth(98)
289 |         self.folder_button.setToolTip("Choose the folder that has your images.")
290 |         self.folder_button.clicked.connect(self._choose_folder)
291 |         path_row.addWidget(self.folder_button)
292 |         directory_layout.addLayout(path_row)
293 |         layout.addWidget(directory_box)

```

Line edit with a placeholder, a clear (×) button, and two connections: `editingFinished` (user pressed Enter or left the field) → `_sync_folder_from_input`; `textChanged` (every keystroke) → `_folder_text_changed`. The *Choose folder* button (fixed width 98) opens the picker.

Lines 295–333 Control panel: rename style box

```

295 |         strategy_box = QGroupBox("RENAME STYLE")
296 |         strategy_box.setToolTip("Choose how your images will be named.")
297 |         strategy_layout = QVBoxLayout(strategy_box)
298 |         strategy_layout.setContentsMargins(8, 13, 8, 7)
299 |         strategy_layout.setSpacing(4)
300 |         self.mode_group = ButtonGroup(self)
301 |         self.numbered_mode_button = QPushButton("Numbered")
302 |         self.prefix_mode_button = QPushButton("Prefix")
303 |         self.date_mode_button = QPushButton("Capture date")
304 |         mode_row = QHBoxLayout()
305 |         mode_row.setSpacing(5)
306 |         for button in (
307 |             self.numbered_mode_button,
308 |             self.prefix_mode_button,
309 |             self.date_mode_button,
310 |         ):
311 |             button.setObjectName("choiceButton")
312 |             button.setCheckable(True)
313 |             self.mode_group.addButton(button)
314 |             button.setToolTip(
315 |                 {
316 |                     self.numbered_mode_button: "Name files Image_001, Image_002, and so on.",
317 |                     self.prefix_mode_button: "Add your own word before each number.",
318 |                     self.date_mode_button: "Use each picture's capture date when available.",
319 |                 }[button]
320 |             )
321 |             mode_row.addWidget(button)
322 |         self.numbered_mode_button.setChecked(True)
323 |         strategy_layout.addLayout(mode_row)
324 |         self.prefix_edit = QLineEdit()
325 |         self.prefix_edit.setPlaceholderText("For example, pic1")
326 |         self.prefix_edit.setToolTip("Type the word to put before each image number.")
327 |         self.prefix_edit.setEnabled(False)
328 |         self.prefix_edit.setVisible(False)
329 |         self.prefix_mode_button.toggled.connect(self.prefix_edit.setEnabled)
330 |         self.prefix_mode_button.toggled.connect(self.prefix_edit.setVisible)
331 |         self.prefix_edit.setFixedHeight(31)

```

```

332 |         strategy_layout.addWidget(self.prefix_edit)
333 |         layout.addWidget(strategy_box)

```

Three **checkable** buttons are put in a **QButtonGroup** — a group makes them **mutually exclusive** (like radio buttons). A small dictionary literal inside the loop creates each button's tooltip. *Numbered* starts checked. The prefix text box starts **disabled and hidden**; its **setEnabled** and **setVisible** slots are connected to the prefix button's **toggled** signal, so it appears only when *Prefix* is chosen. Height is fixed at 31 px.

Lines 335–373 Control panel: output box and options box

```

335 |         output_box = QGroupBox("SAVE AS")
336 |         output_box.setToolTip("Rename the originals or save renamed copies in a ZIP file.")
337 |         output_layout = QHBoxLayout(output_box)
338 |         output_layout.setContentsMargins(8, 13, 8, 7)
339 |         output_layout.setSpacing(6)
340 |         self.output_group = QButtonGroup(self)
341 |         self.in_place_button = QPushButton("In this folder")
342 |         self.zip_export_button = QPushButton("ZIP file")
343 |         for button in (self.in_place_button, self.zip_export_button):
344 |             button.setObjectName("choiceButton")
345 |             button.setCheckable(True)
346 |             self.output_group.addButton(button)
347 |             output_layout.addWidget(button)
348 |         self.in_place_button.setToolTip("Rename the images in their current folder.")
349 |         self.zip_export_button.setToolTip("Keep originals untouched and save renamed copies to a ZIP fil
↳ e.")
350 |         self.in_place_button.setChecked(True)
351 |         self.zip_export_button.toggled.connect(self._update_output_options)
352 |         layout.addWidget(output_box)
353 |
354 |         safety_box = QGroupBox("OPTIONS")
355 |         safety_box.setToolTip("Choose whether to remove image details and keep a backup.")
356 |         safety_layout = QHBoxLayout(safety_box)
357 |         safety_layout.setContentsMargins(8, 13, 8, 7)
358 |         safety_layout.setSpacing(6)
359 |         self.metadata_toggle = QPushButton("Clean image details")
360 |         self.metadata_toggle.setObjectName("toggleButton")
361 |         self.metadata_toggle.setCheckable(True)
362 |         self.metadata_toggle.setChecked(True)
363 |         self.metadata_toggle.setToolTip(
364 |             "Remove extra image details from JPEG and PNG files. Other formats stay unchanged."
365 |         )
366 |         self.backup_toggle = QPushButton("Backup first")
367 |         self.backup_toggle.setObjectName("toggleButton")
368 |         self.backup_toggle.setCheckable(True)
369 |         self.backup_toggle.setChecked(True)
370 |         self.backup_toggle.setToolTip("Save a backup before changing the original files.")
371 |         safety_layout.addWidget(self.metadata_toggle, 1)
372 |         safety_layout.addWidget(self.backup_toggle, 1)
373 |         layout.addWidget(safety_box)

```

In this folder and *ZIP file* form another exclusive group; *In this folder* starts checked.

zip_export_button.toggled calls *_update_output_options*. The **OPTIONS** box has two toggle buttons, both checked at start: *Clean image details* (*metadata_toggle*) and *Backup first* (*backup_toggle*).

Lines 375–394 Control panel: action buttons

```

375 |         actions = QHBoxLayout()
376 |         self.process_button = QPushButton("Rename images")
377 |         self.process_button.setObjectName("primaryButton")
378 |         self.process_button.setEnabled(False)
379 |         self.process_button.setToolTip("Rename the images in the chosen folder.")
380 |         self.process_button.clicked.connect(self._start_processing)
381 |         self.retry_button = QPushButton("Try failed again")
382 |         self.retry_button.setEnabled(False)
383 |         self.retry_button.setToolTip("Try renaming the images that failed last time.")
384 |         self.retry_button.clicked.connect(self._retry_failed)
385 |         self.cancel_button = QPushButton("Cancel")
386 |         self.cancel_button.setEnabled(False)
387 |         self.cancel_button.setToolTip("Stop the current job and restore the originals.")
388 |         self.cancel_button.clicked.connect(self._cancel_processing)

```

```

389 |         actions.setSpacing(6)
390 |         actions.addWidget(self.process_button, 2)
391 |         actions.addWidget(self.retry_button)
392 |         actions.addWidget(self.cancel_button)
393 |         layout.addLayout(actions)
394 |         return panel

```

Rename images is the **primary** button (stretch 2) and starts disabled; *Try failed again* and *Cancel* start disabled too. Clicks are connected to `_start_processing`, `_retry_failed` and `_cancel_processing`.

Lines 396–430 Status panel

```

396 |     def _build_status_panel(self) -> QGroupBox:
397 |         """Create the state banner, progress bar, and compact event log."""
398 |         panel = QGroupBox("STATUS")
399 |         panel.setToolTip("See what is happening and follow the job's progress.")
400 |         layout = QVBoxLayout(panel)
401 |         layout.setContentsMargins(10, 13, 10, 7)
402 |         layout.setSpacing(4)
403 |         self.state_label = QLabel()
404 |         self.state_label.setObjectName("stateBanner")
405 |         self.state_label.setWordWrap(False)
406 |         self.state_label.setMinimumHeight(18)
407 |         self.state_label.setToolTip("Current app state.")
408 |         layout.addWidget(self.state_label)
409 |         self.status_label = QLabel("Ready")
410 |         self.status_label.setWordWrap(False)
411 |         self.status_label.setStyleSheet("color: #aab9b1; font-size: 10px;")
412 |         self.status_label.setToolTip("More detail about the current job.")
413 |         layout.addWidget(self.status_label)
414 |         self.progress_bar = QProgressBar()
415 |         self.progress_bar.setRange(0, 100)
416 |         self.progress_bar.setValue(0)
417 |         self.progress_bar.setToolTip("How much of the current job is complete.")
418 |         layout.addWidget(self.progress_bar)
419 |         self.log_view = QPlainTextEdit()
420 |         self.log_view.setReadOnly(True)
421 |         self.log_view.setPlaceholderText("Processing details will appear here.")
422 |         self.log_view.setToolTip("A short log of what happened during the last job.")
423 |         self.log_view.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
424 |         self.log_view.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAlwaysOff)
425 |         self.log_view.setMinimumHeight(34)
426 |         self.log_view.setMaximumHeight(38)
427 |         self.log_view.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)
428 |         self.log_view.setFont(QFont("Consolas", 9))
429 |         layout.addWidget(self.log_view)
430 |         return panel

```

Contains: `state_label` (banner, no word-wrap), `status_label` (starts “Ready”), `progress_bar` (0–100, height 9 px by style) and `log_view`, a **read-only** plain-text box 34–38 px high with both scroll bars off and a Consolas 9 pt font, showing the last lines of the event log.

Lines 432–474 Small handlers: volume, folder text, folder validation, output options

```

432 |     def _change_volume(self, value: int) -> None:
433 |         """Update the volume label and apply the selected audio level."""
434 |         self.volume_label.setText(f"{value}%")
435 |         self._audio.set_volume(value / 100)
436 |
437 |     def _folder_text_changed(self, text: str) -> None:
438 |         """Enable the main action only while the typed path is a real folder."""
439 |         path = Path(text).expanduser() if text.strip() else None
440 |         self._folder = path if path is not None and path.is_dir() else None
441 |         self.process_button.setEnabled(self._folder is not None and not self._is_worker_running())
442 |
443 |     def _sync_folder_from_input(self) -> bool:
444 |         """Validate and normalize the folder field before starting a job."""
445 |         text = self.folder_edit.text().strip()
446 |         if not text:
447 |             self._folder = None
448 |             self.process_button.setEnabled(False)
449 |             return False

```

```

450 |         folder = Path(text).expanduser()
451 |         if not folder.is_dir():
452 |             self._folder = None
453 |             self.process_button.setEnabled(False)
454 |             self.status_label.setText("That folder does not exist. Choose another one.")
455 |             return False
456 |         self._folder = folder
457 |         self.folder_edit.setText(str(folder))
458 |         self.folder_edit.setCursorPosition(0)
459 |         self.process_button.setEnabled(not self._is_worker_running())
460 |         return True
461 |
462 |     def _update_output_options(self) -> None:
463 |         """Apply the safety and action-button settings for the selected output."""
464 |         zip_export = self.zip_export_button.isChecked()
465 |         self.backup_toggle.setEnabled(not zip_export)
466 |         self.metadata_toggle.setEnabled(not zip_export)
467 |         if zip_export:
468 |             self.metadata_toggle.setChecked(True)
469 |             self.process_button.setText("Create ZIP" if zip_export else "Rename images")
470 |             self.process_button.setToolTip(
471 |                 "Save renamed copies in a ZIP and leave originals alone."
472 |                 if zip_export
473 |                 else "Rename the images in the chosen folder."
474 |             )

```

`_change_volume(value)`: sets the % label and sends `value/100` to the audio controller.

`_folder_text_changed(text)`: runs on every keystroke. It expands `~` (`expanduser`) and sets `_folder` to the path **only if it is an existing directory**; the main button is enabled only if there is a folder *and* no worker is running.

`_sync_folder_from_input()` → `bool`: the strict validator used before a job. Empty → disable and return False. Not a directory → show “That folder does not exist. Choose another one.” → False. Valid → store it, show the cleaned path in the box, move the cursor to the start, enable the button (if idle), return True.

`_update_output_options()`: in ZIP mode, **Backup and Clean are disabled** (clean is also *forced checked*), the main button text becomes “Create ZIP” and the tooltip changes; in-place mode restores them.

Lines 476–523 Showing a state: banner, colour, picture, sound

```

476 |     def play_startup_audio(self) -> None:
477 |         """Start the optional startup track; safe to call more than once."""
478 |         self._audio.play_startup()
479 |
480 |     def _set_visual_state(self, state: str, play_audio: bool = True) -> None:
481 |         """Update the banner, preview image, color, and optional state sound."""
482 |         presentation = STATE_PRESENTATION.get(state)
483 |         if presentation is None:
484 |             logger.warning("Ignoring unknown worker state %r", state)
485 |             return
486 |
487 |         self._current_state = state
488 |         banner, image_names = presentation
489 |         self.state_label.setText(banner)
490 |         self.state_label.setProperty("renamerState", state)
491 |         state_colors = {
492 |             "ready": "#83c9e2",
493 |             "working": "#68c5e7",
494 |             "caution": "#d5ad65",
495 |             "error": "#d9847a",
496 |             "success": "#76c99b",
497 |         }
498 |         self.state_label.setStyleSheet(
499 |             f"color: {state_colors[state]}; font-size: 13px; font-weight: 700;"
500 |         )
501 |
502 |         image_path = next(
503 |             (
504 |                 self._assets_directory / name
505 |                 for name in image_names
506 |                 if (self._assets_directory / name).is_file()

```

```

507 |         ),
508 |         None,
509 |     )
510 |     pixmap = QPixmap(str(image_path)) if image_path else QPixmap()
511 |     if pixmap.isNull():
512 |         self._pending_pixmap = QPixmap()
513 |         self._image_fade.stop()
514 |         self.image_label.setPixmap(QPixmap())
515 |         self.image_label.setText(banner)
516 |         self._image_opacity.setOpacity(1.0)
517 |         logger.warning("No usable state image is available for %s", state)
518 |     else:
519 |         self.image_label.setText("")
520 |         self._transition_image(pixmap)
521 |
522 |     if play_audio:
523 |         self._audio.play_state_cue(state)

```

`play_startup_audio` just forwards to the controller. `_set_visual_state(state, play_audio=True)`: unknown keys are logged and ignored. Otherwise: store the state, set banner text, store a Qt *property* `renamerState` (handy for styling), set the banner colour from a dict (ready `#83c9e2`, working `#68c5e7`, caution `#d5ad65`, error `#d9847a`, success `#76c99b`; font 13 px bold).

Picture choice: a *generator expression* with `next(..., None)` takes the **first picture name whose file exists**. If it loads → clear any text and start `_transition_image`. If none is usable → stop the fade, clear the picture, show the banner text in the label instead, reset opacity, log a warning. Finally the state's sound cue is played unless `play_audio=False`.

Lines 525–577 The cross-fade between pictures

```

525 |     def _transition_image(self, pixmap: QPixmap) -> None:
526 |         """Fade from the displayed state image to the pending image."""
527 |         if self._image_fade.state() == QPropertyAnimation.State.Running:
528 |             self._image_fade.stop()
529 |             self._pending_pixmap = pixmap
530 |             current = self.image_label.pixmap()
531 |             if current is None or current.isNull():
532 |                 self._set_display_pixmap(pixmap)
533 |                 self._animate_image_opacity(0.0, 1.0, "in")
534 |             else:
535 |                 self._animate_image_opacity(self._image_opacity.opacity(), 0.0, "out")
536 |
537 |     def _animate_image_opacity(self, start: float, end: float, phase: str) -> None:
538 |         """Run one half of the image fade and remember its transition phase."""
539 |         self._fade_phase = phase
540 |         self._image_fade.setStartValue(start)
541 |         self._image_fade.setEndValue(end)
542 |         self._image_fade.start()
543 |
544 |     def _finish_image_fade(self) -> None:
545 |         """Swap images after fade-out or reset opacity after fade-in."""
546 |         if self._fade_phase == "out":
547 |             self._set_display_pixmap(self._pending_pixmap)
548 |             self._animate_image_opacity(0.0, 1.0, "in")
549 |         else:
550 |             self._fade_phase = "idle"
551 |             self._image_opacity.setOpacity(1.0)
552 |
553 |     def _set_display_pixmap(self, pixmap: QPixmap) -> None:
554 |         """Store the original image and scale it to the current preview size."""
555 |         self._display_pixmap = pixmap
556 |         self._scale_display_pixmap()
557 |
558 |     def _scale_display_pixmap(self) -> None:
559 |         """Scale the active image smoothly so it fills the preview area."""
560 |         pixmap = getattr(self, "_display_pixmap", QPixmap())
561 |         if pixmap.isNull():
562 |             return
563 |         available = self.image_label.contentsRect().size()
564 |         if available.width() <= 0 or available.height() <= 0:
565 |             return

```

```

566 |         self.image_label.setPixmap(
567 |             pixmap.scaled(
568 |                 available,
569 |                 Qt.AspectRatioMode.KeepAspectRatioByExpanding,
570 |                 Qt.TransformationMode.SmoothTransformation,
571 |             )
572 |         )
573 |
574 |     def resizeEvent(self, event: QResizeEvent) -> None:
575 |         """Rescale the preview image after the window itself is resized."""
576 |         super().resizeEvent(event)
577 |         self._scale_display_pixmap()

```

This is a small **state machine** driven by `_fade_phase`. `_transition_image(new)`: stop a running fade, remember the new picture in `_pending_pixmap`. If the label has no picture yet → show immediately and fade **in** (0→1). Otherwise fade **out** from the current opacity to 0 with phase **"out"**. When the animation ends, `_finish_image_fade` runs: after **"out"** it swaps in the pending picture and starts the **"in"** fade; after **"in"** it resets to **idle** and forces opacity 1.0.

`_set_display_pixmap` stores the full-size picture and calls `_scale_display_pixmap`, which reads the label's inner rectangle and scales the picture with ****KeepAspectRatioByExpanding**** (fills the whole area, cropping the overflow — no empty bars) and **SmoothTransformation** (high-quality). It is also called from `resizeEvent` so the picture follows window size changes. If the area has zero size nothing happens.

Lines 579–596 Choose folder, start, retry

```

579 |     def _choose_folder(self) -> None:
580 |         """Open the folder picker and validate the selected directory."""
581 |         selected = QFileDialog.getExistingDirectory(self, "Choose a folder")
582 |         if selected:
583 |             self.folder_edit.setText(selected)
584 |             self._sync_folder_from_input()
585 |             logger.info("Selected image folder %s", self._folder)
586 |
587 |     def _start_processing(self) -> None:
588 |         """Start a batch using every supported image in the chosen folder."""
589 |         self._launch_processing()
590 |
591 |     def _retry_failed(self) -> None:
592 |         """Retry only the files recorded as failed by the previous batch."""
593 |         if self._failed_files:
594 |             logger.info("Retrying %d failed image(s)", len(self._failed_files))
595 |             self._launch_processing(self._failed_files)
596 |

```

`QFileDialog.getExistingDirectory` returns a path or an empty string if cancelled. A chosen path is put in the box and validated; the choice is logged. `_start_processing` starts a full run (`_launch_processing()` without files). `_retry_failed` starts a run limited to `_failed_files`, but only if that tuple is not empty.

Lines 597–656 _launch_processing: from click to running worker

```

597 |     def _launch_processing(self, files: tuple[Path, ...] | None = None) -> None:
598 |         """Validate options, connect a worker, and start a full or retry batch."""
599 |         if self._is_worker_running() or not self._sync_folder_from_input():
600 |             return
601 |         if files is None:
602 |             self._failed_files = ()
603 |         if self.prefix_mode_button.isChecked() and not self.prefix_edit.text().strip():
604 |             self.status_label.setText("Add a prefix first.")
605 |             self.prefix_edit.setFocus()
606 |             return
607 |
608 |         mode = "default"
609 |         if self.prefix_mode_button.isChecked():
610 |             mode = "prefix"
611 |         elif self.date_mode_button.isChecked():
612 |             mode = "date"
613 |

```

```

614 |         output_archive_path: Path | None = None
615 |         if self.zip_export_button.isChecked():
616 |             archive_name, _ = QFileDialog.getSaveFileName(
617 |                 self,
618 |                 "Save renamed images as a ZIP",
619 |                 str(self._folder / "renamed_images.zip"),
620 |                 "ZIP archives (*.zip)",
621 |             )
622 |             if not archive_name:
623 |                 return
624 |             output_archive_path = Path(archive_name)
625 |             if output_archive_path.suffix.lower() != ".zip":
626 |                 output_archive_path = output_archive_path.with_suffix(".zip")
627 |
628 |         self.progress_bar.setValue(0)
629 |         self.log_view.clear()
630 |         self._set_controls_enabled(False)
631 |         self.retry_button.setEnabled(False)
632 |         self.cancel_button.setEnabled(True)
633 |         self._set_visual_state("working" if output_archive_path else "caution")
634 |         self.status_label.setText(
635 |             "Preparing ZIP export copies..."
636 |             if output_archive_path
637 |             else "Making sure your originals are backed up..."
638 |         )
639 |
640 |         self._worker = ImageWorker(
641 |             self._folder,
642 |             mode=mode,
643 |             prefix=self.prefix_edit.text(),
644 |             keep_backup=self.backup_toggle.isChecked(),
645 |             strip_metadata_enabled=self.metadata_toggle.isChecked(),
646 |             output_archive_path=output_archive_path,
647 |             parent=self,
648 |             files=files,
649 |         )
650 |         self._worker.progress_changed.connect(self.progress_bar.setValue)
651 |         self._worker.status_changed.connect(self.status_label.setText)
652 |         self._worker.state_changed.connect(self._set_visual_state)
653 |         self._worker.file_failed.connect(self._show_file_failure)
654 |         self._worker.completed.connect(self._processing_finished)
655 |         self._worker.finished.connect(self._worker_thread_finished)
656 |         self._worker.start()

```

Guards: do nothing if a worker is already running or the folder is invalid. For a normal (not retry) run, the failure list is cleared. If *Prefix* is selected but the box is empty: show “Add a prefix first.”, focus the box and stop.

The mode string: `"default"`, or `"prefix"`, or `"date"`. In ZIP mode a **Save dialog** (default name `renamed_images.zip` inside the folder) asks for the target; cancelling ends the launch; a missing `.zip` extension is added. Then the UI goes into “busy” mode: progress 0, log cleared, controls disabled, Retry disabled, Cancel enabled. The first state shown is *working* for ZIP and *caution* for in-place, with a matching message.

The worker is created with all options (`parent=self` ties its lifetime to the window), the **six signal connections** are made (progress, status, state, file_failed, completed, finished) and `start()` launches the thread.

Lines 658–709 Failure display, cancel, lock controls, finish handlers

```

658 |     def _show_file_failure(self, path: str, error: str) -> None:
659 |         """Append one failed image and its error message to the event log."""
660 |         self.log_view.appendPlainText(f"{ERROR_ISOLATION_TAG}: {Path(path).name}: {error}")
661 |
662 |     def _cancel_processing(self) -> None:
663 |         """Request safe worker cancellation without force-stopping its thread."""
664 |         if self._is_worker_running():
665 |             self.cancel_button.setEnabled(False)
666 |             self.status_label.setText("Cancelling and restoring originals...")
667 |             self._worker.requestInterruption()
668 |
669 |     def _set_controls_enabled(self, enabled: bool) -> None:
670 |         """Lock or restore controls while a worker is processing images."""
671 |         for control in (
672 |             self.folder_edit,

```

```

673 |         self.folder_button,
674 |         self.numbered_mode_button,
675 |         self.prefix_mode_button,
676 |         self.date_mode_button,
677 |         self.prefix_edit,
678 |         self.in_place_button,
679 |         self.zip_export_button,
680 |         self.metadata_toggle,
681 |         self.backup_toggle,
682 |     ):
683 |         control.setEnabled(enabled)
684 |         self.retry_button.setEnabled(enabled and bool(self._failed_files))
685 |         self.process_button.setEnabled(enabled and self._folder is not None)
686 |
687 |     def _processing_finished(self, succeeded: bool, message: str) -> None:
688 |         """Update failure history, progress, state, and the final status text."""
689 |         if self._worker is not None:
690 |             self._failed_files = tuple(self._worker.failed_files)
691 |             self.retry_button.setEnabled(bool(self._failed_files))
692 |         if succeeded:
693 |             self.progress_bar.setValue(100)
694 |             self._set_visual_state("success")
695 |             self.status_label.setText(message)
696 |
697 |     def _worker_thread_finished(self) -> None:
698 |         """Release the worker reference, restore controls, and honor close requests."""
699 |         self._worker = None
700 |         self.cancel_button.setEnabled(False)
701 |         self._set_controls_enabled(True)
702 |         self._update_output_options()
703 |         self.process_button.setEnabled(self._folder is not None)
704 |         if self._close_when_worker_stops:
705 |             self.close()
706 |
707 |     def _is_worker_running(self) -> bool:
708 |         """Return whether the current worker exists and is still active."""
709 |         return self._worker is not None and self._worker.isRunning()

```

`_show_file_failure` adds a line `[ERROR ISOLATION] ...: <file name>: <error>` to the log box.

`_cancel_processing` (only if running): disable Cancel, show “Cancelling and restoring originals...” and call `requestInterruption()` — **no force-kill**.

`_set_controls_enabled(enabled)` flips ten controls in one loop, then sets Retry (needs failures) and the main button (needs a folder). `_processing_finished(ok, msg)` copies the worker's `failed_files` into the window, enables Retry if any, and on success sets 100% and shows `success` again; the message is shown last.

`_worker_thread_finished` runs when the thread has fully stopped: forget the worker, disable Cancel, unlock controls, re-apply output options, enable the main button if there is a folder, and — if the user tried to close during the run — close now.

`_is_worker_running()` is a helper: worker exists **and** `isRunning()`.

Lines 711–721 `closeEvent`: closing during a job is safe

```

711 |     def closeEvent(self, event: QCloseEvent) -> None:
712 |         """Cancel active work before closing and shut down the audio controller."""
713 |         if self._is_worker_running():
714 |             self._close_when_worker_stops = True
715 |             self._worker.requestInterruption()
716 |             self.status_label.setText("Cancelling and restoring originals before closing...")
717 |             event.ignore()
718 |             return
719 |         self._root_logger.removeHandler(self._log_handler)
720 |         self._audio.shutdown()
721 |         super().closeEvent(event)

```

If a job is running, the window **refuses to close** (`event.ignore()`), remembers the wish in

`_close_when_worker_stops`, asks the worker to cancel and shows “Cancelling and restoring originals before closing...”. When the thread ends, `_worker_thread_finished` calls `close()` again — and this time it goes

through: the UI log handler is removed from the root logger (so no signals are sent to a dead window), the audio mixer is shut down, and Qt closes the window.

Two small observations

(1) A successful run applies the `success` state twice — once from the worker's `state_changed('success')` and once in `_processing_finished` — so the sound cue may play twice. (2) `_display_pixmap` is first created inside `_set_display_pixmap` and read with `getattr(..., QPixmap())`, because it is not set in `__init__`.

3.9 The other files

requirements.txt

```
1 | Pillow>=10.0
2 | PySide6>=6.6
3 | pygame>=2.5
```

Three runtime libraries, each with a minimum version (`>=`): **Pillow 10** (image checking and EXIF), **PySide6 6.6** (Qt GUI) and **pygame 2.5** (sound). The file has no trailing newline. Install with `python -m pip install -r requirements.txt`.

installer/Renamer.iss (Inno Setup script)

```
1 | #define MyAppName "Renamer"
2 | #define MyAppVersion "1.0.0"
3 | #define MyAppPublisher "Mohamed Yassin Khmiri"
4 | #define MyAppExeName "Renamer.exe"
5 |
6 | [Setup]
7 | AppId={D7D04437-5C38-4B7D-A8C0-6CF65B514067}
8 | AppName={#MyAppName}
9 | AppVersion={#MyAppVersion}
10 | AppPublisher={#MyAppPublisher}
11 | DefaultDirName={localappdata}\Programs\{#MyAppName}
12 | DefaultGroupName={#MyAppName}
13 | DisableProgramGroupPage=yes
14 | PrivilegesRequired=lowest
15 | OutputDir=..\dist
16 | OutputBaseFilename=Renamer-Setup
17 | SetupIconFile=..\assets\icon.ico
18 | UninstallDisplayIcon={app}\{#MyAppExeName}
19 | LicenseFile=..\LICENSE
20 | WizardStyle=modern
21 | MinVersion=10.0.10240
22 | ArchitecturesAllowed=x64
23 | ArchitecturesInstallIn64BitMode=x64
24 | Compression=lzma2
25 | SolidCompression=yes
26 |
27 | [Languages]
28 | Name: "english"; MessagesFile: "compiler:Default.isl"
29 |
30 | [Files]
31 | Source: "..\dist\Renamer\*"; DestDir: "{app}"; \
32 |     Flags: ignoreversion recursesubdirs createallsubdirs
33 |
34 | [Icons]
35 | Name: "{autoprograms}\{#MyAppName}"; Filename: "{app}\{#MyAppExeName}"
36 | Name: "{autodesktop}\{#MyAppName}"; Filename: "{app}\{#MyAppExeName}"
37 |
38 | [Run]
```

Section / line	Meaning
<code>#define ...</code>	Four constants: app name <i>Renamer</i> , version 1.0.0, publisher, exe name <i>Renamer.exe</i> .
<code>AppId={{D7D04437-...}}</code>	A fixed GUID that identifies the app so updates and uninstall find it (<code>{{</code> is an escaped <code>}</code>).
<code>DefaultDirName={localappdata}\Programs\Renamer</code>	Installs per user, no admin rights.
<code>PrivilegesRequired=lowest</code>	Never asks for administrator.
<code>OutputDir=..\dist,</code> <code>OutputBaseFilename=Renamer-Setup</code>	Where the setup file is built and how it is named (<i>Renamer-Setup.exe</i>).
<code>SetupIconFile, UninstallDisplayIcon</code>	Uses <i>assets\icon.ico</i> .
<code>LicenseFile=..\LICENSE</code>	Shows the MIT licence in the wizard.
<code>WizardStyle=modern,</code> <code>MinVersion=10.0.10240</code>	Modern look; needs Windows 10 or newer.
<code>ArchitecturesAllowed=x64</code>	64-bit PCs only; installs in 64-bit mode.
<code>Compression=lzma2,</code> <code>SolidCompression=yes</code>	Small installer.
<code>[Files]</code>	Takes everything from <code>..\dist\Renamer\</code> (the PyInstaller output) recursively.
<code>[Icons]</code>	Start-menu and desktop shortcuts.
<code>[Run]</code>	Offers “Launch Renamer” at the end (skipped in silent installs).

assets/

File	Used by	Notes
<i>icon.ico</i>	window icon, installer icon	3 KB Windows icon
<i>here you re lad .jpg</i>	state <i>ready</i>	720×480
<i>OH WAIT working on it.jpg</i>	state <i>working</i>	735×525
<i>caution.jpg</i>	state <i>caution</i>	736×552
<i>error.jpg</i>	state <i>error</i>	735×546 (<i>errore.jpg</i> is listed first in code but not present)
<i>Happy for using my tool xD.jpg</i>	state <i>success</i>	434×433
<i>audio/main_theme.mp3</i>	<i>AudioController.play_startu</i> <i>p</i>	2.1 MB looping theme

Optional files *ready.wav*, *working.wav*, *caution.wav*, *error.wav*, *success.wav* can be added to *assets/audio/* to replace the generated beeps.

Documentation, legal and repo files

File	Content
<i>README.md</i>	Public description: features, formats, install, layout, audio/logs, license, screenshots, fan-theme notice.

File	Content
DEV.md	Developer notes: setup, structure, application flow, installer, the PyInstaller command and rules for comments.
LICENSE	MIT License, © 2026 Mohamed Yassin Khmiri.
COPYRIGHT.md	Copyright notice; third-party assets keep their own licences.
.gitignore	Ignores .venv/, __pycache__/, *.py[cod], .pytest_cache/, /build/, /dist/, /Renamer.spec.
docs/screenshots/*.png	Four screenshots used in the README (and in this guide).

PART 4

Mechanisms deep dive

This part explains the *ideas* behind the code. Each section answers: what problem is solved, how, and why it is safe.

4.1 Atomic file replacement

Problem: if the program crashes while writing a file, the file is left half-written and the picture is lost. **Solution:** never write into the real file. Write a new file next to it, then call `os.replace(new, old)`. The operating system performs this swap as one indivisible step, so another program (or a power cut) sees either the complete old file or the complete new file, never something in between. It is used in `strip_metadata`, `_restore_from_bytes`, the worker's per-file publish and the final ZIP.

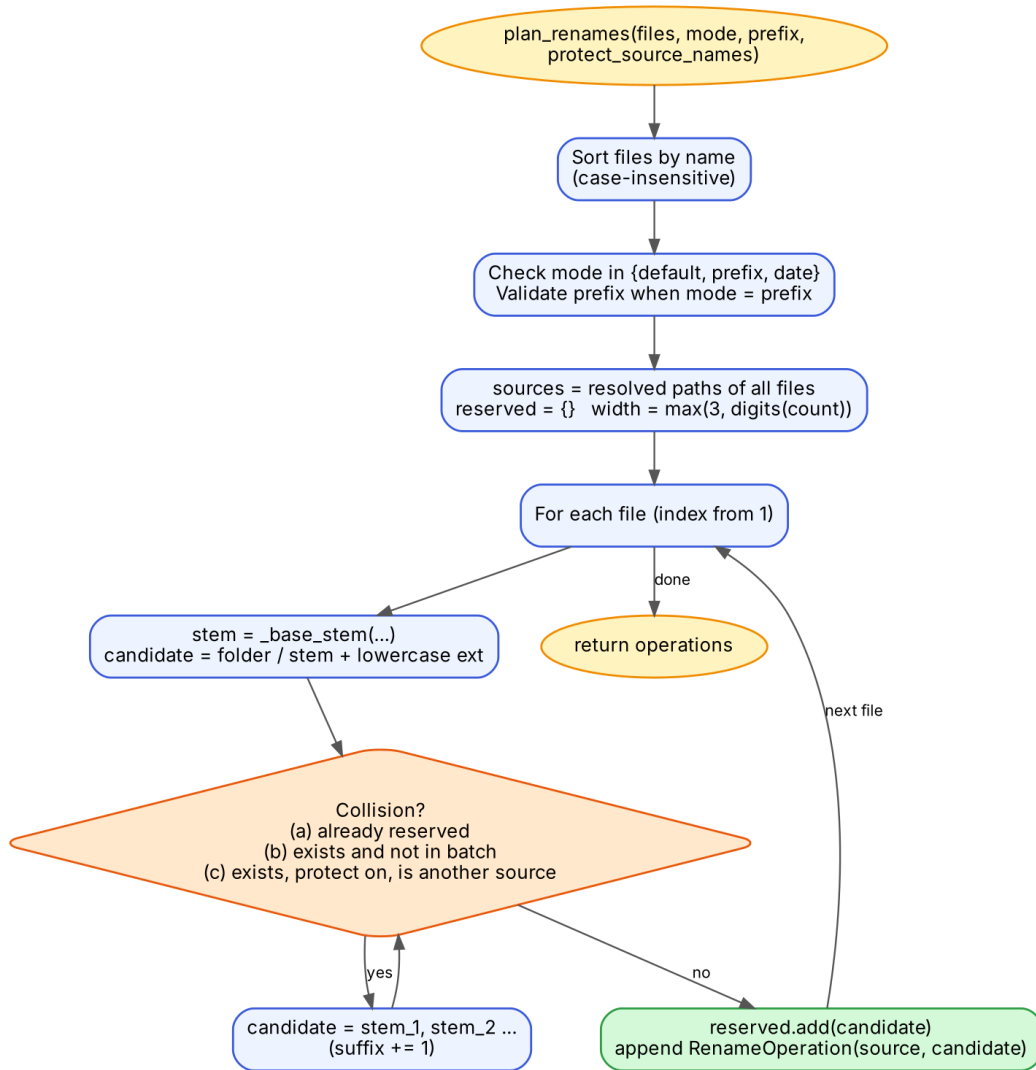
Why “in the same folder”? An atomic rename only works inside one drive. Temp files are therefore always created with `dir=<same folder>`.

4.2 Exclusive creation (`O_EXCL`)

`os.replace` overwrites without asking. `_publish_without_overwrite` first creates the destination name with the flags `O_CREAT | O_EXCL | O_WRONLY`. `O_EXCL` means “create, and fail if it exists”, enforced by the operating system. So even if another program creates a file with that name between planning and publishing, Renamer notices and treats the picture as failed, rather than destroying the other file.

4.3 Naming and collision avoidance

The planner works in memory first and only then does the worker touch the disk. It sorts files, numbers them and checks three collision rules (reserved, exists-and-not-ours, exists-and-another-source). Example with the folder `a.jpg`, `b.jpg`, `Image_001.jpg`, `Image_002.jpg` in default mode:

Figure 21. Flow of `plan_renames`.

Index	Source	Wanted name	Problem?	Final name
1	a.jpg	Image_001.jpg	Exists and is another source (protected)	Image_001_1.jpg
2	b.jpg	Image_002.jpg	Exists and is another source (protected)	Image_002_1.jpg
3	Image_001.jpg	Image_003.jpg	Free	Image_003.jpg
4	Image_002.jpg	Image_004.jpg	Free	Image_004.jpg

Sorting is case-insensitive: `a.jpg < b.jpg < Image_001.jpg < Image_002.jpg`. Notice that a user-visible side effect is that already well-named files can get re-numbered. This is expected behaviour of numbered style.

Retry numbering

Try failed again plans only the failed files. They are numbered from 1 again, but names like `Image_001.jpg` already exist (from the first run) and are not part of the retry batch, so the planner adds suffixes: `Image_001_1.jpg`, `Image_002_1.jpg`... This is safe (nothing is overwritten) but the numbers will not continue the first run's sequence.

4.4 Backup, verification and rollback

Three promises are made: (1) *the backup is complete* — checked by SHA-256 against the original; (2) *the backup is made before any change*; (3) *rollback restores bytes, permissions and times*. The next table shows what each backup type protects.

	Persistent ZIP (default)	Temporary shadow copy
Created in	<images folder>/ <code>.RENAMER_backups/</code>	System temp folder
Compression	None (stored)	None (plain copies)
Verified by	member list, <code>testzip</code> , SHA-256 per file	SHA-256 per file
After success	Kept for the user	Deleted
After rollback	Kept	Deleted (copied back, then removed)
Restores times	Yes (<code>os.utime</code> with saved ns times)	Yes (<code>copy2</code>)

4.5 Cancellation and closing

`requestInterruption()` sets a flag on the thread. `isInterruptionRequested()` is checked: before each file in the loop and once after the loop in the in-place path; in the ZIP path while staging, before each file, and before writing the archive. So a cancel can take as long as **one file** to be noticed. Then `BatchCancelled` bubbles to the outer `except`, which restores everything. If the user closes the window instead, the close is postponed (`event.ignore`) until the worker finishes its rollback.

4.6 Error handling strategy

Level	Where	What happens
File	worker loop	Caught per file; file skipped; others continue (in-place). In ZIP mode the first failure aborts.
Batch	worker outer <code>try</code>	Caught; rollback; state <i>error</i> .
Library	audio	Warnings only; never fatal.
Input	window	Invalid folder / empty prefix is shown in the status line; the job does not start.
Program	<code>sys.excepthook</code>	Logged as CRITICAL with traceback.

4.7 Logging

- **File log** (`main.py`): root logger, INFO, rotating 2 MB × 5, includes thread name.
- **UI log** (`RenamerWindow`): root logger, INFO, short time format, shown in the small box and cleared at each new job.
- Every module uses `logging.getLogger(__name__)` so lines show which file wrote them (e.g. `workers.image_worker`).
- `logger.exception(...)` records the message **and the traceback**; `logger.info` the normal story.

4.8 Lazy and optional audio

pygame is imported inside `_ensure_mixer` through `importlib.import_module`. Benefits: start-up is faster, a missing pygame or sound card only disables sound, and the source can run on machines without audio. Cost: PyInstaller cannot see the import, so the build command needs `--collect-all pygame` (Part 5).

4.9 Complete life of one run (summary)

#	Step	Where
1	User picks folder, style, mode and presses the button	<code>RenamerWindow</code>
2	Folder, prefix and ZIP target validated; UI locked; worker created and started	<code>_launch_processing</code>
3	Images collected, filtered and sorted	<code>ImageWorker.run</code>
4	Verified backup created (caution)	<code>BackupManager.create_backup</code>
5	Rename plan built in memory	<code>plan_renames</code>
6	For each file: work copy → clean → publish → delete source → progress	<code>ImageWorker.run</code> loop
7	Temp backup discarded; final message; completed and finished signals	<code>_finish</code>
8	UI shows success/error, re-enables controls, offers retry	<code>_processing_finished</code> , <code>_worker_thread_finished</code>

PART 5

Reference

5.1 Index of all functions and methods

File	Name	One-line job
main.py	<code>_configure_logging</code>	Set up rotating file log, return path
main.py	<code>_handle_uncaught_exception</code>	Log crashes
main.py	<code>main</code>	Run the application
core/renamer.py	<code>plan_renames</code>	Create collision-free rename plan
core/renamer.py	<code>_base_stem</code>	Name part for one file
core/renamer.py	<code>_validate_prefix</code>	Clean and check prefix
core/renamer.py	<code>_image_timestamp</code>	EXIF or file date as text
core/backup_manager.py	<code>BackupManager.create_backup</code>	Verified ZIP or shadow backup
core/backup_manager.py	<code>BackupManager.rollback</code>	Restore originals
core/backup_manager.py	<code>BackupManager.discard_temporary_backup</code>	Delete shadow folder
core/backup_manager.py	<code>BackupManager._restore_from_bytes</code>	Atomic restore of one file
core/backup_manager.py	<code>_file_digest, _stream_digest</code>	SHA-256 helpers
core/metadata.py	<code>strip_metadata</code>	Validate and clean one image
core/metadata.py	<code>_strip_jpeg, _strip_png</code>	Copy stream without metadata
core/metadata.py	<code>_keep_png_chunk</code>	Keep/drop rule for PNG chunks
core/metadata.py	<code>MetadataStripResult.changed</code>	True if something was removed
core/audio.py	<code>AudioController.__init__</code>	Store paths and defaults
core/audio.py	<code>_ensure_mixer</code>	Lazy pygame init
core/audio.py	<code>play_startup</code>	Loop main theme (once)
core/audio.py	<code>play_state_cue</code>	WAV or generated beep
core/audio.py	<code>_play_generated_tone</code>	Sine-wave beep
core/audio.py	<code>set_volume, set_muted</code>	Volume control
core/audio.py	<code>_effective_volume, _apply_volume</code>	Compute and push volume
core/audio.py	<code>shutdown</code>	Release mixer
workers/image_worker.py	<code>ImageWorker.run</code>	In-place batch with rollback
workers/image_worker.py	<code>_run_zip_export</code>	ZIP batch
workers/image_worker.py	<code>_publish_without_overwrite</code>	Exclusive create + atomic replace
workers/image_worker.py	<code>_set_state, _finish</code>	Emit state / final result
ui/main_window.py	<code>RenamerWindow.__init__</code>	Build window, logging bridge

File	Name	One-line job
ui/main_window.py	<code>_configure_fonts</code>	Qt font folder
ui/main_window.py	<code>_build_interface</code> + 4 <code>_build_*</code> methods	Create widgets
ui/main_window.py	<code>_change_volume</code>	Slider handler
ui/main_window.py	<code>_folder_text_changed</code> , <code>_sync_folder_from_input</code>	Folder validation
ui/main_window.py	<code>_update_output_options</code>	In-place vs ZIP UI rules
ui/main_window.py	<code>play_startup_audio</code>	Start music
ui/main_window.py	<code>_set_visual_state</code>	Banner, picture, sound
ui/main_window.py	<code>_transition_image</code> , <code>_animate_image_opacity</code> , <code>_finish_image_fade</code>	Fade picture
ui/main_window.py	<code>_set_display_pixmap</code> , <code>_scale_display_pixmap</code> , <code>resizeEvent</code>	Scale picture
ui/main_window.py	<code>_choose_folder</code>	Folder picker
ui/main_window.py	<code>_start_processing</code> , <code>_retry_failed</code> , <code>_launch_processing</code>	Start jobs
ui/main_window.py	<code>_show_file_failure</code>	Log one failure
ui/main_window.py	<code>_cancel_processing</code>	Ask worker to stop
ui/main_window.py	<code>_set_controls_enabled</code>	Lock/unlock UI
ui/main_window.py	<code>_processing_finished</code> , <code>_worker_thread_finished</code>	End-of-job handlers
ui/main_window.py	<code>_is_worker_running</code>	Helper
ui/main_window.py	<code>closeEvent</code>	Safe close

5.2 All constants and magic numbers

Value	Where	Meaning
<code>2_000_000, 5</code>	main.py	Log file size (bytes) and number of backups
<code>3</code>	renamer.py <code>width</code>	Minimum digits in file numbers
<code>36867, 306, 36868</code>	renamer.py	EXIF tags DateTimeOriginal, DateTime, DateTimeDigitized
<code>"%Y:%m:%d %H:%M:%S"</code>	renamer.py	EXIF date format
<code>1024 * 1024</code>	backup_manager.py	1 MB hashing chunk
<code>0o700</code>	backup_manager.py	Backup folder only for owner
<code>64 * 1024</code>	metadata.py	PNG read buffer
<code>0xFFD8, 0xFFD9, 0xFFDA, 0xFFFE</code>	metadata.py	JPEG SOI, EOI, SOS, COM
<code>22_050, -16, 1</code>	audio.py	Mixer rate, 16-bit signed, mono
<code>7000, 0.12</code>	audio.py	Tone amplitude and length in seconds

Value	Where	Meaning
0.7	audio.py / window	Default volume 70%
0.6 / 90	image_worker.py	Compression level 6; ZIP loop progress share of 90%
0o600	image_worker.py	Placeholder file permissions
760×650, 1120×720, 210 ms	main_window.py	Min size, start size, fade time

5.3 Quirks and observations found while reading the code

These are not complaints: the project is a finished hobby tool. They are things a future contributor should know.

#	Observation	Effect
1	<code>errore.jpg</code> is listed first for the error state but does not exist.	Falls back to <code>error.jpg</code> ; harmless.
2	The success state is applied twice (worker signal and <code>_processing_finished</code>).	Success cue may play twice.
3	<code>_set_controls_enabled(True)</code> re-enables <code>prefix_edit</code> even when Prefix is not selected.	Invisible (the box is hidden); no effect.
4	A batch with at least one failed file ends with <code>completed(False, ...)</code> and error state although other files succeeded.	UI shows an error banner for partial success.
5	Retry numbering restarts at 1 and uses <code>_1</code> suffixes.	Safe but non-continuous numbers.
6	<code>pHYs</code> is stripped from PNGs.	DPI hint lost.
7	Cancellation is checked between files, not inside one file.	Latency up to one file.
8	<code>strip_metadata_enabled</code> is forced on for ZIP mode in the worker, and the UI also forces the toggle.	Double protection; consistent.
9	Rollback after a partial in-place run deletes all paths in <code>changed_paths</code> , including already-renamed files, then restores originals.	Correct and complete when the snapshot is valid.
10	Only the window object <code>RenamerWindow</code> is created; no <code>QApplication</code> guard for headless tests.	Unit-testing the UI needs Qt's offscreen platform.
11	The project has no automated tests; <code>.gitignore</code> mentions <code>.pytest_cache/</code> .	Core modules are pure Python and easy to test.

5.4 How to extend the project

Add a new rename style (for example “Prefix + date”)

- `core/renamer.py`: add the name to `RenameMode`, to the set in `plan_renames`, and a branch in `_base_stem`.
- `ui/main_window.py`: add a checkable button in `_build_control_panel` (inside the mode group) and set `mode` in `_launch_processing`.
- Add an example to the README and this guide.

Add metadata cleaning for WebP

- `core/metadata.py`: add a `_strip_webp` that walks RIFF chunks and drops EXIF and XMP . Call it from `strip_metadata` for `image_format == "WEBP"`.
- Update the README promise ("other formats left unchanged") and the UI tooltip.

Add another state or picture

- Add an entry in `STATE_PRESENTATION`, a colour in `state_colors`, a frequency in `_STATE_FREQUENCIES`, a cue file in `_STATE_CUES`, and the new key to `WorkerState`.

Write tests

- `plan_renames` with a temporary folder (collision cases).
- `strip_metadata` with tiny hand-made JPEG/PNG byte strings.
- `BackupManager` round trip: backup → modify → rollback → compare hashes.

5.5 Packaging for Windows

Two steps. **Step 1 — build the app folder with PyInstaller:**

```
python -m PyInstaller --clean --noconfirm --noconsole --onedir --name Renamer \
  --icon "assets\icon.ico" --add-data "assets;assets" --collect-all pygame main.py
```

Flag	Meaning
<code>--clean</code>	Remove old build caches first.
<code>--noconfirm</code>	Do not ask before overwriting <code>dist/</code> .
<code>--noconsole</code>	No black console window (so the log file is essential).
<code>--onedir</code>	Output a folder (<code>dist/Renamer</code>) rather than one big exe; faster start.
<code>--name Renamer</code>	Output name.
<code>--icon</code>	Exe icon.
<code>--add-data "assets;assets"</code>	Copy the assets folder (pictures and music) into the bundle. On Windows ; separates source and target.
<code>--collect-all pygame</code>	Bundle pygame's modules, native SDL/mixer libraries and data, because the lazy import is invisible to PyInstaller.

****Step 2 — compile `installer/Renamer.iss` with Inno Setup**** after checking that both the pygame package and the audio assets exist under `dist\Renamer\_internal`. The result is `dist\Renamer-Setup.exe`, published through GitHub Releases.

5.6 Quick troubleshooting for developers

Symptom	Likely cause	Fix
No sound in the packaged app	pygame or assets missing in the bundle	Rebuild with <code>--collect-all pygame</code> and <code>--add-data "assets;assets"</code>
Window opens without pictures	<code>assets</code> not found next to <code>ui/</code>	Keep the folder layout; check <code>_assets_directory</code>

Symptom	Likely cause	Fix
<code>ModuleNotFoundError: core</code>	Run from the wrong folder	Start with <code>python main.py</code> from the project root
Strange fonts on Linux/macOS	Windows font folder logic is skipped	Normal; Qt uses system fonts
Job aborted with <i>rollback failed</i>	Disk full or files locked	Open the log; restore manually from the ZIP in <code>.RENAMER_backups</code>

5.7 Credits and licence

Renamer is © 2026 Mohamed Yassin Khmiri and released under the **MIT License**. The cartoon theme is fan-made and non-commercial; the original characters belong to their respective owners and the project is not affiliated with or endorsed by them. This guide was written from the project's source code and documentation and describes the program as it exists in that snapshot.